

Operating Systems Interview Mastery

The Senior Software Engineer Track
Complete Master Edition — Modules 1–10

Targeting interviews at Google · Meta · Amazon · Microsoft · Apple · Uber · Netflix · Stripe · LinkedIn · Cloudflare

Each topic: why it's asked · internals · diagrams · production examples · trade-offs · traps · follow-ups · practice

Generated July 02, 2026

Contents

MODULE 1 — Process & Thread

MODULE 2 — CPU Scheduling

MODULE 3 — Synchronization (Highest Priority)

MODULE 4 — Deadlocks

MODULE 5 — Memory Management

MODULE 6 — Memory (Page) Replacement

MODULE 7 — File Systems

MODULE 8 — I/O

MODULE 9 — Linux Concepts

MODULE 10 — Production Interview Scenarios

MODULE 1 — Process & Thread

Senior SWE Interview Track — Operating Systems

1.1 Process vs Thread

1. Why Interviewers Ask This

It is the #1 OS screening question at Google, Meta, and Amazon because your answer reveals whether you understand isolation, memory layout, and the cost model of concurrency — the foundation for every follow-up on scaling backend services.

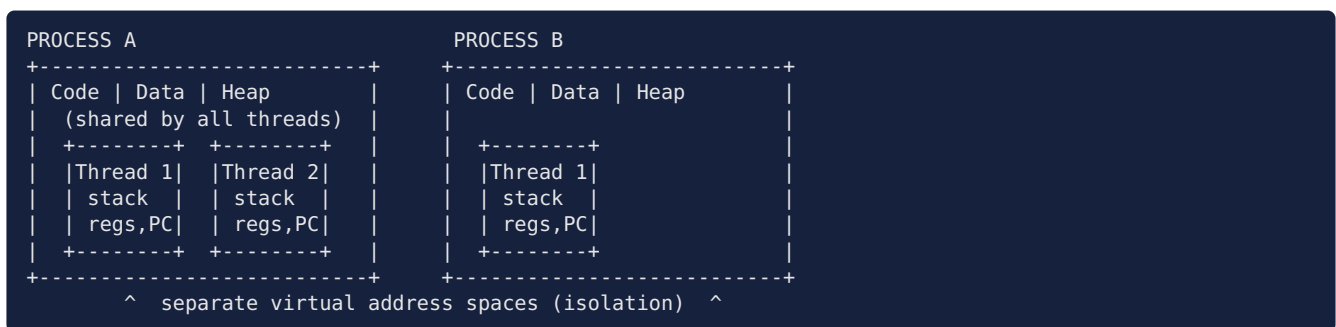
2. Core Concept

- A **process** is an instance of a running program: its own virtual address space, file descriptor table, credentials, and at least one thread.
- A **thread** is an execution context (registers + stack + program counter) *inside* a process. All threads in a process share the address space, heap, globals, and open file descriptors; each has its own stack and register state.
- Isolation boundary = process. Concurrency unit = thread.

3. Internal Working

- On Linux both are created via `clone()`. A process (`fork()`) gets a new `mm_struct` (address space); a thread (`pthread_create`) passes `CLONE_VM | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD`, so the kernel reuses the parent's address space and FD table.
- The kernel schedules **tasks** (`task_struct`) — it barely distinguishes process vs thread; the difference is what is *shared*.
- Crash semantics: a segfault in one thread kills the whole process (shared address space); a crashed child process leaves siblings alive.

4. ASCII Diagram



5. Real Production Example

- **Chrome**: one process per tab/site → a crashing tab or exploited renderer can't touch other tabs (isolation over memory cost).
- **Nginx**: multi-process workers (no shared-state locking bugs, per-core scaling).
- **JVM services (e.g., LinkedIn, Netflix backends)**: one process, hundreds of threads sharing heap — cheap data sharing, but must synchronize.

6. Advantages

- Threads: cheap creation, fast context switch (no address-space switch), zero-copy data sharing.
- Processes: fault isolation, security isolation, independent lifecycle, no data races by default.

7. Trade-offs

- Threads: one bad pointer corrupts everyone; locking complexity; harder debugging.
- Processes: IPC is slower (copies/syscalls), higher memory footprint, slower spawn.

8. Common Mistakes

- Saying “threads are always faster” — ignores lock contention and cache-line bouncing.
- Claiming threads share stacks (they don’t — stacks are per-thread).
- Forgetting FDs are shared across threads but *copied* across `fork()`.

9. Performance Implications

- Thread context switch $\approx 1\text{--}2\ \mu\text{s}$; process switch adds TLB flush cost (unless ASID/PCID tagged).
- Thread creation $\approx$ tens of μs ; process fork of large process is dominated by page-table copy (mitigated by COW).
- Shared heap = cache coherence traffic under contention (false sharing).

10. Common Interview Questions

- “What exactly is shared between threads and what is private?”
- “Why did Chrome pick processes per tab despite the memory cost?”
- “What happens to threads when a process forks?” (Only the calling thread exists in the child — classic trap.)

11. Follow-up Questions

- “How does the kernel represent threads vs processes?” $\rightarrow$ `task_struct` + shared `mm_struct`.
- “How would you share memory between processes?” $\rightarrow$ `shm/mmap(MAP_SHARED)`, discussed in Module 9.
- “What is a thread pool and why not spawn a thread per request?”

12. Coding/Debugging Scenarios

- Multithreaded server occasionally exits entirely: one worker thread segfaulted $\rightarrow$ whole process died. Fix: isolate risky work in child processes or harden the code.
- After `fork()` in a threaded program, child deadlocks: a lock was held by a thread that doesn’t exist in the child. Fix: `fork()` then immediately `exec()`, or `pthread_atfork`.

13. Best Practices

- Use processes for isolation boundaries (plugins, tenants, sandboxes); threads for parallelism within one trust domain.
- Prefer thread pools over unbounded thread creation.
- In threaded programs, only `async-signal-safe` calls between `fork()` and `exec()`.

14. Practice Questions

1. Design a browser architecture: what runs in which process and why?
2. A 16-core box runs a CPU-bound job — how many processes/threads and why?
3. Explain why `fork()` in a 200-thread JVM is dangerous.

1.2 Process Control Block (PCB)

1. Why Interviewers Ask This

Tests whether you know what the kernel must save/restore — the direct input to explaining context-switch cost.

2. Core Concept

The PCB is the kernel’s per-process data structure holding everything needed to stop and later resume a process. On Linux this is `task_struct`.

3. Internal Working

Key fields: - **Identification**: PID, PPID, UID/GID, process group, session. - **CPU state**: saved registers, program counter, stack pointer (in kernel stack / `pt_regs`). - **Scheduling**: state (running/ready/blocked), priority/nice, vruntime (CFS/EEVDF), CPU affinity. - **Memory**: pointer to `mm_struct` → page table root (CR3 value), VMA list. - **I/O & files**: FD table pointer, current working directory, root. - **Signals**: pending set, handler table. - **Accounting**: CPU time, limits (rlimits, cgroup membership). PCBs live on kernel run queues / wait queues; scheduler picks the next PCB and restores its state.

4. ASCII Diagram

```
task_struct (PCB)
+-----+
| pid, ppid, uid |
| state: RUNNING/READY/... |
| sched: prio, vruntime |
| *mm ---> page tables |
| *files -> fd table |
| signals: pending, handlers |
| CPU ctx: regs, PC, SP |
| accounting, cgroups |
+-----+
    linked into: run queue | wait queue | parent's child list
```

5. Real Production Example

`ps`, `top`, and `/proc/<pid>/*` are windows into PCBs. Kubernetes kills a container by signaling PIDs tracked via cgroups — all PCB-level metadata.

6. Advantages

Central, O(1)-accessible bookkeeping enables preemptive multitasking, fair scheduling, resource accounting, and clean teardown.

7. Trade-offs

Each task costs kernel memory (~few KB of `task_struct` + kernel stack, typically 8–16 KB) → hard limits on process/thread counts (`pid_max`, `threads-max`).

8. Common Mistakes

- Confusing the PCB with the user-space stack.
- Saying registers are stored “in the PCB” literally on Linux — they’re saved on the kernel stack (`pt_regs`), referenced from the task.
- Forgetting a zombie is exactly “a PCB kept around for the parent to read the exit status.”

9. Performance Implications

PCB size × task count bounds scalability (e.g., 100k threads ≈ ~1–2 GB of kernel stacks alone). Scheduler operations touch PCB fields — cache locality of run queues matters.

10. Common Interview Questions

- “What must be saved on a context switch?”
- “What remains after a process exits but before `wait()`?” (zombie = PCB shell)

11. Follow-up Questions

- “Where is the page-table pointer stored and when is it loaded?” (in `mm_struct`; loaded into CR3 on switch)
- “How does `top` know per-process CPU%?” (accounting fields exposed via `/proc`)

12. Coding/Debugging Scenarios

- `fork()` returns `EAGAIN` under load → hit `pid_max`/`threads-max` or cgroup pids limit; inspect `/proc/sys/kernel/pid_max`.

13. Best Practices

- Cap concurrency (pools) so task counts stay bounded; monitor `/proc` metrics rather than guessing.

14. Practice Questions

1. Walk through every PCB field the kernel touches when a blocked process becomes runnable and is scheduled.
2. Why do zombies consume almost no memory yet can still exhaust the system?

1.3 Thread Control Block (TCB)

1. Why Interviewers Ask This

Follow-up to PCB: do you understand *which* state is per-thread — the key to explaining why thread switches are cheaper.

2. Core Concept

The TCB stores per-thread state: thread ID, register set, program counter, stack pointer, scheduling state/priority, signal mask, and thread-local storage (TLS) pointer. Shared process resources (address space, FDs) are referenced, not owned.

3. Internal Working

- Linux: each thread is a `task_struct` sharing `mm`, `files`, `sighand` with siblings; `tgid` = process ID, `pid` = thread ID.
- User-level threading libraries (Go goroutines, Java virtual threads) keep their own tiny TCBs in user space and multiplex over kernel threads.
- TLS is reached via a dedicated register (`FS/GS` on x86-64) pointing at the thread's TLS block.

4. ASCII Diagram

```

Process (shared): mm_struct, fd table, signal handlers
      |
      +-----+
      | TCB #1 |      | TCB #2 |      | TCB #3 |
      +-----+      +-----+      +-----+
      | tid    |      | tid    |      | tid    |
      | regs/PC |      | regs/PC |      | regs/PC |
      | stack ptr |      | stack ptr |      | stack ptr |
      | sigmask |      | sigmask |      | sigmask |
      | TLS ptr  |      | TLS ptr  |      | TLS ptr  |
      +-----+      +-----+      +-----+
  
```

5. Real Production Example

Go runtime `g` structs (goroutine TCBs) are ~2–4 KB starting stacks → millions of goroutines per process, versus ~8 MB default pthread stacks.

6. Advantages

Minimal per-thread state → fast switch, cheap creation, natural data sharing.

7. Trade-offs

No isolation; per-thread stacks still consume virtual memory; signal handling across threads is subtle (handlers shared, masks per-thread).

8. Common Mistakes

- Assuming each thread has its own heap.
- Assuming signal *handlers* are per-thread (only the *mask* is).

9. Performance Implications

Default 8 MB stacks × 10k threads = 80 GB *virtual* (fine) but resident touch grows real memory; tune `pthread_attr_setstacksize` for massive thread counts.

10. Common Interview Questions

- “What’s in a TCB vs PCB?”
- “Why can Go run 1M goroutines but pthreads can’t do 1M threads comfortably?”

11. Follow-up Questions

- “How does `errno` work with threads?” (TLS variable)
- “Which thread receives a `SIGTERM` sent to the process?” (any thread with it unblocked)

12. Coding/Debugging Scenarios

- Stack overflow in a deep-recursion worker thread → crashes with `SIGSEGV` near guard page; fix by larger stack or iterative algorithm.

13. Best Practices

- Size stacks deliberately for high-thread-count services; use TLS instead of global mutable state.

14. Practice Questions

1. Compare memory cost of 100k pthreads vs 100k goroutines.
2. Explain how a user-level scheduler switches between two TCBs without entering the kernel.

1.4 User Threads vs Kernel Threads

1. Why Interviewers Ask This

Directly relevant to modern runtimes (Go, Java virtual threads, async runtimes). Interviewers use it to test whether you understand *who schedules what* and the blocking-syscall trap.

2. Core Concept

- **Kernel thread**: known to and scheduled by the kernel (1 `task_struct` each). Can run in parallel on multiple cores; blocking syscalls block only that thread.
- **User thread (green thread)**: scheduled by a user-space runtime; the kernel sees only the carrier kernel threads. Switches are pure user-space function calls.

3. Internal Working

- User-level switch: save callee-saved registers + SP, swap to another user stack — no syscall, ~tens of ns.
- Kernel-level switch: syscall/interrupt into kernel, scheduler runs, restores another task — ~1–2 μs.
- The classic failure: a user thread makes a blocking syscall → the carrier kernel thread blocks → *all* user threads on it stall. Runtimes fix this with non-blocking I/O + event loop (Go netpoller) or by handing off blocked carriers (Go `sysmon`, Java virtual thread unmount).

4. ASCII Diagram



5. Real Production Example

- Go: goroutines (M:N, GMP scheduler) — powers high-concurrency services at Uber, Cloudflare.
- Java 21 virtual threads: M:N mounted on carrier platform threads.
- Node.js: single kernel thread + event loop (cooperative, callback-based).

6. Advantages

- User threads: ~100× cheaper switch/creation, millions of concurrent tasks, custom scheduling.
- Kernel threads: true parallelism, kernel handles blocking, preemption for free.

7. Trade-offs

- User threads: blocking-syscall hazard, can't be preempted by kernel (runtime must insert preemption points), poor fit for CPU-bound work without M:N.
- Kernel threads: heavier memory and switch cost; ~10k practical ceiling for many workloads.

8. Common Mistakes

- “Green threads run in parallel” on a single carrier — they don't; only M:N with multiple carriers gives parallelism.
- Ignoring that file I/O on Linux is effectively always blocking (no epoll for regular files) — Go compensates with more carrier threads.

9. Performance Implications

M:N wins for I/O-bound high-concurrency (chat servers, proxies). 1:1 is simpler and fine up to thousands of threads. Context-switch overhead at 100k+ concurrent connections makes 1:1 infeasible → C10K/C10M problem.

10. Common Interview Questions

- “How does Go run a million goroutines on 8 cores?”
- “What happens when a goroutine calls a blocking syscall?”

11. Follow-up Questions

- “How does the runtime preempt a tight loop with no function calls?” (Go: async preemption via signals since 1.14)
- “Compare virtual threads to an async/await event loop.”

12. Coding/Debugging Scenarios

- Latency spikes in a Go service: goroutines pile up behind a blocking cgo/syscall hogging carrier threads → check `runtime` metrics, move blocking work to a bounded pool.

13. Best Practices

- I/O-bound massive concurrency → green/virtual threads or async I/O. CPU-bound → ~1 kernel thread per core.
- Never do blocking calls inside an event loop thread.

14. Practice Questions

1. Design the threading model for a websocket gateway with 2M idle connections.
2. Why does Java virtual threads' “pinning” (synchronized blocks) hurt, and how do you detect it?

1.5 Context Switching

1. Why Interviewers Ask This

Cost of context switching is the quantitative backbone of every “threads vs async” and “why is my p99 bad” discussion.

2. Core Concept

A context switch is the kernel saving the CPU state of the current task and restoring another's. Triggers: time-slice expiry, blocking (I/O, lock), higher-priority wakeup, explicit yield, interrupts.

3. Internal Working

1. Interrupt/syscall enters kernel mode.
2. Save current registers/PC/SP into the task's kernel stack (`pt_regs`).
3. Scheduler (`schedule()`) picks next task from run queue.
4. If a different process: load new page-table root (CR3 write → TLB implications; PCID mitigates full flush).
5. `switch_to`: swap kernel stacks and registers; return in the new task's context. Indirect cost dominates: TLB misses and cold CPU caches after the switch (can be tens of μs of degraded IPC).

4. ASCII Diagram

```
Task A running --> timer interrupt
| save A: regs, PC, SP -> A's PCB/kstack
| scheduler: pick B
| if B in other process: load B's CR3 (TLB!)
| restore B: regs, PC, SP <- B's PCB/kstack
Task B running
Direct cost ~1-2us; indirect (cache/TLB refill) often >> direct
```

5. Real Production Example

A trading system or Redis pins threads to cores (`taskset`, `isolcpus`) precisely to avoid switch + cache-pollution jitter. Overloaded API servers show high `cs` in `vmstat` and rising p99 while CPU% looks “fine”.

6. Advantages

Enables preemptive multitasking, fairness, and responsiveness on limited cores.

7. Trade-offs

Pure overhead — no user work done. Voluntary (blocking) switches are a signal of I/O-bound behavior; involuntary ones signal CPU contention.

8. Common Mistakes

- Quoting only the direct cost and ignoring cache/TLB pollution.
- Confusing a *mode* switch (user → kernel syscall, no task change) with a *context* switch.
- Thinking thread switches within a process flush the TLB (they don't — same address space).

9. Performance Implications

At 100k switches/sec × ~2 μs direct ≈ 20% of a core gone, before cache effects. `pidstat -w` splits voluntary vs involuntary; high involuntary = too many runnable threads per core.

10. Common Interview Questions

- “Walk me through exactly what happens during a context switch.”
- “Why is switching between threads of one process cheaper?”

11. Follow-up Questions

- “What is PCID/ASID and why does it matter?” (tagged TLB entries avoid full flush)
- “How does this cost drive the design of epoll-based servers?” (fewer threads → fewer switches)

12. Coding/Debugging Scenarios

- Service p99 degrades as thread pool grows from 200 → 2000 threads on 16 cores: involuntary switches skyrocket. Fix: bounded pool ≈ cores × (1 + wait/compute ratio), or async I/O.

13. Best Practices

- Keep runnable-threads $\approx$ core count; pin latency-critical threads; measure with `vmstat`, `pidstat -w`, `perf sched`.

14. Practice Questions

1. Estimate switch overhead for 50k RPS where every request blocks twice.
2. Explain why Meltdown mitigations (KPTI) made syscalls/context switches costlier.

1.6 Process States

1. Why Interviewers Ask This

State transitions explain every `top` output and every “process is stuck in D state” incident — a favorite senior debugging probe.

2. Core Concept

Canonical five-state model: **New** → **Ready** → **Running** → **Waiting(Blocked)** → **Terminated**, with Ready ↔ Running via scheduler and Running → Waiting on blocking. Linux states: **R** (running/runnable), **S** (interruptible sleep), **D** (uninterruptible sleep), **T** (stopped), **Z** (zombie).

3. Internal Working

- Ready: on a per-CPU run queue.
- Running: currently on a CPU.
- Waiting: on a wait queue tied to an event (I/O completion, lock, timer). **S** can be woken by signals; **D** cannot (usually mid-I/O in the kernel — unkillable).
- Zombie: exited; PCB retained until parent `wait()`s (Module 9).

4. ASCII Diagram

```

      admit      dispatch
NEW -----> READY <-----> RUNNING ----> TERMINATED (-> ZOMBIE until wait())
      ^         |         |         |
      |         |         |         |
      |         |         |         |
      +----- WAITING <-----+
              event completes
Linux: R = ready+running, S = interruptible wait,
      D = uninterruptible wait, T = stopped, Z = zombie

```

5. Real Production Example

NFS server outage → processes pile up in **D** state, load average climbs into the hundreds while CPU is idle (load counts R and D on Linux). `kill -9` does nothing — classic on-call scenario.

6. Advantages

Explicit states let the scheduler skip blocked tasks and let operators reason about system health.

7. Trade-offs

D state is required for data integrity mid-I/O but creates unkillable processes; `TASK_KILLABLE` is the kernel's compromise.

8. Common Mistakes

- Believing `kill -9` kills anything (“**D** state and zombies are immune — zombies are already dead”).
- Reading Linux load average as CPU-only.

9. Performance Implications

Many **D**-state tasks = storage/network-storage bottleneck, not CPU. Ready-queue length per core $\gg 1$ sustained = CPU saturation.

10. Common Interview Questions

- “Draw the process state diagram.”
- “Load average is 80 on an 8-core box but CPU is 10% — explain.” (D-state pileup)

11. Follow-up Questions

- “Difference between S and D sleep?” “Why can’t you kill a D-state process?”

12. Coding/Debugging Scenarios

- `ps aux | awk '$8 ~ /D/'` to find D-state tasks; `cat /proc/<pid>/stack` shows they’re stuck in a filesystem/driver path → fix the storage layer.

13. Best Practices

- Use timeouts on I/O (soft NFS mounts, socket timeouts) so tasks sleep interruptibly; alert on D-state counts.

14. Practice Questions

1. Trace all state transitions for one HTTP request handled by a blocking server.
2. Why does a zombie stay in **Z** even after `kill -9`?

1.7 Thread Lifecycle

1. Why Interviewers Ask This

Java/backend interviews love it (**NEW/RUNNABLE/BLOCKED/WAITING/TIMED_WAITING/TERMINATED**), and it maps to thread-dump reading — a real production skill.

2. Core Concept

A thread goes: created → runnable → running → (blocked on lock | waiting on condition | sleeping) → terminated → joined/detached. Java’s states in thread dumps: **BLOCKED** (waiting for a monitor), **WAITING** (`wait()`/`park()`), **TIMED_WAITING** (`sleep`, timed wait).

3. Internal Working

- `pthread_create` → `clone()` → new task in ready state.
- Blocking on a mutex parks the thread on a futex wait queue (kernel), state = sleeping.
- On exit, resources aren’t freed until `pthread_join` (or the thread is detached) — the thread equivalent of zombie.

4. ASCII Diagram

```

NEW --start()--> RUNNABLE <---scheduler--> RUNNING --run ends--> TERMINATED --join()
      ^         ^         ^
lock acquired --+ | +-- notify/signal | | +-- sleep(t)/timed wait
                  | |                 | | +----> WAITING (wait/park)
                  | |                 | | (contended lock)
                  +-----+ (TIMED_WAITING for sleep/timed ops)

```

5. Real Production Example

Diagnosing a frozen Java service: `jstack` dump shows 200 threads **BLOCKED** on one monitor held by a thread doing a slow DB call — lifecycle states are the vocabulary of that diagnosis.

6. Advantages

Well-defined states → thread dumps, profilers, and deadlock detectors work.

7. Trade-offs

Unjoined (non-detached) finished threads leak stack memory; too many threads in **RUNNABLE** = CPU thrash; in **BLOCKED** = contention.

8. Common Mistakes

- Confusing **BLOCKED** (lock) with **WAITING** (condition) in Java dumps.
- Forgetting to join/detach → resource leak.
- Assuming **RUNNABLE** means “on CPU” (it includes ready-and-waiting; and in Java, even threads in native I/O).

9. Performance Implications

Thread-dump state histograms are the fastest triage: mostly **BLOCKED** → lock contention; mostly **WAITING** on pool queue → underutilized or upstream-starved; mostly **RUNNABLE** → CPU-bound.

10. Common Interview Questions

- “Explain Java thread states and what each means in a thread dump.”
- “Difference between `sleep()` and `wait()`?” (wait releases the monitor; sleep doesn’t)

11. Follow-up Questions

- “What does `park()/unpark()` do vs `wait()/notify()`?”
- “How do you find which thread holds the lock others are blocked on?” (dump shows monitor owner)

12. Coding/Debugging Scenarios

- Take 3 thread dumps 10s apart; threads stuck on the same stack across dumps are truly stuck, not just sampled mid-work.

13. Best Practices

- Name threads; always join or detach; use pools with bounded queues; automate periodic thread dumps on latency alerts.

14. Practice Questions

1. Given a dump with 150 **BLOCKED** threads on `ConnectionPool`, walk through your diagnosis.
2. Map pthread lifecycle calls to Java `Thread` states.

1.8 Multithreading Models

1. Why Interviewers Ask This

Wrap-up question that checks you can connect user/kernel threads into the 1:1, N:1, M:N taxonomy and justify what real runtimes chose.

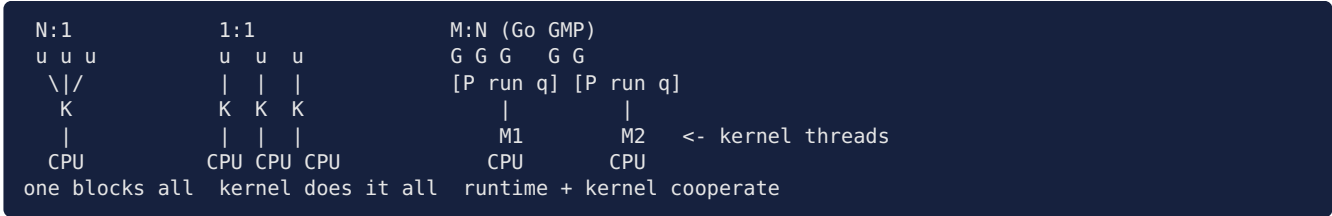
2. Core Concept

- **N:1 (many-to-one)**: all user threads on one kernel thread. Cheap, but no parallelism; one blocking call stalls all.
- **1:1 (one-to-one)**: every user thread is a kernel thread (Linux pthreads, Windows). Simple, parallel, but heavier.
- **M:N (many-to-many)**: runtime multiplexes M user threads over N kernel threads (Go GMP, Java virtual threads). Best of both, most complex.

3. Internal Working

Go's GMP: **G** (goroutine) queued on **P** (logical processor, = GOMAXPROCS) run queues, executed by **M** (kernel threads). Work stealing balances P queues; blocked M's are replaced so P's stay busy; netpoller converts socket I/O to readiness events instead of blocking M's.

4. ASCII Diagram



5. Real Production Example

- 1:1: C++/Rust services, classic Java platform threads.
- M:N: Go (Uber, Cloudflare, Kubernetes itself), Java 21+ virtual threads, Erlang/BEAM.
- N:1 pattern survives as single-threaded event loops (Node.js, Redis) — concurrency without parallelism.

6. Advantages

1:1 — simplicity + true parallelism. M:N — millions of tasks, cheap switches, still parallel.

7. Trade-offs

M:N needs a sophisticated runtime (preemption, syscall handoff, work stealing) and complicates debugging/FFI; 1:1 hits memory/switch limits at $\sim 10^4$ threads.

8. Common Mistakes

- Calling Node.js “M:N” (it’s an event loop on one thread + a small worker pool).
- Assuming M:N eliminates the need to think about blocking (cgo/FFI and file I/O still pin kernel threads).

9. Performance Implications

For 1M mostly-idle connections: 1:1 needs $\sim 1\text{M}$ kernel threads (infeasible); M:N or event loop handles it with $N \approx$ cores. CPU-bound work performs identically across models — cores are the limit.

10. Common Interview Questions

- “Compare 1:1 vs M:N; why did Linux pthreads choose 1:1?” (simplicity, kernel already fast)
- “Why did Java add virtual threads after 25 years of platform threads?”

11. Follow-up Questions

- “What is work stealing and why per-P local queues?” (cache locality, less lock contention)
- “How does GOMAXPROCS interact with container CPU limits?” (classic prod gotcha — set it to the cgroup quota)

12. Coding/Debugging Scenarios

- Go service in a 2-CPU-limit container defaults GOMAXPROCS to the host's 64 cores → heavy throttling; fix with `automaxprocs` or explicit setting.

13. Best Practices

- Choose the model per workload: event loop / M:N for massive I/O concurrency; 1:1 thread-per-core for CPU-bound; hybrid (pools) in between.

14. Practice Questions

1. Sketch how Go handles 100k goroutines each doing a blocking file read.
2. Argue for/against migrating a thread-per-request Java service to virtual threads.

Module 1 Cheat Sheet (one page)

Concept	One-liner	Key numbers/facts
Process	Isolation unit: own address space, FDs	fork = new <code>mm_struct</code> ; crash contained
Thread	Execution unit: own stack/regs, shared heap	segfault kills whole process
PCB	Kernel per-task record (<code>task_struct</code>)	PID, state, regs ref, <code>mm</code> , FDs, signals
TCB	Per-thread: tid, regs, stack, sigmask, TLS	pthread stack default ~8 MB
User vs kernel threads	Who schedules: runtime vs kernel	user switch ~ns, kernel ~1–2 μ s
Context switch	Save/restore task state	indirect cache/TLB cost dominates
Process states	R, S, D, T, Z on Linux	D = uninterruptible, unkillable; Z = dead PCB
Thread lifecycle	NEW $\rightarrow$ RUNNABLE $\rightarrow$ BLOCKED/ WAITING $\rightarrow$ TERMINATED	BLOCKED=lock, WAITING=condition
Models	N:1, 1:1 (pthreads), M:N (Go, vthreads)	M:N $\rightarrow$ millions of tasks

Rules of thumb: threads for sharing, processes for isolation; runnable threads $\approx$ cores; blocking syscalls are the enemy of green threads; load average counts D-state.

Top Interview Questions

1. Process vs thread — what's shared, what's private, crash semantics?
2. What happens during a context switch, and what's the real cost?
3. Why can Go run 1M goroutines but pthreads can't?
4. Load average 80, CPU 10% — diagnose. (D-state)
5. What happens when a multithreaded process forks?
6. Explain Java thread-dump states and how you'd triage a frozen service.
7. 1:1 vs M:N — trade-offs and real examples.

Common Mistakes (module-wide)

- “Threads are lightweight processes, end of answer” — no memory-layout detail.
- Ignoring indirect (cache/TLB) context-switch cost.
- kill -9 as a universal fix; misreading load average.
- Unbounded thread creation instead of pools.
- Forgetting fork+threads and GOMAXPROCS-in-containers traps.

Mock Interview (self-test, ~20 min)

1. (Warm-up) Chrome uses a process per tab. Defend and attack that decision.
2. (Depth) Walk through, register by register, what happens when the timer interrupt preempts thread T1 of process P1 and schedules thread T2 of process P2.
3. (Design) You must handle 500k concurrent mostly-idle websockets on 16 cores. Pick a threading model, justify memory and switch-cost math.
4. (Debugging) A Java API's p99 jumped 10 $\times$. A thread dump shows 300 threads **BLOCKED** on one monitor. What do you do next, step by step?

5. (Trap) After `fork()` your child process hangs inside `malloc`. Why?

MODULE 2 — CPU Scheduling

Senior SWE Interview Track — Operating Systems

2.0 The Metrics First (Waiting / Turnaround / Response Time)

1. Why Interviewers Ask This

Every scheduling question is scored with these three numbers. Interviewers watch whether you compute them correctly and, more importantly, whether you know which metric each algorithm optimizes.

2. Core Concept

For a process with **arrival time** A, **burst (service) time** B, **completion time** C, and **first-scheduled time** F: - **Turnaround time** = C - A (total time in system) — batch throughput metric. - **Waiting time** = Turnaround - B (time spent ready but not running). - **Response time** = F - A (time until *first* CPU) — interactivity metric. Also: **throughput** (jobs/sec) and **CPU utilization**.

3. Internal Working

The scheduler runs at every timer tick, on task wakeup, and on block/exit. Its policy decides who is dispatched next from the ready queue — the policy alone determines these metrics for a fixed workload.

4. ASCII Diagram

```

arrival A          first run F                      completion C
|----wait----|====run====|--wait--|===run===|
|<----- turnaround = C - A ----->|
response = F - A      waiting = turnaround - burst

```

5. Real Production Example

Latency SLOs are response-time thinking (p99 time-to-first-byte); batch ETL pipelines optimize turnaround/throughput. A queueing web server *is* a scheduling system: request queue = ready queue.

6–9. Advantages / Trade-offs / Mistakes / Performance

- No scheduler optimizes all three at once: SJF minimizes average waiting; RR minimizes response variance at the cost of turnaround (switch overhead).
- Mistake: computing waiting time as “time before first run” — it includes *all* ready-but-not-running gaps under preemptive schedulers.
- Mistake: forgetting context-switch overhead when asked “what’s the ideal quantum”.

10–14. Interview Prep

- Q: “Given this table of arrivals/bursts, compute avg waiting time under FCFS and SJF.” (Practice by hand — asked verbatim at Amazon/Microsoft.)
- Follow-up: “Which metric matters for a web server vs a nightly batch job?”
- Practice: build a 4-process table and compute all three metrics under FCFS, SJF, RR(q=2).

2.1 FCFS (First Come, First Served)

1. Why Interviewers Ask This

Baseline algorithm — used to test if you can spot the **convoy effect** and reason about why simple FIFO fails for latency.

2. Core Concept

Non-preemptive FIFO: run each job to completion in arrival order.

3. Internal Working

Single FIFO ready queue; dispatch head; no preemption; only blocking/exit yields the CPU.

4. ASCII Diagram

```
Arrivals: P1(burst 24), P2(3), P3(3) all at t=0
Gantt: |-----P1(24)-----|P2(3)|P3(3)|
Wait: P1=0, P2=24, P3=27 -> avg 17
Reversed order: |P2|P3|----P1----| -> avg wait (0+3+6)/3 = 3
CONVOY EFFECT: one long job makes everyone wait.
```

5. Real Production Example

A single-threaded worker consuming a FIFO queue (e.g., one Kafka partition consumer): one 30-second job starves hundreds of 10 ms jobs behind it — the convoy effect in service clothing (head-of-line blocking).

6. Advantages

Trivial, no starvation (everyone eventually runs), minimal overhead, fair in arrival order.

7. Trade-offs

Terrible average waiting time with mixed burst lengths; awful response time; convoy effect.

8. Common Mistakes

- Not naming the convoy effect.
- Saying FCFS starves jobs (it doesn't — that's priority/SJF).

9. Performance Implications

Average wait is order-dependent: schedule-short-first can cut it by an order of magnitude — the motivation for SJF.

10–11. Interview & Follow-ups

- “Compute avg wait for these arrivals.” “What is the convoy effect and where do you see it in real systems?” (HOL blocking in queues, TCP, HTTP/1.1 pipelining)

12. Coding/Debugging Scenario

Job queue p99 explodes when a “reindex-everything” task lands: split queues by expected duration or add preemption/chunking.

13. Best Practices

Separate fast/slow lanes; chunk long jobs; FCFS only where fairness-by-arrival is the actual requirement.

14. Practice Questions

1. P1..P4 arrive at 0,1,2,3 with bursts 8,4,9,5 — Gantt chart + all metrics.
2. Give two production systems where FCFS-style HOL blocking bit you and the fix.

2.2 SJF (Shortest Job First)

1. Why Interviewers Ask This

It's *provably optimal* for average waiting time — interviewers test whether you know that, and whether you spot that it's unimplementable without predicting the future.

2. Core Concept

Non-preemptive: among ready jobs, run the one with the smallest total burst. Optimal average waiting time (exchange argument: any long-before-short pair can be swapped to reduce total wait).

3. Internal Working

Ready queue = min-heap keyed by burst estimate. Real systems can't know bursts → estimate with exponential averaging: $\tau(n+1) = \alpha \cdot t(n) + (1-\alpha) \cdot \tau(n)$.

4. ASCII Diagram

```
t=0: P1(6), P2(8), P3(7), P4(3) all ready
Gantt: |P4(3)|P1(6)|P3(7)|P2(8)|
Wait: P4=0, P1=3, P3=9, P2=16 -> avg 7 (FCFS order would give 10.25)
Risk: continuous stream of short jobs -> P2 starves.
```

5. Real Production Example

Web servers doing shortest-expected-processing-first on request queues; OS I/O schedulers favoring small reads; cluster schedulers running “small jobs fast lane” (Hadoop fair-scheduler small-job preference).

6. Advantages

Minimal average waiting/turnaround time — mathematically optimal (non-preemptive, all-at-once arrivals).

7. Trade-offs

Needs burst prediction; **starvation of long jobs** under a stream of short ones; non-preemptive → bad response time when a long job is mid-run.

8. Common Mistakes

- Not mentioning starvation + aging as the fix.
- Claiming optimality without the caveat (preemption/arrivals change the picture → SRTF).

9. Performance Implications

Prediction error degrades toward FCFS behavior; heavy-tailed job sizes (common in practice) make short-first policies hugely beneficial.

10–11. Interview & Follow-ups

- “Prove/argue SJF optimality.” “How would you estimate burst length?” “How do you prevent starvation?” (aging)

12. Coding/Debugging Scenario

Implement a job runner with an exponential-average duration predictor per job type; measure avg wait vs FIFO on a heavy-tailed workload.

13. Best Practices

Use size-based scheduling where job sizes are known/predictable; always add aging or a max-wait bound.

14. Practice Questions

1. Same 4-process table: compute SJF vs FCFS average waiting time.
2. $\alpha=0.5$, $\tau_0=10$, actual bursts 6,4,6 — compute successive predictions.

2.3 SRTF (Shortest Remaining Time First)

1. Why Interviewers Ask This

The preemptive twist on SJF — tests whether you understand preemption mechanics and can re-compute schedules when arrivals interleave.

2. Core Concept

Preemptive SJF: whenever a new job arrives with remaining time shorter than the running job's remaining time, preempt. Optimal average waiting time among *all* algorithms (with known bursts).

3. Internal Working

On every arrival/completion, compare remaining times; min-heap keyed on remaining time; preemption triggers a context switch.

4. ASCII Diagram

```
P1 arr 0 burst 8 | P2 arr 1 burst 4 | P3 arr 2 burst 9 | P4 arr 3 burst 5
t: 0    1        5        10       17       26
   |P1--|P2(4)---|P4(5)|P1(rem7)|P3(9)-----|
t=1: P2(4) < P1 rem(7) -> preempt P1.
Avg wait = ((10-1-... )) : P1=9, P2=0, P3=15, P4=2 -> 6.5
```

5. Real Production Example

The idea shows up as “preempt long batch work for interactive traffic”: e.g., Linux CFS effectively favors tasks that have consumed little CPU; big-data schedulers preempt long tasks for short ad-hoc queries.

6. Advantages

Best possible average waiting/turnaround; short jobs get near-instant service.

7. Trade-offs

More context switches; worse starvation for long jobs than SJF; requires continuous remaining-time knowledge (unrealistic).

8. Common Mistakes

- Forgetting to re-evaluate at *every* arrival in hand-simulations.
- Ignoring context-switch cost when comparing to SJF.

9. Performance Implications

In heavy-tailed workloads SRTF-like policies dramatically cut mean latency, at the price of tail latency for the biggest jobs — a mean-vs-p99 trade-off interviewers love.

10–11. Interview & Follow-ups

- “Simulate this arrival table under SRTF.” “When does SRTF equal SJF?” (no arrivals mid-run) “Mean vs p99 — who wins, who loses?”

12. Coding/Debugging Scenario

Write a discrete-event simulator: jobs with Pareto-distributed sizes; compare FCFS/SJF/SRTF mean and p99 turnaround.

13. Best Practices

Use SRTF-flavored policies only with aging/priority floors so giant jobs still finish.

14. Practice Questions

1. Recompute the diagram above if P3's burst were 2.
2. Explain why SRTF minimizes mean waiting time (exchange argument with remaining times).

2.4 Priority Scheduling

1. Why Interviewers Ask This

It's how real OSes and job systems actually work, and it sets up the two must-know failure modes: **starvation** and **priority inversion**.

2. Core Concept

Each task has a priority; scheduler runs the highest-priority ready task. Variants: preemptive vs non-preemptive; static vs dynamic priorities. SJF is priority scheduling with priority = 1/burst.

3. Internal Working

- Linux: nice -20..+19 for normal tasks (CFS/EEVDF weights the *share*, not absolute order), and real-time classes `SCHED_FIFO/SCHED_RR` (priorities 1–99) that strictly preempt normal tasks.
- Ready structure: per-priority queues or a weighted virtual-runtime tree.
- **Priority inversion**: low-prio task holds a lock the high-prio task needs, while medium-prio tasks run — fixed by **priority inheritance** (lock holder temporarily inherits waiter's priority) or priority ceiling.

4. ASCII Diagram

```
Priority inversion:
H (high) --- blocked on lock L
M (medium) - RUNNING (preempts Lo)
Lo (low) --- holds lock L, never scheduled -> H waits on M!
Fix: priority inheritance -> Lo temporarily runs at H's priority.
```

5. Real Production Example

- **Mars Pathfinder (1997)**: system resets from priority inversion; fixed by enabling priority inheritance remotely — the canonical interview story.
- Kubernetes pod priority/preemption; kernel threads for interrupts at RT priority; DB systems boosting lock-holding transactions.

6. Advantages

Expresses business importance; low latency for critical work; flexible (dynamic priorities).

7. Trade-offs

Starvation of low priority; priority inversion; priority assignment becomes a config-management problem ("priority inflation" — everything ends up critical).

8. Common Mistakes

- Not knowing priority inversion or its fixes — an instant senior-level red flag.
- Assuming nice values strictly order execution on Linux (they weight CPU shares).

9. Performance Implications

A runaway `SCHED_FIFO` task can freeze a CPU (no time slicing within RT unless RR); kernel throttles RT to 95% by default (`sched_rt_runtime_us`).

10–11. Interview & Follow-ups

- "What is priority inversion? How do priority inheritance and priority ceiling differ?"

- “How does Linux nice actually affect scheduling?” “How do you stop starvation?” (aging → next section)

12. Coding/Debugging Scenario

An RT audio thread stutters: a low-prio thread holds a shared mutex. Fix:

```
pthread_mutexattr_setprotocol(PTHREAD_PRIO_INHERIT) .
```

13. Best Practices

Few, well-defined priority tiers; PI mutexes when RT and normal threads share locks; never busy-loop in `SCHED_FIFO`.

14. Practice Questions

1. Tell the Mars Pathfinder story: bug, mechanism, fix.
2. Design priority tiers for a ride-hailing dispatch service (matching vs analytics vs logging).

2.5 Round Robin (RR)

1. Why Interviewers Ask This

The canonical preemptive algorithm — tests quantum-size reasoning: the tension between responsiveness and context-switch overhead.

2. Core Concept

FIFO queue + fixed time quantum q ; a task runs at most q , then is preempted to the tail. Every task gets $1/n$ of the CPU within $(n-1) \cdot q$ time — bounded response time, no starvation.

3. Internal Working

Timer interrupt at quantum expiry → preempt → enqueue at tail → dispatch head. Tasks that block before quantum end simply leave the queue (interactive tasks are naturally favored).

4. ASCII Diagram

```
q=4: P1(24), P2(3), P3(3)
|P1:4|P2:3|P3:3|P1:4|P1:4|P1:4|P1:4|P1:1|
Response: P1=0, P2=4, P3=7 (vs FCFS: 0,24,27)
q too big -> degrades to FCFS
q too small-> switch overhead dominates (e.g., q=2*switch cost -> 33% waste)
```

5. Real Production Example

- `SCHED_RR` for equal-priority real-time tasks; time-slicing in hypervisors (vCPU scheduling); token-bucket-style fair sharing among tenants; classic timesharing.
- CFS is *not* RR but inherits the “everyone gets a bounded slice” goal (targeted latency / sched_latency window).

6. Advantages

Starvation-free; predictable, bounded response time; great for interactive fairness.

7. Trade-offs

Higher turnaround for long jobs than FCFS; quantum tuning; more switches → cache pollution.

8. Common Mistakes

- No rule for choosing q . Rule of thumb: $q \gg$ context-switch cost (e.g., 100×), and sized so ~80% of interactive bursts finish within one quantum. Typical: 1–100 ms.
- Forgetting arrival-vs-requeue ordering in hand simulations (new arrival enqueues before the preempted task, in most conventions — state your convention).

9. Performance Implications

Overhead fraction $\approx \text{switch_cost} / (q + \text{switch_cost})$. $q=10\text{ ms}$, $\text{switch}=10\text{ }\mu\text{s} \rightarrow \sim 0.1\%$ overhead; $q=100\text{ }\mu\text{s} \rightarrow \sim 9\%$.

10–11. Interview & Follow-ups

- “What happens as $q \rightarrow \infty$ and $q \rightarrow 0$?” (FCFS; processor sharing with infinite overhead)
- “Simulate RR($q=2$) for this table.” “How would you pick q for a mixed workload?”

12. Coding/Debugging Scenario

Multi-tenant worker uses RR over tenant queues; one tenant’s tasks are $100\times$ longer $\rightarrow$ apply per-quantum chunking (cooperative yield every N items).

13. Best Practices

Quantum $\geq 100\times$ switch cost; combine RR with priorities (RR within each priority level — that’s `SCHED_RR`).

14. Practice Questions

1. Bursts 5,15,4,3 at $t=0$, $q=4$ — Gantt + avg response and turnaround.
2. Derive the overhead formula and find q for $<1\%$ overhead at $5\text{ }\mu\text{s}$ switch cost.

2.6 Multilevel Queue (MLQ)

1. Why Interviewers Ask This

Tests whether you can combine algorithms into a policy: different classes of work deserve different scheduling.

2. Core Concept

Partition the ready queue into fixed queues by task class (e.g., system > interactive > batch), each with its own algorithm (RR for interactive, FCFS for batch). Between queues: strict priority or weighted CPU shares. Tasks are **permanently assigned** to a queue.

3. Internal Working

Scheduler serves the highest non-empty queue (strict) or divides bandwidth (e.g., 80/20). Strict priority $\rightarrow$ lower queues starve when upper queues stay busy.

4. ASCII Diagram

```
Q0 System/RT      [RR]      highest  --served first
Q1 Interactive    [RR q=8ms]
Q2 Batch          [FCFS]     lowest   --runs only when Q0,Q1 empty
Task class fixed at creation -> misclassification is permanent.
```

5. Real Production Example

Linux scheduling classes are exactly this: `stop` > `deadline` > `rt` (FIFO/RR) > `fair` (CFS/EEVDF) > `idle`.
Kubernetes QoS classes (Guaranteed/Burstable/BestEffort) are MLQ thinking at the cluster level.

6. Advantages

Simple; strong guarantees for the top class; per-class tailored policy.

7. Trade-offs

Rigid — no migration between queues; starvation of low queues; misclassification is unfixable at runtime.

8. Common Mistakes

- Confusing MLQ with MLFQ (the *feedback*/migration is the whole difference).
- Not mentioning starvation under strict inter-queue priority.

9. Performance Implications

Strict-priority MLQ gives excellent latency for the top tier and unbounded tail latency for the bottom tier — say this explicitly.

10–11. Interview & Follow-ups

- “MLQ vs MLFQ?” “How does Linux layer its scheduling classes?” “Fixed priority between queues vs time-slicing between queues?”

12. Coding/Debugging Scenario

Batch jobs never run during business hours in a strict-MLQ job system → add a bandwidth share (e.g., batch guaranteed 10%).

13. Best Practices

Give every queue a minimum bandwidth; keep classes few and observable.

14. Practice Questions

1. Design MLQ tiers for a database server (replication, queries, vacuum/compaction).
2. When is strict priority between queues actually correct? (hard real-time)

2.7 Multilevel Feedback Queue (MLFQ)

1. Why Interviewers Ask This

The most-asked “design a scheduler” answer: it *learns* job behavior without prior knowledge — and it lets interviewers probe starvation, aging, and gaming.

2. Core Concept

Multiple queues by priority; **tasks move between queues based on observed behavior**: use your full quantum (CPU-bound) → demoted; block quickly (interactive/I/O-bound) → stay high or get promoted. Approximates SRTF with zero foreknowledge.

3. Internal Working

Classic rules: 1. Higher queue runs first; RR within a queue. 2. New tasks enter the top queue. 3. Use entire quantum (cumulative, to prevent gaming) → drop one level; longer quanta at lower levels. 4. **Aging/priority boost**: periodically move everyone to the top queue → prevents starvation and adapts to phase changes.

4. ASCII Diagram

```

Q0 (q=8ms)  [I/O-bound stay here] <---- periodic boost (aging)
  | used full quantum                ^
  v                                |
Q1 (q=16ms)  | used full quantum      |
  | used full quantum              |
  v                                |
Q2 (q=64ms, FCFS-ish) [CPU hogs end here] --+

```

5. Real Production Example

Windows scheduler and Solaris TS class are MLFQ-flavored (priority decay + boosts, e.g., foreground-window and I/O-completion boosts). Historical BSD/ULE similarly. Linux CFS took a different route (fair virtual runtime) but achieves the same “interactive tasks feel snappy” goal — a great compare-and-contrast to volunteer.

6. Advantages

No burst prediction needed; excellent interactive latency; adapts to behavior changes; starvation-free *with* boosting.

7. Trade-offs

Many knobs (levels, quanta, boost period); **gameable** (yield just before quantum end to stay high — fixed by charging cumulative CPU time); boosts cause periodic latency blips for the top queue.

8. Common Mistakes

- Omitting the priority boost → your design starves batch jobs; interviewers *will* ask.
- Not addressing gaming.

9. Performance Implications

Approximates SRTF's mean-latency win for heavy-tailed workloads; the boost period trades starvation bound vs interactive purity.

10–11. Interview & Follow-ups

- “Design a scheduler that needs no knowledge of job lengths.” (this)
- “How can a process game MLFQ, and how do you fix it?” “What does the boost period control?”
- “Compare MLFQ with CFS's virtual-runtime approach.”

12. Coding/Debugging Scenario

Implement MLFQ in a request-processing framework: requests demoted after N ms of service; measure short-request p50 improvement vs FIFO.

13. Best Practices

Charge cumulative CPU; boost on a period $\approx$ seconds; expose queue-level metrics.

14. Practice Questions

1. Simulate: A(CPU-bound, 200ms) and B(1ms CPU + 10ms I/O loop) through a 3-level MLFQ.
2. Choose quanta and boost period for a mixed API+batch worker; justify.

2.8 Starvation & Aging

1. Why Interviewers Ask This

The standard follow-up to *every* scheduling answer: “what breaks, and how do you fix it?”

2. Core Concept

Starvation: a runnable task waits unboundedly because the policy always prefers others (SJF/SRTF long jobs, low priorities, readers-vs-writers). **Aging**: gradually increase the priority of waiting tasks so waiting time itself becomes priority — converts unbounded starvation into a bounded delay.

3. Internal Working

- Implementations: priority $\pm$ f(wait time) recomputed on a timer; MLFQ boost; CFS-style vruntime (waiting tasks accumulate less vruntime → become “owed” CPU — aging built into the algorithm).
- Same idea beyond CPU: lock fairness (ticket locks), DB lock queues (FIFO grant), network schedulers (deficit round robin).

4. ASCII Diagram

```

prio
^
|      aging:  $p(t) = p_0 + k \cdot \text{wait}$ 
|      /----- low-prio task finally exceeds
|      -----/      competing tasks and runs
|_____/
+-----> wait time
Starvation = wait unbounded; aging bounds it.

```

5. Real Production Example

- Message consumers where “hot” partitions always win → cold partitions starve; fix with deficit-weighted polling.
- Writer starvation with reader-preferring RW locks (Module 3).
- Linux RT throttling (95%) = built-in anti-starvation for normal tasks under runaway RT threads.

6–7. Advantages / Trade-offs

Aging guarantees progress; but weakens the intended priority discipline and adds tuning (rate k). Too-fast aging $\approx$ FIFO; too-slow $\approx$ starvation anyway.

8. Common Mistakes

- Confusing starvation with deadlock (starved task *can* run if given CPU; deadlocked can’t ever).
- Proposing aging without a concrete mechanism.

9. Performance Implications

Anti-starvation mechanisms cost tail latency of top-priority work — quantify it (boost period, aging rate).

10–11. Interview & Follow-ups

- “Which of the algorithms we discussed can starve, and why?” (SJF, SRTF, strict priority, strict MLQ; not FCFS/RR)
- “Starvation vs deadlock vs livelock?”

12. Coding/Debugging Scenario

Low-priority reconciliation loop never runs during traffic peaks → add a minimum-share guarantee or aging to the internal scheduler.

13. Best Practices

Every prioritized system needs an explicit starvation story: aging, minimum shares, or FIFO fallback; monitor max wait per class.

14. Practice Questions

1. Identify starvation risk in: SJF, RR, strict MLQ, reader-preferring RW lock — and the fix for each.
2. Design aging for a support-ticket queue with P0–P3 severities.

Module 2 Cheat Sheet (one page)

Algorithm	Preemptive?	Optimizes	Starvation?	Killer fact
FCFS	No	Simplicity	No	Convoy effect
SJF	No	Avg waiting (optimal)	Yes (long jobs)	Needs burst prediction (exp. averaging)
SRTF	Yes	Avg waiting (optimal overall)	Yes (worse)	Re-evaluate at every arrival
Priority	Either	Business importance	Yes	Priority inversion → inheritance (Mars Pathfinder)
RR	Yes	Response time	No	$q \gg$ switch cost; $q \rightarrow \infty =$ FCFS
MLQ	Yes	Per-class policy	Yes (low queues)	Fixed assignment; Linux sched classes
MLFQ	Yes	SRTF w/o foreknowledge	No (with boost)	Demote CPU hogs; periodic boost; anti-gaming

Metrics: turnaround = $C - A$; waiting = turnaround – burst; response = first-run – A. **Formulas:** RR overhead $\approx s/(q+s)$; exp. average $\tau' = \alpha t + (1-\alpha)\tau$. **Linux reality:** classes stop>deadline>rt>fair(CFS/EEVDF)>idle; nice = weight not order; RT throttled at 95%.

Top Interview Questions

1. Compute waiting/turnaround for a table under FCFS, SJF, SRTF, RR (do these by hand until fast).
2. What is the convoy effect? Where does it appear outside CPU scheduling?
3. Priority inversion: mechanism, Mars Pathfinder, priority inheritance vs ceiling.
4. Design a scheduler with unknown job lengths → MLFQ, with gaming + starvation defenses.
5. How does Linux actually schedule (CFS/EEVDF, nice weights, RT classes)?
6. How do you pick an RR quantum?
7. Starvation vs deadlock vs livelock.

Common Mistakes (module-wide)

- Metric arithmetic slips (waiting vs response); not stating tie-break conventions.
- SJF/SRTF answers without starvation+aging; MLFQ without boost or anti-gaming.
- Ignoring context-switch overhead; treating nice as strict ordering.
- Confusing MLQ (fixed) with MLFQ (feedback).

Mock Interview (self-test, ~20 min)

1. (Compute) P1..P4 arrive 0,2,4,5, bursts 7,4,1,4: Gantt + avg waiting under SRTF and RR($q=2$).
2. (Design) Build the scheduler for a multi-tenant CI system: interactive PR builds + nightly full builds. Policy, starvation defense, metrics.
3. (Depth) Your MLFQ demotes a video-encoding job to the bottom queue, but it's the paying customer's job. Reconcile priorities with feedback scheduling.
4. (Trap) "SJF is optimal, so why doesn't Linux use it?" (prediction, starvation, interactivity, fairness)
5. (Prod) On a pinned 4-core VM, one **SCHED_FIFO** thread spins at 100%. What happens to normal tasks, and what saves them?

MODULE 3 — Synchronization (Highest Priority)

Senior SWE Interview Track — Operating Systems

3.1 Race Condition

1. Why Interviewers Ask This

The single most common OS/concurrency screen. Interviewers check that you can explain *why* `count++` is broken at the instruction level, not just recite a definition.

2. Core Concept

A race condition occurs when the result depends on the uncontrolled interleaving of concurrent accesses to shared state, with at least one write. A **data race** is the narrower memory-model term: two unsynchronized accesses, same location, one is a write — undefined behavior in C/C++/Go.

3. Internal Working

`count++` compiles to load → add → store. Two threads can both load the same value, both add, both store → one increment lost. Beyond interleaving: compilers reorder/cache values in registers, and CPUs reorder memory operations (store buffers) — so races break even on a single “logical” interleaving model. Correctness requires mutual exclusion or atomic read-modify-write plus proper memory ordering.

4. ASCII Diagram

```
count = 5
T1: load r=5      |
                  | T2: load r=5
T1: r=6, store 6  |
                  | T2: r=6, store 6
Result: 6 (should be 7) -> LOST UPDATE

Also: check-then-act race (TOCTOU):
T1: if (map.get(k)==null)  T2: if (map.get(k)==null)
T1: map.put(k, v1)         T2: map.put(k, v2)  -> both "won"
```

5. Real Production Example

- Double-spend / double-booking bugs: two requests read balance=100, both withdraw 100.
- Therac-25 radiation machine (race → lethal overdoses) — canonical story.
- TOCTOU security bugs: `access()` then `open()` on a path an attacker swaps in between.

6. Advantages

(N/A — a defect.) The relevant “advantage” is knowing benign-looking patterns that are actually racy: lazy init, check-then-act, read-modify-write.

7. Trade-offs

Fixes trade throughput for correctness: coarse locks (simple, contended), fine-grained locks (fast, deadlock-prone), atomics (fast, hard), immutability/message passing (clean, allocation cost).

8. Common Mistakes

- “It works in my tests” — races are probabilistic and load-dependent.
- Believing `volatile` (Java) fixes read-modify-write (it fixes visibility, not atomicity).
- Fixing the write but leaving unsynchronized reads (still a data race; may see torn/stale values).

9. Performance Implications

Races themselves are “fast”; every fix costs: uncontended atomic ~10–20 ns wasn’t free, contended cache-line ping-pong can be 100×. Design to *reduce sharing* first.

10. Common Interview Questions

- “Why is `i++` not thread-safe? Show the interleaving.”
- “Race condition vs data race?” “What is check-then-act / TOCTOU?”

11. Follow-up Questions

- “How would you find races in a large codebase?” (TSan, Go `-race`, code review of shared mutable state)
- “Can a race exist even if every individual operation is atomic?” (yes — atomicity of the *composite* is what matters: check-then-act)

12. Coding/Debugging Scenarios

- Counter off by ~0.1% only under load → lost updates; fix with atomic increment.
- Singleton occasionally constructed twice → unsynchronized lazy init; fix with once-initialization (`std::call_once`, `sync.Once`, holder idiom).

13. Best Practices

Minimize shared mutable state; prefer immutability and message passing; make composite operations atomic (one lock per invariant); run TSan/ `-race` in CI.

14. Practice Questions

1. Find the race: `if (!cache.contains(k)) cache.put(k, compute(k));`
2. Two services decrement inventory in a DB without transactions — describe the failure and three fixes (transaction, atomic UPDATE, optimistic version check).

3.2 Critical Section

1. Why Interviewers Ask This

Frames every locking discussion: interviewers want the three correctness requirements and whether you keep critical sections *small*.

2. Core Concept

A critical section is code that accesses shared state and must not be executed by more than one thread at a time. A correct solution needs: 1. **Mutual exclusion** — at most one thread inside. 2. **Progress** — if empty, someone waiting can enter (no needless blocking). 3. **Bounded waiting** — no thread waits forever (fairness/starvation bound).

3. Internal Working

Enforced by hardware primitives (atomic RMW: **LOCK**-prefixed instructions, LL/SC) wrapped by locks. Entry/exit protocol: acquire → critical section → release. Historical software-only solutions (Peterson’s algorithm) matter only to show why hardware atomics are needed on modern out-of-order CPUs (they break without memory barriers).

4. ASCII Diagram

```
T1: ---[acquire]===CS===[release]-----
T2: -----[acquire: BLOCKED].....[acquire]===CS===[release]--
Rule of thumb: shrink === (the CS) — do I/O, allocation,
logging, RPC *outside* the lock.
```

5. Real Production Example

Checkout flow: `reserve inventory + create order` is one logical critical section — implemented as a DB transaction (locks) or a distributed lock. The Big Kernel Lock removal from Linux is the classic “coarse CS → fine CS” scalability story.

6. Advantages

Restores sequential reasoning: invariants hold at CS boundaries.

7. Trade-offs

Serialization: by Amdahl’s law, a CS that is 5% of execution caps speedup at 20× regardless of cores.

8. Common Mistakes

- Doing blocking I/O or RPC inside a lock (an interview red flag and a prod outage pattern).
- Protecting *code* instead of *data* — the lock must be associated with the invariant/data it guards.
- Two code paths touching the same data, only one takes the lock.

9. Performance Implications

Lock hold time × acquisition rate = utilization of the “lock server”. Hold 1 ms at 2000 acq/s → the lock is 200% utilized → unbounded queueing. Measure hold times.

10. Common Interview Questions

- “What are the three requirements of a critical-section solution?”
- “How large should a critical section be?”

11. Follow-up Questions

- “What does Amdahl’s law say about your lock?” “How do you find your hottest lock?” (perf lock, mutex profilers, JFR)

12. Coding/Debugging Scenarios

- p99 spikes traced to a lock held across a network call → move the call out, hold the lock only to swap results in.

13. Best Practices

One lock per invariant; document what each lock guards; no I/O under locks; prefer data partitioning (sharded locks) over one hot lock.

14. Practice Questions

1. Rewrite: `lock { row = db.read(k); row.v++; db.write(row); }` to minimize the CS.
2. Which of mutual exclusion / progress / bounded waiting does a plain spinlock violate?

3.3 Mutex

1. Why Interviewers Ask This

“How does a mutex actually work?” is the definitive senior filter — they want futex mechanics, not “it’s a lock”.

2. Core Concept

A mutex (mutual exclusion lock) grants exclusive ownership: acquire blocks until available; only the owner may release. Sleeps the waiter rather than spinning (vs spinlock), has an owner (vs binary semaphore).

3. Internal Working

Modern Linux mutexes (pthread) are built on **futex** (“fast userspace mutex”): - **Fast path** (uncontended): one atomic CAS on a user-space word (0 → 1 locked). No syscall. ~15–25 ns. - **Slow path** (contended): set state to “locked with

waiters” (e.g., 2), call `futex(FUTEX_WAIT)` — kernel sleeps the thread on a wait queue keyed by that memory address. - **Unlock**: atomic set 0; if waiters flag was set, `futex(FUTEX_WAKE)` one thread. - Many implementations spin briefly before sleeping (adaptive mutex) to win when hold times are tiny. Variants: recursive (re-entrant), timed (`trylock`/timeout), PI (priority inheritance), robust (owner died).

4. ASCII Diagram

```
lock():
  CAS(state, 0 -> 1) success? ----> in critical section (NO syscall)
    | fail
    set state=2; futex_wait(&state,2) -> kernel sleep
unlock():
  state=0; if had waiters: futex_wake(&state,1)
Uncontended cost ~ one atomic op; contended cost ~ 2 syscalls + ctx switch (~ us)
```

5. Real Production Example

Every language runtime: Java `synchronized` (biased → thin → fat lock inflation, fat = OS mutex), Go `sync.Mutex` (CAS + semaphore/futex, with a “starvation mode” for fairness after 1 ms of waiting), glibc `pthread_mutex_t`.

6. Advantages

Simple ownership semantics; blocked waiters consume no CPU; ubiquitous and well-optimized; PI/robust variants for hard cases.

7. Trade-offs

Contended path costs syscalls + context switches (μ s); unfair by default (barging/lock convoys); can deadlock; priority inversion without PI.

8. Common Mistakes

- Saying “mutex always makes a syscall” — the uncontended fast path doesn’t (that’s the whole point of futexes).
- Unlocking from a different thread (UB for mutexes; legal for semaphores).
- Using recursive mutexes to paper over unclear ownership.

9. Performance Implications

Uncontended: nanoseconds. Contended: microseconds each (wait + wake + switch) plus cache-line transfer of the lock word between cores. Throughput of a hot mutex collapses past ~50–70% utilization — shard or redesign before that.

10. Common Interview Questions

- “Explain how a futex-based mutex works, fast and slow path.”
- “Mutex vs spinlock — when each?” “Mutex vs binary semaphore?”

11. Follow-up Questions

- “What is an adaptive mutex?” “What is lock convoy/barging?” “What happens if the owner dies?” (robust mutex, EOWNERDEAD)

12. Coding/Debugging Scenarios

- `perf` shows heavy `futex` syscall time → contended mutex; find it (`perf lock record`, JFR monitor-blocked events), then shrink CS/shard.
- Java service: profiler shows lock inflation on one `synchronized` map → replace with `ConcurrentHashMap` or striped locks.

13. Best Practices

Scope-based lock guards (RAII/defer/try-with); consistent acquisition order; prefer trylock+timeout on cross-subsystem locks; keep the lock word on its own cache line if adjacent hot data (false sharing).

14. Practice Questions

1. Implement a mutex on top of `futex_wait/futex_wake` with the 0/1/2 state machine — why is state 2 needed? (avoid lost wakeups / needless FUTEX_WAKE)
2. Go's mutex "starvation mode": what problem does it fix and at what cost?

3.4 Semaphore

1. Why Interviewers Ask This

Semaphores test whether you understand *counting resources* and signaling between threads — plus the classic producer-consumer implementation.

2. Core Concept

A semaphore is a counter with two atomic ops: `wait/P/acquire` (decrement; block if would go below 0) and `signal/V/release` (increment; wake a waiter). **Counting** semaphore = N permits (resource pool); **binary** = 0/1 (mutex-like, but no ownership — any thread may signal).

3. Internal Working

Kernel/runtime keeps `{count, wait queue}`; wait: `count--; if (count<0) sleep(queue)`; signal: `count++; if (count==0) wake_one(queue)`. On Linux built on futexes (`sem_t`), in Java on AQS (`java.util.concurrent.Semaphore`).

4. ASCII Diagram

```
Bounded buffer (size N) with 3 semaphores:
empty = N, full = 0, mutex = 1

Producer:          Consumer:
wait(empty)         wait(full)
wait(mutex)         wait(mutex)
  buffer.put(item)    item = buffer.take()
signal(mutex)       signal(mutex)
signal(full)         signal(empty)
ORDER MATTERS: wait(mutex) before wait(empty) => deadlock!
```

5. Real Production Example

- Connection pools (DB pool of 50 = `semaphore(50)`).
- Rate limiting concurrent requests to a fragile downstream (bulkhead pattern, e.g., Semaphore isolation in resilience libraries).
- Bounded job queues; JDK `Semaphore` guarding native resource handles.

6. Advantages

Counts resources naturally; works across processes (named/POSIX semaphores); signaler needn't be the "owner" → usable for ordering/notification between different threads.

7. Trade-offs

No ownership → can't detect misuse (double signal silently raises capacity), no priority inheritance; easy to deadlock with wrong wait ordering; "who signals whom" logic is implicit and fragile vs condition variables.

8. Common Mistakes

- Swapping `wait(empty)` / `wait(mutex)` order in producer-consumer (deadlock — memorize why).
- Using a binary semaphore as a mutex and claiming they're identical (ownership, PI, error checking differ).
- Forgetting a `signal` on an error path (permit leak → pool drains to zero over days).

9. Performance Implications

Same futex cost model as mutex. A semaphore-guarded pool serializes on the semaphore's cache line at high rates — for very hot paths, use per-CPU/sharded pools.

10. Common Interview Questions

- “Implement producer-consumer with semaphores.” (top-5 classic — write it cold)
- “Binary semaphore vs mutex?” “How would you implement a semaphore with a mutex + condition variable?”

11. Follow-up Questions

- “What happens with 2 producers and 2 consumers — does your code still work?” (yes, mutex protects the buffer)
- “How do you implement readers-writers with semaphores?” “What if a consumer crashes holding the mutex?” (robust/futex cleanup, or don't share across processes casually)

12. Coding/Debugging Scenarios

- Service gradually loses DB pool capacity → permit leak on exception path; fix with try/finally release.
- Throughput capped exactly at N → semaphore bound is the bottleneck; resize or shard.

13. Best Practices

Always release in `finally`; name semaphores for the resource they count; prefer higher-level bounded queues/pools from your language's stdlib.

14. Practice Questions

1. Write producer-consumer for buffer size 1 — which semaphore values change?
2. Implement `Semaphore` using one mutex + one condition variable + int.
3. Use a semaphore to limit a crawler to 100 concurrent fetches — sketch code.

3.5 Spinlock

1. Why Interviewers Ask This

Spin vs sleep is a fundamental cost trade-off; kernel code and low-latency systems live on it. Interviewers probe whether you know *when spinning wins*.

2. Core Concept

A spinlock busy-waits (loops on an atomic test) instead of sleeping. Wins when expected hold time < ~2 context switches (a few μ s); mandatory in contexts that cannot sleep (interrupt handlers, holding a raw kernel spinlock).

3. Internal Working

- Naive: loop on `CAS(lock, 0->1)` — hammers the cache line with exclusive ownership requests (bus traffic).
- **Test-and-test-and-set (TTAS)**: spin on a *read* (line stays Shared) and only CAS when observed free — far less coherence traffic.
- Add exponential backoff and `PAUSE` instruction (SMT-friendly, saves power).
- **Ticket lock**: `my = fetch_add(next_ticket); while (serving != my) spin;` — FIFO fair, but all spinners watch one word.
- **MCS/qspinlock**: each waiter spins on its *own* cache line in a queue node — scalable; Linux uses qspinlocks.
- Kernel rule: spinlock holders disable preemption (and IRQ variants disable interrupts) — never sleep while holding one.

4. ASCII Diagram

```
TAS spinner: while (xchg(&l,1)) ;           <- line ping-pongs (bad)
TTAS spinner: while (l==1) pause;           <- spin on local cached read
               until CAS(&l,0,1) succeeds
MCS: [holder] -> [w1 spins on w1.flag] -> [w2 spins on w2.flag]
               unlock passes the lock to w1 by writing w1.flag (one line touched)
```

5. Real Production Example

Linux kernel spinlocks protect run queues, drivers, network stack fast paths; DPDK/HFT user-space spinlocks for sub- μ s critical sections; adaptive mutexes spin-then-sleep (glibc, Java, Go all do bounded spinning first).

6. Advantages

No syscall/context switch; latency of handoff $\sim$ nanoseconds; usable where sleeping is illegal.

7. Trade-offs

Burns CPU while waiting; disastrous if the holder is preempted (waiters spin a full time slice — why user-space pure spinlocks are dangerous); fairness and coherence-traffic issues unless queue-based.

8. Common Mistakes

- Using spinlocks in user space with more runnable threads than cores.
- Not knowing TTAS vs TAS or why MCS exists (senior-level differentiator).
- Sleeping/allocating (which may sleep) while holding a kernel spinlock.

9. Performance Implications

Break-even: spin if hold time $< \sim 1\text{--}2\times$ context-switch cost. Under oversubscription, spinlocks invert: throughput collapses as spinners waste the holder's CPU (lock-holder preemption; also hurts VMs — hence paravirtual spinlocks).

10. Common Interview Questions

- “Spinlock vs mutex — when each?” “Implement a spinlock; now make it scalable.”

11. Follow-up Questions

- “Why does the **PAUSE** instruction exist?” “What is lock-holder preemption in VMs?” “Why does Linux use queued spinlocks?”

12. Coding/Debugging Scenarios

- 100% CPU but low throughput; perf shows cycles in lock loop $\rightarrow$ oversubscribed spinlock; switch to adaptive mutex or reduce thread count to core count.

13. Best Practices

Default to adaptive mutexes; pure spinlocks only for pinned-thread, cannot-sleep, sub- μ s sections; bound the spin then fall back to sleeping.

14. Practice Questions

1. Implement TAS, TTAS, and ticket locks; explain the coherence traffic of each.
2. Your VM guest's spinlock throughput drops $50\times$ under host CPU contention — explain.

3.6 Read-Write Lock

1. Why Interviewers Ask This

Tests whether you can exploit workload asymmetry (read-mostly) and whether you know the hidden costs: writer starvation and the shared reader counter.

2. Core Concept

Allows either **many concurrent readers** or **one exclusive writer**. Ideal for read-mostly data (config, routing tables, caches).

3. Internal Working

State $\approx$ {reader count, writer flag, wait queues}. Policies: - **Reader-preference**: readers enter if no active writer $\rightarrow$ continuous readers **starve writers**. - **Writer-preference**: arriving writer blocks new readers $\rightarrow$ readers can starve under write bursts. - **Fair/FIFO**: grant in arrival order (phase-fair). pthread default is implementation-defined; Java `ReentrantReadWriteLock(fair=true)` optional; Go `sync.RWMutex` blocks new readers once a writer waits. Every reader acquire/release atomically updates a shared counter $\rightarrow$ the counter's cache line bounces across cores, so at high core counts read-lock overhead can exceed the read itself.

4. ASCII Diagram

```
Readers: R1 ===  R2 ===  R3 ===      (concurrent, overlapping)
Writer:  W .....===      (waits for all readers, then exclusive)
Reader-pref: R4,R5,R6 keep arriving -> W waits forever (starvation)
Writer-pref: once W queued, new readers queue behind W
```

5. Real Production Example

In-memory config/feature-flag store read on every request, updated every few minutes; kernel `rwsem`s; DB shared/exclusive latches on B-tree pages; RCU (below) as the kernel's answer when even reader counting is too slow.

6. Advantages

Read parallelism for read-mostly workloads; exclusive writes keep invariants simple.

7. Trade-offs

Heavier than a mutex (two queues, counter); writer or reader starvation depending on policy; reader-counter cache-line contention; upgrade (read $\rightarrow$ write) is deadlock-prone (two upgraders wait on each other) and usually banned.

8. Common Mistakes

- Using RW locks for write-heavy or tiny critical sections (a plain mutex is faster).
- Not knowing the starvation policy of your platform's lock.
- Attempting lock upgrade.

9. Performance Implications

Rule of thumb: wins when reads $\geq$ ~90% *and* the read section is long enough to amortize the heavier acquire. For very hot read paths, prefer RCU / epoch-based reclamation / immutable-snapshot-swap (readers pay ~0).

10. Common Interview Questions

- "Readers-writers problem: implement with semaphores or CV; discuss writer starvation."
- "When is an RW lock *slower* than a mutex?"

11. Follow-up Questions

- "What is RCU / how do you get zero-cost readers?" (publish new version via atomic pointer swap; reclaim old after grace period)
- "Why is read $\rightarrow$ write upgrade dangerous?"

12. Coding/Debugging Scenarios

- Config updates take minutes to apply under load → writer starvation with reader-preferring lock; switch policy or snapshot-swap.
- Profiling shows `rwlock_rdlock` hot with 64 cores → counter contention; move to RCU/immutable snapshot.

13. Best Practices

Read-mostly + long reads → RW lock; read-mostly + short reads → immutable snapshot + atomic pointer; write-mixed → mutex; document the starvation policy you chose.

14. Practice Questions

1. Implement first-readers-writers (reader pref) with semaphores; then fix writer starvation.
2. Design the flag store: 1M reads/s, 1 write/min — which mechanism and why?

3.7 Condition Variable

1. Why Interviewers Ask This

CVs expose the two subtlest concurrency rules — the *predicate loop* and *spurious wakeups*. Getting `while` vs `if` wrong is an instant signal.

2. Core Concept

A condition variable lets threads sleep until a **condition over shared state** becomes true. Always used with a mutex: `wait(cv, m)` atomically releases `m` and sleeps; on wakeup it re-acquires `m`. `signal` wakes one waiter; `broadcast` wakes all.

3. Internal Working

- `wait`: enqueue self on cv's queue **and** release mutex atomically (otherwise a signal between “release” and “sleep” is lost — the **lost wakeup** problem); sleep (futex); on wake, re-acquire mutex, return.
- **Mesa semantics** (all real systems): signal only *hints*; the condition may be false again by the time the waiter runs → must recheck: `while (!pred) wait(cv, m);`
- **Spurious wakeups**: the OS may wake you with no signal at all (permitted by POSIX) — same `while` loop handles it.
- Signal with the mutex held (safest); broadcast when different waiters wait for different predicates or when state change may satisfy many.

4. ASCII Diagram

```

Consumer:                                Producer:
lock(m)                                  lock(m)
while (queue.empty())                    queue.push(x)
    wait(cv, m) <--- atomically          signal(cv)
item = queue.pop()                       releases m, unlock(m)
unlock(m)                                sleeps,
                                         re-acquires m on wake
WHY while? Mesa semantics + spurious wakeups + N waiters racing for 1 item.
```

5. Real Production Example

Every thread pool's task queue (`java.util.concurrent Condition`, Go channels internally, C++ `std::condition_variable`); bounded buffers; “wait until shutdown complete” barriers.

6. Advantages

Expresses “sleep until arbitrary predicate” cleanly; no busy waiting; composable with one mutex protecting the state.

7. Trade-offs

Easy to misuse (if-instead-of-while, signaling without the lock, wrong cv/mutex pairing); broadcast causes thundering herd (all wake, one wins, rest re-sleep).

8. Common Mistakes

- `if (!pred) wait();` — the classic.
- Calling `wait` without holding the mutex (UB) or signaling *before* any waiter and expecting it to be remembered (CVs are memoryless — unlike semaphores).
- Using one CV for two different predicates then `signal` waking the wrong class of waiter (use two CVs, e.g., `notFull/notEmpty`).

9. Performance Implications

Broadcast to N waiters = N wakeups, N mutex re-acquisitions serialized → herd. Prefer `signal` when any single waiter can make progress; two CVs for producer-consumer avoid waking producers for consumer events.

10. Common Interview Questions

- “Why must condition-variable wait be in a while loop?” (Mesa + spurious + multiple waiters)
- “Implement a bounded blocking queue with mutex + 2 CVs.” (top-5 coding classic)

11. Follow-up Questions

- “Mesa vs Hoare semantics?” (Hoare: signal transfers ownership immediately — textbook only)
- “Why must release-and-sleep be atomic?” (lost wakeup)
- “CV vs semaphore for signaling?” (semaphore remembers signals; CV needs state under a mutex)

12. Coding/Debugging Scenarios

- Rare hang: consumer waits forever though queue non-empty → producer signaled before consumer waited *and* consumer checked with `if` under a race, or signaled without lock; fix pattern.
- CPU spike at every enqueue → broadcast thundering herd; switch to signal.

13. Best Practices

Always: hold mutex, `while` predicate, prefer signal over broadcast, one CV per predicate, state changes before signal.

14. Practice Questions

1. Bounded blocking queue (put/take, capacity N) — write it in your language, cold.
2. Implement `CountDownLatch` with mutex + CV.
3. What breaks if `wait` released the mutex *then* enqueued itself?

3.8 Monitor

1. Why Interviewers Ask This

Mostly via Java: `synchronized/wait/notify` is a monitor. Tests whether you can name the pattern and its encapsulation benefit.

2. Core Concept

A monitor = an object bundling **data + mutex + condition variable(s)**, where all public methods run under the mutex. Mutual exclusion is implicit and encapsulated, unlike raw mutexes sprinkled through code.

3. Internal Working

Java: every object has a monitor — `synchronized` acquires it; `wait()`/`notify()`/`notifyAll()` are the built-in single condition queue. HotSpot lock inflation: thin lock (CAS on header word) → fat lock (OS mutex + queues) under contention. C#'s `lock`, Python's condition objects, and “synchronized classes” follow the same pattern. Limitation: one implicit CV per object → use explicit `Lock`+multiple `Condition`s when you need notFull/notEmpty separation.

4. ASCII Diagram

```
+----- Monitor object -----+
| private state (queue, count, ...) |
| implicit mutex (entry set)        |
| condition queue(s) (wait set)     |
| synchronized put(x){ while(full) wait(); |
|     ...; notifyAll(); }            |
+-----+
Threads: [entry set: want lock] [wait set: called wait()]
```

5. Real Production Example

Any pre-java.util.concurrent Java code; `Hashtable/Vector` (fully synchronized monitors — and why they don't scale); guarded objects in Kotlin/C# services.

6. Advantages

Encapsulation: impossible to touch state without the lock; simpler mental model; language support (compiler-enforced release even on exception).

7. Trade-offs

One-lock-per-object coarseness; Java's single wait set forces `notifyAll` in mixed-waiter designs; monitor-based collections serialize everything (vs `ConcurrentHashMap` striping/CAS).

8. Common Mistakes

- `notify()` when waiters wait on different conditions (lost signal to the wrong waiter → hang) — use `notifyAll` or explicit Conditions.
- Synchronizing on a public object/string literal (external code can lock it → accidental contention/deadlock).
- Believing monitors prevent deadlock (nested monitor calls deadlock like any nested locks).

9. Performance Implications

Java uncontended `synchronized` is nearly free (thin lock CAS); contended → inflation to OS mutex. Monitor-per-collection designs cap throughput — measure and move to concurrent structures.

10. Common Interview Questions

- “How does `synchronized` work internally?” (monitor, lock inflation)
- “`wait/notify` vs `Lock/Condition`?” “Why `notifyAll` usually?”

11. Follow-up Questions

- “What happens if you call `wait()` outside `synchronized`?” (`IllegalMonitorStateException`)
- “Monitor vs semaphore as a language design choice?”

12. Coding/Debugging Scenarios

- Thread dump shows threads in `Object.wait()` forever → a `notify` chose the wrong waiter class; change to `notifyAll` or split conditions.

13. Best Practices

Lock on private final objects; keep synchronized methods short; prefer `java.util.concurrent` structures over hand-rolled monitors.

14. Practice Questions

1. Implement the bounded buffer as a Java monitor with `wait/notifyAll`; then with `ReentrantLock` + two `Conditions` — compare.
2. Why is `Vector` “thread-safe” yet `for(i<v.size()) v.get(i)` still racy? (composite operation — check-then-act across method calls)

3.9 Atomic Operations

1. Why Interviewers Ask This

Foundation of lock-free code and of every lock’s implementation. Senior interviews probe memory ordering, not just “it’s indivisible”.

2. Core Concept

An atomic operation completes as one indivisible unit — no thread observes it half-done. Hardware provides atomic load/store (aligned word), exchange, fetch-and-add, compare-and-swap. Atomics also carry **memory-ordering** semantics (acquire/release/seq_cst) that constrain how surrounding reads/writes may be reordered.

3. Internal Working

- x86: `LOCK`-prefixed instructions (e.g., `lock xadd`, `lock cmpxchg`) — the core holds the cache line in **Modified** state exclusively for the duration (modern CPUs lock the line, not the bus).
- ARM: LL/SC (`ldxr/stxr`) retry loops, or newer single-instruction atomics (LSE).
- Cost: uncontended atomic ≈ 10–20 cycles more than a plain op; **contended** atomics serialize on cache-line ownership — every op transfers the line between cores (~40–100+ ns each).
- Ordering: **acquire** (later ops can’t move before it — for lock acquire/reads of a flag), **release** (earlier ops can’t move after — for publish/unlock), **seq_cst** (single global order, priciest), **relaxed** (atomicity only — counters). Java `volatile` ≈ seq_cst load/store; Go’s `sync/atomic` is sequentially consistent.

4. ASCII Diagram

```

Publish pattern (why ordering matters):
T1: data = 42;           T2: while(!flag.load(acquire));
    flag.store(1, release);   use(data); // guaranteed 42
Without release/acquire, CPU/compiler may reorder:
    flag=1 visible BEFORE data=42 -> T2 reads garbage.
  
```

5. Real Production Example

Metrics counters (`fetch_add` relaxed), sequence numbers, ring buffers (Disruptor), refcounts (`shared_ptr`, kernel `atomic_t`), once-init flags, lock implementations themselves.

6. Advantages

No blocking, no deadlock, no priority inversion; nanosecond-scale; signal-handler and interrupt safe (locks are not).

7. Trade-offs

Only single-word granularity — multi-field invariants still need locks or careful protocols; memory-ordering bugs are the hardest class to debug; contended atomics still serialize (they’re not free parallelism).

8. Common Mistakes

- “Atomic = fast under contention” — a contended atomic counter can bottleneck a 64-core box (fix: per-CPU/sharded counters, e.g., `LongAdder`).
- Assuming two separate atomics update atomically *together*.
- Using relaxed ordering for a publish/subscribe flag.

9. Performance Implications

Uncontended: ~ns. Contended: cache-line ping-pong, ~100 ns+/op and falling throughput with more cores (negative scaling). Also **false sharing**: two unrelated atomics on one 64-byte line contend — pad/align hot fields.

10. Common Interview Questions

- “How is `fetch_add` implemented in hardware?” “What does `volatile` do in Java vs C?”
- “Why do we need memory barriers at all?” (compiler + CPU reordering, store buffers)

11. Follow-up Questions

- “Acquire/release vs `seq_cst` — give a use for each.” “What is false sharing and how do you fix it?”
- “Why is a global atomic counter a bad metrics design at 64 cores?” (per-CPU shards, sum on read)

12. Coding/Debugging Scenarios

- Throughput *drops* when adding cores; perf shows one hot cache line → contended atomic/false sharing; shard or pad.
- ARM-only crash of a lock-free queue that “worked on x86” → missing barriers (x86’s stronger ordering hid the bug).

13. Best Practices

Default `seq_cst` until profiling says otherwise; encapsulate lock-free code behind tested abstractions; pad contended fields to cache lines; prefer `stdlib` concurrent primitives.

14. Practice Questions

1. Implement a spinlock with `atomic_exchange` + acquire/release — annotate each ordering choice.
2. Why can’t you implement a correct mutex with plain loads/stores on modern hardware?
3. Design a 64-core-friendly request counter.

3.10 Compare-And-Swap (CAS)

1. Why Interviewers Ask This

CAS is *the* lock-free primitive; interviews test the retry-loop pattern and the ABA problem — a sharp senior discriminator.

2. Core Concept

`CAS(addr, expected, desired)`: atomically, if `*addr == expected` then `*addr = desired`, return success; else return failure (and the current value). Universal building block: locks, lock-free stacks/queues, optimistic updates.

3. Internal Working

- Hardware: x86 `lock cmpxchg`; ARM LL/SC pair.
- **Retry loop** (optimistic concurrency):

```
do { old = load(addr); new = f(old); } while (!CAS(addr, old, new));
```

- **Lock-free** guarantee: some thread always makes progress (a failed CAS means *another* CAS succeeded) — but any individual thread may starve (not wait-free).
- **ABA problem**: value went $A \rightarrow B \rightarrow A$ between your load and CAS; CAS sees “A” and succeeds, but the *structure* changed (e.g., stack node freed and reallocated at same address → corruption). Fixes: version-tagged pointers (double-width CAS), hazard pointers/epoch reclamation, or GC languages (a “same” reference really is the same object).

4. ASCII Diagram

```

Lock-free stack push:
do {
    node->next = top;
} while(!CAS(&top, node->next,
            node));

ABA on pop:
T1: reads top=A, next=B ... (paused)
T2: pop A, pop B, push A  (A reused!)
T1: CAS(&top, A, B) SUCCEEDS
    -> top=B, but B was freed! CORRUPT
Fix: CAS on (pointer, version) pair

```

5. Real Production Example

`java.util.concurrent` everywhere (AtomicLong, ConcurrentHashMap bins, AQS state); Go runtime scheduler queues; kernel lockless fast paths; **optimistic locking in databases** (`UPDATE ... WHERE version = ?`) — CAS at the row level; etcd/ZooKeeper compare-and-set on version = distributed CAS.

6. Advantages

Non-blocking: no deadlock, no priority inversion, no lock-holder-preemption; great under low/moderate contention; composable into optimistic patterns at every layer (memory → DB → distributed).

7. Trade-offs

Retry storms under high contention (wasted work, line ping-pong); single-word scope; ABA; algorithms are notoriously hard to prove correct (use libraries).

8. Common Mistakes

- Recomputing an expensive `f(oid)` in a hot retry loop.
- Ignoring ABA in manual-memory-management languages.
- Claiming lock-free = wait-free (individual starvation still possible).

9. Performance Implications

Low contention: CAS ≈ one atomic (~ns) — beats mutex. High contention: failure rate soars; `fetch_add` (always succeeds) beats CAS-loop for counters; backoff or sharding needed. Rule: CAS for rarely-contended state; locks for complex/contended invariants.

10. Common Interview Questions

- “Implement `AtomicInteger.incrementAndGet` with CAS.” “Explain the ABA problem with a concrete corruption scenario.”
- “How does `ConcurrentHashMap.putIfAbsent` relate to CAS?”

11. Follow-up Questions

- “Lock-free vs wait-free?” “How does LL/SC differ from CAS regarding ABA?” (LL/SC fails on *any* write to the line → naturally ABA-immune, but can livelock)
- “How is DB optimistic locking the same idea?” (read version → attempt conditional write → retry on conflict)

12. Coding/Debugging Scenarios

- Heavy retry loop visible as high CPU with flat throughput → contention collapse; add exponential backoff or switch to a lock/sharding.
- Rare crash in a custom lock-free C++ queue only under load → ABA/reclamation bug; adopt hazard pointers or a proven library.

13. Best Practices

Prefer stdlib lock-free structures; keep `f(oid)` cheap and side-effect-free; bounded retries with backoff; in C/C++, pair CAS with a safe memory-reclamation scheme.

14. Practice Questions

1. Write a lock-free stack (push/pop) and point at the exact ABA line.

2. Implement optimistic row update with a version column; what's the retry policy?
3. When would you deliberately choose a mutex over CAS even at low contention? (multi-field invariant, need fairness/PI)

Module 3 Cheat Sheet (one page)

Primitive	Blocking?	Ownership	Best for	Killer detail
Mutex (futex)	Sleeps	Yes	General mutual exclusion	Uncontended = 1 CAS, no syscall
Spinlock	Busy-waits	Yes	<μs sections, no-sleep contexts	TTAS/MCS reduce cache traffic
Semaphore	Sleeps	No	Counting resources, pools	Any thread may signal; permits leak
RW lock	Sleeps	Yes	Read-mostly, longer reads	Writer starvation; reader-counter contention
Condition var	Sleeps	With mutex	Wait-for-predicate	<code>while</code> loop! Mesa + spurious wakeups
Monitor	Sleeps	Implicit	Encapsulated shared state	Java <code>synchronized</code> ; 1 wait set/object
Atomics	No	—	Counters, flags, publish	Ordering: acquire/release/seq_cst
CAS	No	—	Lock-free, optimistic	ABA; retry loops; lock-free ≠ wait-free

Costs: uncontended atomic/mutex ~ns; contended mutex ~μs (futex syscalls + switch); contended cache line ~100 ns/op and negative scaling. **Golden rules:** lock data not code; no I/O under locks; `while(!pred) wait()`; consistent lock order; shard hot state; prefer stdlib. **Producer-consumer:** `empty=N, full=0, mutex=1`; wait resource-semaphore *before* mutex.

Top Interview Questions

1. Why is `count++` racy — show instruction interleaving.
2. How does a mutex work internally (futex fast/slow path)?
3. Mutex vs spinlock vs semaphore — decision table with costs.
4. Implement producer-consumer (semaphores) and bounded blocking queue (mutex + 2 CVs).
5. Why `while` around `cv.wait()`?
6. Readers-writers with starvation discussion.
7. Explain CAS and the ABA problem with a corruption scenario.
8. False sharing and contended-atomic scaling collapse.

Common Mistakes (module-wide)

- `if` instead of `while` at CVs; signaling without the mutex.
- “Volatile makes ++ atomic”; “atomics are free”; “lock-free = wait-free”.
- Wrong semaphore ordering in producer-consumer (deadlock).
- I/O inside critical sections; lock upgrade attempts; ignoring writer starvation.
- User-space spinlocks with oversubscribed threads.

Mock Interview (self-test, ~25 min)

1. (Code) Bounded blocking queue with mutex + two condition variables — write, then defend every line.
2. (Depth) Walk through a futex mutex: two threads contend; list every atomic op, syscall, and state value.
3. (Design) A 64-core service updates one global stats counter 5M times/sec — it's the bottleneck. Fix it (sharded/per-CPU counters), estimate the win.

4. (Trap) “We replaced our mutex with an RW lock and got slower.” Give three plausible reasons.
5. (Trap) Your lock-free stack corrupts memory once a week in C++, never in the Java port. Explain (ABA + reclamation vs GC).
6. (Scenario) p99 latency spikes; `perf` shows 30% of cycles in `futex_wait`. Diagnose step by step.

MODULE 4 — Deadlocks

Senior SWE Interview Track — Operating Systems

4.1 Coffman Conditions

1. Why Interviewers Ask This

The four conditions are the checklist every deadlock answer must be built on: to prevent deadlock you break exactly one, and interviewers expect you to say *which one* your fix breaks.

2. Core Concept

Deadlock: a set of threads/processes where each waits for a resource held by another in the set — none can ever proceed. All four **Coffman conditions** must hold simultaneously: 1. **Mutual exclusion** — resources are exclusively held. 2. **Hold and wait** — holders request more while holding. 3. **No preemption** — resources can't be forcibly taken. 4. **Circular wait** — a cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$ of “waits for”.

3. Internal Working

Model as a **Resource Allocation Graph (RAG)**: edges $T \rightarrow R$ (request) and $R \rightarrow T$ (assignment). With single-instance resources, deadlock $\Leftrightarrow$ cycle in the RAG. With multi-instance resources, a cycle is necessary but not sufficient (someone outside the cycle may release).

4. ASCII Diagram

```
Classic AB-BA deadlock:
T1: lock(A) ... lock(B)      T2: lock(B) ... lock(A)

      holds          requests
T1 -----> A        T1 ----> B
T2 -----> B        T2 ----> A

RAG cycle: T1 -> B -> T2 -> A -> T1  => DEADLOCK
All 4 conditions hold: exclusive locks, holding-while-waiting,
no forced release, circular wait.
```

5. Real Production Example

- DB transactions: TX1 updates rows (1,2), TX2 updates (2,1) → MySQL/Postgres detect and abort one (“Deadlock found when trying to get lock”).
- Java services: two subsystems each take `lockA` then `lockB` in opposite orders under rare code paths — fires once a month at peak.

6. Advantages

The framework converts “it's stuck” into a diagnosable checklist and names the exact lever each strategy pulls.

7. Trade-offs

Conditions 1–3 are usually *desirable* properties (that's why circular wait is the one we usually break).

8. Common Mistakes

- Listing 3 of 4 conditions; forgetting all four must hold **simultaneously**.
- Calling any hang a “deadlock” (could be livelock, starvation, or a lost wakeup).
- Thinking a RAG cycle always means deadlock (multi-instance caveat).

9. Performance Implications

Deadlocks freeze throughput to zero for the involved set and everything queued behind their locks; even deadlock-prone designs force conservative, coarse locking that costs parallelism.

10. Common Interview Questions

- “List the four Coffman conditions and give a two-lock example.”
- “Which condition does lock ordering break?” (circular wait)

11. Follow-up Questions

- “Deadlock vs livelock vs starvation?” “Draw the RAG for this scenario — is it deadlocked?”

12. Coding/Debugging Scenarios

- `jstack` prints “Found one Java-level deadlock” with both stacks → read the cycle, fix the lock order.
- Two goroutines blocked forever on channels each expecting the other to send — same cycle, no locks (communication deadlock).

13. Best Practices

For every fix you propose, name the broken condition; treat locks + any blocking wait (channels, thread-pool tasks waiting on pool tasks) as RAG resources.

14. Practice Questions

1. Map each of these fixes to the condition it breaks: lock ordering, trylock-with-backoff, lock-free structures, acquiring all locks upfront.
2. Thread pool of size 4; every task submits a subtask to the same pool and waits for it. Deadlock? Which “resource” cycles? (pool threads — thread-pool starvation deadlock)

4.2 Deadlock Prevention

1. Why Interviewers Ask This

“How do you *prevent* deadlocks in your service?” is the practical senior question — the expected headline answer is **global lock ordering**.

2. Core Concept

Prevention = design so one Coffman condition can never hold: - **Break mutual exclusion**: share (immutable data, RW read side) or go lock-free (CAS). - **Break hold-and-wait**: acquire *all* resources at once, or release everything before re-acquiring. - **Break no-preemption**: `trylock` with timeout — on failure release what you hold and retry (preempt yourself). - **Break circular wait**: impose a **total order** on locks; always acquire in ascending order. Cycle impossible.

3. Internal Working

Lock ordering proof: if every thread acquires locks in increasing rank, then a waiting thread always waits for a lock ranked *higher* than all it holds → any wait chain is strictly increasing → no cycle. Implementation: rank by address (e.g., `lock(min(a,b))`; `lock(max(a,b))` for transfers), by fixed hierarchy documented per subsystem, or enforced at runtime (lockdep, custom lock-rank asserts).

4. ASCII Diagram

```
transfer(from, to):
  first = min(from.id, to.id)
  second = max(from.id, to.id)
  lock(first); lock(second)
  ...
  unlock(second); unlock(first)
```

T1: transfer(A,B) -> lock A, lock B
T2: transfer(B,A) -> lock A, lock B (same order!)

Wait-chains only go "up" the order
=> no cycle => no deadlock

5. Real Production Example

- Linux kernel: documented lock ordering + **lockdep** (runtime validator that flags any ordering violation ever observed, even before a real deadlock).
- Bank-transfer double-entry code ordering account locks by ID — the canonical interview snippet.
- DB engines latch B-tree pages in a fixed direction (root → leaf, left → right) for the same reason.

6. Advantages

Deadlock becomes *impossible*, not just unlikely; zero runtime cost for lock ordering; simple to review.

7. Trade-offs

- All-at-once acquisition: poor utilization, must know needs upfront.
- Timeout/backoff: converts deadlock into retries — can livelock; spurious timeouts under load.
- Global order: hard across module boundaries/callbacks; ordering may force holding locks longer than needed.

8. Common Mistakes

- Proposing timeouts as “prevention” without a backoff/jitter story (→ livelock).
- Lock ordering by object identity that can change (use stable IDs/addresses).
- Forgetting that callbacks/virtual methods invoked under a lock can acquire arbitrary locks (order violated invisibly).

9. Performance Implications

Prevention is cheapest at design time. Coarser “grab everything” approaches serialize; ordering costs nothing at runtime — that’s why it’s the default answer.

10. Common Interview Questions

- “You have `transfer(a, b)` called concurrently with `transfer(b, a)` — fix it.” (order by ID; mention tie/self-transfer)
- “Which Coffman condition does each technique break?”

11. Follow-up Questions

- “How do you enforce lock order across a 2M-line codebase?” (lockdep-style runtime checking, lint, lock hierarchy docs)
- “What if you can’t order because locks are discovered dynamically?” (trylock+backoff, or restructure)

12. Coding/Debugging Scenarios

- Implement `transfer` with address-ordered `std::scoped_lock(m1, m2)` (which internally uses a deadlock-avoidance algorithm) — know that `std::lock` exists.
- Add a debug build assert: thread-local list of held lock ranks; acquiring a lower rank than held → abort with both stacks.

13. Best Practices

Document a lock hierarchy; never call unknown code (callbacks, listeners) while holding locks; prefer one lock per subsystem until profiling demands finer grain; use scoped multi-lock APIs.

14. Practice Questions

1. Write deadlock-free `transfer(from, to, amount)` handling `from == to`.
 2. Your fix uses trylock with 50 ms timeout; under peak load transfers start failing in bursts. Explain and improve (jittered backoff, ordering instead).
-

4.3 Deadlock Avoidance

1. Why Interviewers Ask This

Tests whether you know the prevention/avoidance/detection taxonomy precisely — avoidance = runtime decisions using *future knowledge* (max claims), i.e., the Banker's algorithm family.

2. Core Concept

Avoidance grants a resource request **only if the resulting state is safe** — a state from which *some* execution order lets every process finish with its declared **maximum claim**. Unsafe $\neq$ deadlocked; unsafe = no guarantee. The system dodges unsafe states so deadlock can never be reached.

3. Internal Working

Each process declares max resource needs upfront. On each request, the system simulates: “pretend to grant; can I still find a completion order for everyone?” If yes $\rightarrow$ grant; else $\rightarrow$ make the requester wait. The check is the Banker's safety algorithm (next section).

4. ASCII Diagram

```
State space:
[ SAFE states ] --grant--> [ SAFE ]      always allowed
[ SAFE ] --grant--> [ UNSAFE ]          REFUSED (wait instead)
[ UNSAFE ] --unlucky requests--> [ DEADLOCK ]
Avoidance keeps the system inside the SAFE region at all times.
```

5. Real Production Example

Rare in OS kernels (max claims unknown), but the *pattern* is everywhere: admission control — a cluster scheduler (Kubernetes, YARN) only places a pod if the node can satisfy its declared resource *requests*; DB connection admission that refuses work that could exhaust the pool mid-transaction; memory reservations before starting a batch job.

6. Advantages

No deadlock and no aborted work (vs detection/recovery); better utilization than acquire-all-upfront prevention.

7. Trade-offs

Requires accurate max-claim declarations (rarely available); per-request $O(m \cdot n^2)$ safety check; conservative — refuses grants that would in practice be fine; fixed process/resource population assumptions.

8. Common Mistakes

- Conflating avoidance with prevention (prevention = structural impossibility; avoidance = dynamic safe-state checks).
- Saying unsafe = deadlocked.

9. Performance Implications

Safety-check cost per allocation + reduced concurrency from conservative refusals; that's the price of never aborting.

10–11. Interview & Follow-ups

- “Prevention vs avoidance vs detection — one line each + example.”
- “Why don't real OSes use avoidance?” (unknown max claims, dynamic processes, check cost)

12. Coding/Debugging Scenario

Design admission control for a worker that needs up to **m** DB connections per job from a pool of **P**: only admit a job if remaining pool $\geq$ its declared max after reserving others' outstanding maxima — that *is* banker's thinking.

13. Best Practices

Use avoidance-style admission control at system boundaries (declared quotas/requests), not fine-grained locks.

14. Practice Questions

1. Give a state that is unsafe yet never deadlocks. Why did avoidance still refuse it?
2. Map Kubernetes resource requests/limits onto avoidance vocabulary.

4.4 Banker's Algorithm (High Level)

1. Why Interviewers Ask This

Occasionally asked to run one small example by hand; mostly they test that you understand **safe sequence** reasoning.

2. Core Concept

Named for a banker who never lends cash such that clients' *maximum* future needs can't all eventually be met. Data: $Available[m]$, $Max[n][m]$, $Allocation[n][m]$, $Need = Max - Allocation$.

3. Internal Working

Safety check: $Work = Available$; repeatedly find any process with $Need_i \leq Work$; pretend it finishes ($Work += Allocation_i$); mark finished. If all can finish $\rightarrow$ safe (that order is a **safe sequence**). **Request check:** if $request \leq Need_i$ and $\leq Available$, tentatively grant and run the safety check; commit if safe, else the process waits.

4. ASCII Diagram

```
Available = 3      Alloc  Max  Need
                   P0:   5    10   5
                   P1:   2     4   2
                   P2:   2     9   7
Safe? Work=3: P1 (2<=3) finishes -> Work=5
                P0 (5<=5) finishes -> Work=10
                P2 (7<=10) finishes -> SAFE, sequence P1,P0,P2
If instead P2 held 3 (Available=2): P1->Work=4; P0 needs 5 >4; P2 needs 6 >4
-> UNSAFE (grant would be refused)
```

5. Real Production Example

Directly: some embedded/RTOS and mainframe allocators. Spiritually: cloud capacity reservation systems and pool admission control ("never commit capacity you can't cover if everyone calls their max").

6. Advantages

Provably deadlock-free with no aborts; handles multi-instance resources (where RAG cycles aren't sufficient).

7. Trade-offs

Needs max claims upfront; $O(m \cdot n^2)$ per request; assumes fixed populations; conservative refusals.

8. Common Mistakes

- Arithmetic slips in hand simulation (always recompute $Need = Max - Alloc$ first).
- Declaring "unsafe" after trying only one candidate order — you must try all (any process with $Need \leq Work$ works; greedy is fine because granting only increases $Work$).

9. Performance Implications

Fine for coarse, infrequent allocations (jobs, pools); far too slow for per-lock granularity.

10–11. Interview & Follow-ups

- “Run the banker’s check on this 3×3 table.” “What exactly is a safe sequence?” “Why is it impractical for general OS use?”

12. Coding/Debugging Scenario

Implement `is_safe(available, alloc, max)` in 20 lines; property-test it against a brute-force order search.

13. Best Practices

Reserve banker-style logic for admission control with declared quotas; keep the checked resource coarse.

14. Practice Questions

1. 5 processes, 3 resource types (classic textbook table) — find a safe sequence.
2. Prove: granting a request that passes the safety check can never lead the checker itself to deadlock.

4.5 Deadlock Detection

1. Why Interviewers Ask This

This is what real systems (databases, JVM tooling) actually do — expect “how does MySQL/`jstack` find deadlocks?”

2. Core Concept

Let deadlocks happen; periodically (or on lock-wait) search for cycles in the **wait-for graph** (WFG: edge $T_i \rightarrow T_j$ if T_i waits for a resource T_j holds); on detection, break the cycle via recovery.

3. Internal Working

- Single-instance resources: DFS cycle detection in WFG, $O(V+E)$.
- Multi-instance: run a banker-like reduction with *current requests* (no Max needed): repeatedly “finish” any process whose request $\leq$ Work; unfinished set at fixpoint = deadlocked.
- Triggers: on each blocking wait (InnoDB checks when a lock wait starts), on a timer, or on demand (`jstack`, `SHOW ENGINE INNODB STATUS`).

4. ASCII Diagram

```
Wait-for graph:
TX1 --waits-for--> TX2 --waits-for--> TX3
  ^                               |
  +----- waits-for -----+
DFS finds cycle {TX1,TX2,TX3} -> pick victim (e.g., fewest undo records)
-> abort TX -> its locks release -> others proceed
```

5. Real Production Example

- **InnoDB**: detects on lock wait, rolls back the transaction with the smallest undo log, returns error 1213 — your app must retry.
- **Postgres**: checks the WFG after `deadlock_timeout` (default 1 s) of waiting.
- **JVM**: `jstack/JMX findDeadlockedThreads()` reports monitor cycles (but can’t fix them — process restart).
- Distributed systems mostly use **timeouts + retry** because building a global WFG is impractical.

6. Advantages

No upfront knowledge; zero overhead until contention; maximum concurrency in the common (non-deadlocked) case.

7. Trade-offs

Work is lost on abort; detection latency = stalled throughput window; requires recovery machinery; in-process deadlocks (mutexes) usually have no safe recovery except restart.

8. Common Mistakes

- Suggesting detection for plain in-process mutexes without noting you can't safely "abort" a thread holding locks (state corruption) — detection fits *transactional* systems where rollback exists.
- Forgetting the app-side retry after a DB deadlock error (a real prod bug: users see 1213).

9. Performance Implications

Detection cost scales with waiters (InnoDB deadlock checks can themselves become hot under extreme contention — it has a `innodb_deadlock_detect` off-switch that falls back to `innodb_lock_wait_timeout`).

10. Common Interview Questions

- "How does a database detect deadlocks?" "How would you detect a deadlock in a running JVM/Go service?" (jstack; Go: goroutine dump via SIGQUIT/pprof)

11. Follow-up Questions

- "How does the victim get chosen?" (least work to undo / fewest locks / lowest priority)
- "Why do distributed systems prefer lock timeouts over WFG detection?" (no global view, partial failure)

12. Coding/Debugging Scenarios

- Prod incident: order-service errors `1213 Deadlock found`. Steps: read `SHOW ENGINE INNODB STATUS` latest deadlock section → identify the two statements and index order → fix access order or add retry with jitter.
- Hung Java service: `jstack <pid>` → "Found one Java-level deadlock" → capture, restart, then fix ordering.

13. Best Practices

Always wrap transactional writes in idempotent retry; keep transactions short and touch rows in consistent order; alert on deadlock counters, not just errors.

14. Practice Questions

1. Implement WFG cycle detection given `waits_for: map<Tid, Tid>`.
2. Two UPDATE statements deadlock via different secondary indexes — explain how index choice creates opposite row-lock orders.

4.6 Deadlock Recovery

1. Why Interviewers Ask This

Completes the detection story: once found, *someone must lose*. Tests judgment about victim selection and starvation.

2. Core Concept

Options: 1. **Abort processes**: all in the cycle (brutal) or one at a time (re-detect after each) — victim selection by priority, work done, rollback cost, locks held, interactivity. 2. **Resource preemption**: forcibly take resources; requires the victim to **rollback** to a safe point; risk of the same victim losing repeatedly → **starvation** (bound it by counting rollbacks / aging).

3. Internal Working

Transactional systems make recovery clean: undo logs restore state, locks release atomically. Non-transactional (in-memory mutexes): no safe preemption — recovery = kill/restart the process (crash-only design), which is why prevention matters more inside a process.

4. ASCII Diagram

```
detect cycle {TX1,TX2,TX3}
  victim = argmin(rollback_cost) = TX2
  rollback TX2 -> release its locks
  TX1, TX3 proceed; TX2 retried (with backoff; count retries!)
  Repeated victimization of TX2 -> starvation -> raise its priority (aging)
```

5. Real Production Example

InnoDB rolls back the smallest transaction; Postgres aborts the waiter that triggered detection; Kubernetes-style systems “recover” by killing and rescheduling pods (preemption); microservices recover via request timeout + retry, treating the whole call as the abortable unit.

6. Advantages

Lets you run optimistically at full concurrency and pay only when deadlock actually happens (rare in well-ordered systems).

7. Trade-offs

Lost work, retry storms if uncoordinated (add jitter), starvation of repeat victims, complexity of choosing rollback points (checkpointing).

8. Common Mistakes

- No starvation bound on victim selection.
- Retrying immediately without jitter → the same collision replays (retry storm / livelock).
- Assuming you can kill one *thread* in a process safely.

9. Performance Implications

Recovery cost = wasted work × deadlock rate; if deadlock rate climbs, redesign locking rather than tuning recovery.

10–11. Interview & Follow-ups

- “You detected a deadlock among 3 transactions — who dies and why?”
- “How do you prevent the victim from starving?” (rollback counter → eventually never the victim)

12. Coding/Debugging Scenario

Implement DB-deadlock retry middleware: catch 1213/40P01, exponential backoff with jitter, max 3 attempts, idempotency guard.

13. Best Practices

Make units of work idempotent and retryable; short transactions; bounded, jittered retries; log every deadlock with both parties for offline fixing.

14. Practice Questions

1. Design victim selection for a payment system where TXs differ 1000× in size.
2. Why does “abort all in the cycle” ever make sense? (fast recovery when re-detection is expensive; cycles usually size 2 anyway)

4.7 Livelock

1. Why Interviewers Ask This

The favorite trap after you propose “trylock and retry”: states change, but no progress. Distinguishing deadlock/ livelock/starvation crisply is a senior marker.

2. Core Concept

Livelock: threads are *not blocked* — they actively run, respond to each other, and keep changing state — yet no one makes progress. Deadlock frozen; livelock busy.

3. Internal Working

Classic genesis: both threads take lock A/B respectively, fail trylock on the other, *both* release, *both* retry at the same cadence → perpetual collision. Same dynamics: two people side-stepping in a hallway; distributed leader election with synchronized retries; message redelivery ping-pong between two queues.

4. ASCII Diagram

```
T1: lock(A) tryB FAIL release(A) lock(A) tryB FAIL release(A) ...
T2: lock(B) tryA FAIL release(B) lock(B) tryA FAIL release(B) ...
    |_____ perfectly synchronized retries = livelock _____|
Fix: randomized jitter -> T1 retries at +7ms, T2 at +23ms -> one wins.
```

5. Real Production Example

- Retry storms: all clients time out together, retry together, overload the server, time out again (thundering-herd livelock) — fixed by exponential backoff **with jitter** (AWS's canonical guidance).
- CSMA/CD Ethernet used randomized backoff for exactly this.
- Two services each rolling back and retrying a distributed transaction on conflict, colliding forever.

6–7. Advantages / Trade-offs

(N/A — defect.) The relevant trade-off: retry-based deadlock fixes liveness risk; jitter + backoff + retry budget is the standard antidote.

8. Common Mistakes

- Defining livelock as “a kind of deadlock” — states differ (RUNNING vs BLOCKED), detection differs (livelock invisible to WFG/deadlock detectors!).
- Backoff without randomness (synchronized clients stay synchronized).

9. Performance Implications

Livelock burns 100% CPU/network while delivering zero goodput — often *worse* than deadlock operationally, and harder to spot because everything “looks busy”.

10–11. Interview & Follow-ups

- “Deadlock vs livelock vs starvation — definitions + one example each.”
- “Your trylock-retry fix — what new failure mode did you introduce and how do you mitigate it?” (livelock; jittered exponential backoff, retry budgets)

12. Coding/Debugging Scenario

CPU pegged, throughput ~0, no thread is blocked (all RUNNABLE in dumps), logs show rapid acquire/release cycles → livelock; add jitter and a progress metric alert.

13. Best Practices

Every retry loop: exponential backoff + full jitter + max attempts + circuit breaker; prefer ordering (prevents) over retry (avoids badly).

14. Practice Questions

1. Convert the AB-BA trylock livelock into a correct solution two ways (ordering; jittered backoff).
2. How would you *detect* livelock in production metrics? (high CPU + flat goodput + high retry counters)

4.8 Starvation (in the deadlock context)

1. Why Interviewers Ask This

Completes the liveness taxonomy. Asked as: “no deadlock, no livelock — can a thread still wait forever?” Yes: starvation.

2. Core Concept

Starvation: a thread is perpetually *able* to proceed but never chosen — others always win the resource. System-wide progress exists; individual progress doesn't. Causes: strict priorities, unfair locks (barging), reader-preferring RW locks vs writers, repeated deadlock-victim selection, greedy neighbors.

3. Internal Working

Unfair (barging) mutexes hand the lock to whoever CASes first on release — a just-arrived spinning thread beats a long-sleeping waiter (it's cache-hot and already running) → the sleeper can lose indefinitely. Fair (FIFO/ticket) locks fix this at ~2× handoff cost (must wake a specific sleeper instead of letting a runner barge). Java `ReentrantLock(true)`, Go mutex starvation mode (FIFO after 1 ms wait), ticket spinlocks are all “fairness switches”.

4. ASCII Diagram

```
Unfair lock timeline (T3 starves):
lock free -> T1 (barged) -> free -> T2 (barged) -> free -> T1 ...
                T3 asleep in wait queue: loses every race
Fair/FIFO lock: grant order = arrival order -> T3 bounded wait
Trade: fairness costs throughput (forced wakeups, cold caches)
```

5. Real Production Example

Go added mutex starvation mode (1.9) after real services saw tail-latency blowups from barging; writer starvation on reader-heavy RW locks delaying config updates; low-priority batch pods never scheduled in saturated clusters (fixed by priority aging / guaranteed quotas).

6–7. Advantages / Trade-offs

Unfairness is a deliberate *throughput optimization* (barging exploits cache-hot threads); fairness bounds tail latency at the cost of mean throughput. Say this trade explicitly — it's the senior insight.

8. Common Mistakes

- Confusing starvation with deadlock (starved thread proceeds instantly if granted; deadlocked never can).
- Assuming locks are fair by default (most aren't).
- Fixing starvation with fair locks everywhere (pay the cost only where tail latency matters).

9. Performance Implications

Symptom signature: healthy p50, monstrous p99.9 for a subset of threads/requests; overall throughput fine. Detect by per-thread wait-time histograms, not averages.

10–11. Interview & Follow-ups

- “Deadlock vs livelock vs starvation table.” “Why are unfair locks the default?” “How does Go's mutex starvation mode work?”

12. Coding/Debugging Scenario

One consumer thread of ten processes ~0 items under load; dumps show it always in lock wait → barging starvation; enable fair mode for that lock or shard the queue.

13. Best Practices

Bound waiting somewhere (fair mode after threshold — Go's design is the template); monitor max wait per waiter class; aging for priority systems (Module 2).

14. Practice Questions

1. Fill the 3×4 table: {deadlock, livelock, starvation} × {thread state, CPU usage, detectable by WFG?, canonical fix}.
2. Design a lock that is unfair for throughput but guarantees a 10 ms starvation bound.

Module 4 Cheat Sheet (one page)

Coffman (all four required): mutual exclusion · hold-and-wait · no preemption · circular wait.

Strategy	Idea	Breaks	Real example
Prevention	Make a condition impossible	Usually circular wait (lock ordering)	Order account locks by ID; kernel lockdep
Avoidance	Grant only if state stays safe	Dodges unsafe states	Banker's; K8s admission by declared requests
Detection	Find WFG cycles, then recover	—	InnoDB (abort smallest TX), Postgres 1 s check, jstack
Recovery	Abort victim / preempt+rollback	—	DB rollback + app retry w/ jitter
Ignore	"Ostrich" — restart on hang	—	Most in-process mutex code, honestly

Liveness taxonomy:

	Thread state	CPU	WFG-detectable	Fix
Deadlock	Blocked forever	0%	Yes	Ordering / detection+abort
Livelock	Running, no progress	High	No	Jittered backoff
Starvation	Ready, never chosen	others'	No	Fairness / aging

Golden answers: fix AB-BA by total lock order (min/max ID); DB deadlock → automatic victim rollback → app must retry with jittered backoff; trylock-retry introduces livelock → add jitter; unfair locks trade tail latency for throughput.

Top Interview Questions

1. Four Coffman conditions + two-thread example.
2. Fix `transfer(a,b)` vs `transfer(b,a)` — write the ordered-locking code.
3. Prevention vs avoidance vs detection — one-liners + where each is used.
4. Run Banker's safety check on a small table.
5. How does MySQL/Postgres handle deadlocks end-to-end (detection → victim → app retry)?
6. Deadlock vs livelock vs starvation.
7. Thread-pool deadlock: tasks waiting on subtasks in the same pool.

Common Mistakes (module-wide)

- Missing "all four simultaneously"; calling every hang a deadlock.
- Trylock-retry without jitter (livelock); retries without idempotency.
- Believing DB handles deadlocks "fully" — the app retry is your job.
- Proposing thread-kill recovery for in-process mutexes.

- No starvation bound in victim selection or priority schemes.

Mock Interview (self-test, ~20 min)

1. (Code) Write deadlock-free `transfer` including self-transfer and same-ID edge cases; state the invariant that proves no cycle.
2. (Scenario) Payments deadlock ~50×/day in InnoDB between two known queries. Walk from `SHOW ENGINE INNODB STATUS` to the code fix.
3. (Depth) Your colleague replaces all locks with `tryLock(100ms)` + retry. List every new failure mode and your mitigation.
4. (Design) A workflow engine runs user-defined steps that lock arbitrary entities. Design its anti-deadlock strategy (ordering by entity ID? detection? timeouts?) and defend it.
5. (Trap) `jstack` shows no monitor cycle, but the service is frozen and CPU is 95%. What class of failure do you hunt, and how? (livelock)

MODULE 5 — Memory Management

Senior SWE Interview Track — Operating Systems

5.1 Virtual Memory

1. Why Interviewers Ask This

The single most important OS abstraction. Interviewers use it to test whether you understand *why* your process sees 128 TB of address space on a 16 GB machine, and what RSS vs VSZ actually mean.

2. Core Concept

Virtual memory gives every process its own private address space; the MMU translates virtual → physical addresses per access. Benefits: **isolation** (processes can't touch each other), **overcommit** (allocate more than RAM; back with demand paging/swap), **sharing** (map the same physical page into many processes — libc, page cache), **contiguity illusion** (fragmented physical RAM looks contiguous).

3. Internal Working

- Address space carved into fixed **pages** (4 KB default); RAM into **frames**. Page tables map pages → frames with permission bits (R/W/X, user/kernel).
- CPU's MMU walks the page table on each access (cached by TLB). Invalid/missing mapping → **page fault** → kernel decides: load from disk (legit), COW copy, grow stack, or SIGSEGV (bug).
- Linux tracks a process's mappings as **VMAs** (see `/proc/<pid>/maps`); physical pages are allocated lazily on first touch.
- Overcommit: `malloc` reserves address space; RAM is committed on touch — why `VSZ` >> `RSS`, and why the **OOM killer** exists (the kernel's promise can outstrip reality).

4. ASCII Diagram

```

Process A virtual space      Physical RAM      Process B virtual space
+-----+                   +-----+         +-----+
| stack      | --\           | frame 9 | /-----| stack      |
| ...        | \            | frame 8 | /         | ...        |
| heap p2     | ---+-----> | frame 5 | <--+    | heap p7     |
| heap p1     | -----\    | frame 4 | <-----+ | libc (shared)|---+
| libc (shared)|--+ \-----> | frame 2 | <-----+ |
+-----+         | frame 1 | <-----+
                   +-----> | frame 0 | same physical libc frame
MMU + page tables do per-process translation; disk (swap) extends RAM.

```

5. Real Production Example

- Redis `fork()`-based RDB snapshots rely on virtual memory + COW: the child “copies” 60 GB instantly.
- Containers on one host: each process believes it owns the address space; cgroups cap the physical side.
- JVM `-Xmx` reserves huge virtual ranges upfront; resident grows with the live heap.

6. Advantages

Isolation/security, simple linking model (fixed layout + ASLR), overcommit efficiency, memory-mapped files, sharing.

7. Trade-offs

Translation overhead (TLB pressure), page-fault latency cliffs, overcommit → OOM-killer surprises, an extra layer to reason about (RSS vs VSZ vs shared confuses monitoring).

8. Common Mistakes

- Reading `VSZ` as memory usage (JVMs “use” 20 GB VSZ with 2 GB RSS).
- “Virtual memory = swap” — swap is just one backing store; VM exists even with swap off.
- Not knowing that `malloc` success $\neq$ RAM available (overcommit; failure comes later as OOM kill on touch).

9. Performance Implications

TLB miss $\rightarrow$ page-table walk (up to 4 memory accesses on x86-64, ~ 100 ns); major fault $\rightarrow$ disk (μ s on NVMe, ms on HDD). Random access over a working set larger than RAM = thrashing (Module 6).

10. Common Interview Questions

- “What happens, in full detail, when your program dereferences a pointer?”
- “RSS vs VSZ vs shared memory in `top` — explain.”

11. Follow-up Questions

- “How can total RSS of all processes exceed RAM?” (shared pages counted per-process — see PSS)
- “What is memory overcommit and what’s the failure mode?” (OOM killer; `vm.overcommit_memory` modes)

12. Coding/Debugging Scenarios

- Container OOM-killed though heap limit $<$ container limit $\rightarrow$ RSS includes native/mmap/metaspace beyond the heap; measure the whole process.
- Latency spikes correlated with first access to a big `mmap`ed file $\rightarrow$ page-fault storms; pre-touch or `madvise(WILLNEED)`.

13. Best Practices

Monitor RSS/PSS not VSZ; set container limits with headroom over the true working set; understand your runtime’s native overhead.

14. Practice Questions

1. Trace a load instruction: TLB hit path, TLB miss path, minor fault path, major fault path.
2. Two processes each report 1 GB RSS; the host shows 1.2 GB used. Explain.

5.2 Paging

1. Why Interviewers Ask This

The mechanism behind virtual memory. Tests address-splitting math and whether you know why fixed-size pages beat variable segments.

2. Core Concept

Divide virtual space into fixed **pages** and RAM into same-size **frames**; any page $\rightarrow$ any frame. Eliminates **external fragmentation** (all chunks same size); costs **internal fragmentation** (avg $\sim \frac{1}{2}$ page waste per allocation tail).

3. Internal Working

Virtual address = [page number | offset]. 4 KB pages $\rightarrow$ offset = low 12 bits. Translation: page number $\rightarrow$ (page table) $\rightarrow$ frame number; physical = frame $\cdot 4096$ + offset. Permissions checked at the same time (present, R/W/X, U/K, accessed/dirty bits maintained by hardware).

4. ASCII Diagram

```
32-bit VA, 4KB pages:
| 20 bits: virtual page number (VPN) | 12 bits: offset |
VPN --page table--> PFN (physical frame number)
PA = PFN<<12 | offset
```

```
VA 0x00403ABC -> VPN 0x403, offset 0xABC
PT[0x403] = frame 0x1F2 => PA = 0x1F2ABC
```

5. Real Production Example

Everything on x86-64/ARM64. Databases and JVMs also use **huge pages** (2 MB/1 GB) to cut TLB pressure — e.g., Postgres `huge_pages=on`, JVM `-XX:+UseLargePages`.

6. Advantages

No external fragmentation; simple allocation (any free frame); enables sharing, COW, demand paging, per-page permissions.

7. Trade-offs

Internal fragmentation; page-table memory; every access needs translation (TLB saves it); page granularity means 1-byte-used still costs 4 KB resident.

8. Common Mistakes

- Mixing up internal (paging) vs external (segmentation/heap) fragmentation.
- Botching the math: 4 KB = 2^{12} → 12 offset bits; know $2^{10}=1\text{K}$, $2^{20}=1\text{M}$, $2^{30}=1\text{G}$ cold.

9. Performance Implications

Page size trade-off: bigger pages → fewer TLB misses, smaller tables, but more internal fragmentation and slower/coarser I/O; 4 KB default, 2 MB huge pages for big heaps (Transparent Huge Pages can help or hurt — khugepaged stalls, why some DBs say disable THP).

10–11. Interview & Follow-ups

- “64-bit VA, 4 KB pages: how many offset bits? How big would a flat page table be?” (leads to multi-level)
- “Why did huge pages help your database?” (TLB reach: 1536 entries × 4 KB = 6 MB vs × 2 MB = 3 GB)

12. Coding/Debugging Scenario

`perf stat -e dTLB-load-misses` high on a 100 GB in-memory store → enable explicit huge pages; verify miss-rate and p99 drop.

13. Best Practices

Align hot data structures to pages; consider huge pages past ~10 GB heaps; know your platform’s THP policy.

14. Practice Questions

1. 48-bit VA, 4 KB pages, 8-byte PTEs — size of a flat page table? (2^{36} PTEs × 8 B = 512 GB → motivates multi-level)
2. Compute PA for VA 0x7FFF12345 given a tiny page table you invent.

5.3 Segmentation

1. Why Interviewers Ask This

Mostly for contrast with paging and to explain the *meaning* of “segmentation fault”. Keep it brief and comparative.

2. Core Concept

Divide the address space into variable-length **segments** with semantic meaning (code, data, stack, heap), each with base + limit + permissions. Translation: segment base + offset, bounds-checked.

3. Internal Working

Segment table holds (base, limit, perms). VA = (segment selector, offset); hardware checks offset < limit then adds base. Variable sizes → **external fragmentation** (free RAM in useless gaps) → compaction needed. x86 had real segmentation; x86-64 effectively flattens it (bases 0) and uses paging — %fs/%gs survive for TLS.

4. ASCII Diagram

```
Segment table:      Physical memory:
code: base 0x1000 lim 4K |code (4K)| gap |heap(16K)| gap |stack(8K)|
heap: base 0x8000 lim 16K      ^ external fragmentation ^
stack: base 0xF000 lim 8K
VA (heap, 0x123) -> check 0x123 < 16K -> PA 0x8123
Offset >= limit -> segmentation fault (the historical name)
```

5. Real Production Example

Modern relevance: the *concept* survives as VMAs/memory regions (`/proc/maps` shows code/heap/stack/mmap regions with permissions — “paged segmentation” in spirit); “segfault” = access outside any valid region.

6. Advantages

Semantic protection units; natural sharing of whole segments; grows to fit logical structure.

7. Trade-offs

External fragmentation + compaction; complex allocation (best/first fit); segments limited in number.

8. Common Mistakes

- Claiming modern x86-64 “uses segmentation” for isolation (it’s paging; segmentation is vestigial).
- Not knowing why the fault is *named* segmentation fault.

9. Performance Implications

Historical; the enduring lesson is variable-size allocation → external fragmentation — the same math that governs heap allocators (5.9).

10–11. Interview & Follow-ups

- “Paging vs segmentation — table of differences.” “What actually happens on a segfault today?” (page-level permission/present violation → SIGSEGV)

12. Coding/Debugging Scenario

`cat /proc/<pid>/maps` of a crashed process + the faulting address from `dmesg` → determine which region boundary was violated (NULL page? stack guard? freed mmap?).

13. Best Practices

Use the paging-vs-segmentation contrast to *show* you understand fragmentation types — that’s its interview job.

14. Practice Questions

1. Fill the table: fragmentation type, size unit, protection granularity, modern usage — paging vs segmentation.
2. Why does the kernel place a guard gap below the stack VMA?

5.4 TLB (Translation Lookaside Buffer)

1. Why Interviewers Ask This

The TLB is why virtual memory is *fast enough to be free-ish*, and TLB reasoning explains real performance cliffs (context switches, huge pages, random access).

2. Core Concept

The TLB is a small, very fast cache of recent VPN → PFN translations inside the MMU. Hit: translation in <1 cycle. Miss: hardware walks the page table (~tens–100+ ns), then caches the entry.

3. Internal Working

- Typical: L1 dTLB/iTLB 64–128 entries, L2 STLB ~1.5–2K entries, per core.
- **Reach:** 1536 entries × 4 KB ≈ 6 MB — touch more than that randomly and you miss constantly; 2 MB huge pages raise reach to ~3 GB.
- Context switch to another address space: TLB entries invalid → flush, unless entries are tagged (**PCID/ASID**) — the hidden cost of process switches (Module 1).
- **TLB shutdown:** when one core changes a mapping (munmap, COW break, page migration), it must IPI all cores that may cache the entry — expensive, scales badly with core count (why frequent mmap/munmap hurts multithreaded services).

4. ASCII Diagram

```
CPU: VA -> [TLB lookup]
      hit (99%+): PA in <1 cycle
      miss: [HW page walk: PGD->PUD->PMD->PTE, up to 4 mem reads]
            -> install in TLB -> PA
Reach: 4KB pages: ~6MB | 2MB huge pages: ~3GB
Shutdown: core0 munmap -> IPI-> core1..N flush entry (stalls all)
```

5. Real Production Example

- Databases enabling huge pages for buffer pools (Postgres, Oracle) — measured double-digit gains on TLB-bound workloads.
- Hash-join/random-pointer-chasing analytics: TLB misses dominate; sorted/partitioned access patterns fix it.
- glibc `MALLOC_ARENA` churn or allocators doing frequent `munmap` → shutdown storms in many-threaded services.

6. Advantages

Makes paging ~free for hot working sets; transparent.

7. Trade-offs

Limited reach; flush costs on switches; shutdowns serialize cores; another layer of “works until it cliffs”.

8. Common Mistakes

- Explaining paging with zero mention of the TLB (translation would be 2–5× slowdowns otherwise).
- Not knowing shutdowns exist (senior differentiator).

9. Performance Implications

Random access across >TLB-reach memory: every access ≈ +100 ns walk → can halve throughput. Measure with `perf stat -e dTLB-load-misses,dtlb_load_misses.walk_duration`.

10–11. Interview & Follow-ups

- “What makes virtual memory fast despite per-access translation?” “Why do huge pages help?” “What is a TLB shutdown and when does it bite?”

12. Coding/Debugging Scenario

In-memory graph traversal 3× slower than back-of-envelope: perf shows 30% cycles in page walks → switch node pool to 2 MB huge pages; misses collapse.

13. Best Practices

Favor locality (arrays over pointer soup); huge pages for large hot heaps; avoid mmap/munmap churn on hot paths (pool your mappings).

14. Practice Questions

1. Compute TLB reach for 1024 entries with 4 KB vs 2 MB pages; relate to a 50 GB working set.
2. Why does a process context switch cost more than the register save/restore suggests? (TLB/cache refill)

5.5 Page Tables & 5.6 Multi-Level Page Tables

1. Why Interviewers Ask This

“Why multi-level?” is a top-5 OS interview question with a beautiful quantitative answer (sparse address spaces), and the 4-level walk is the standard “explain translation end-to-end” test.

2. Core Concept

- Page table: per-process map VPN → PFN + flags (present, R/W, U/K, NX, accessed, dirty).
- Flat table for 48-bit VA at 4 KB pages = 2^{36} entries × 8 B = **512 GB per process** — impossible.
- **Multi-level** tables are a radix tree: allocate only the branches that map something. Sparse address spaces (typical: few hundred MB used of 128 TB) → tables cost only ~MBs.

3. Internal Working

x86-64 4-level (PGD → PUD → PMD → PTE): 48-bit VA = 9+9+9+9 index bits + 12 offset. CR3 holds the root’s physical address. Hardware walker: CR3 → PGD[i1] → PUD[i2] → PMD[i3] → PTE[i4] → frame. Any non-present entry at any level cuts the whole subtree (unmapped 512 GB costs one empty PGD slot). PMD-level “huge” entry = 2 MB page (skips one level). 5-level paging (57-bit VA) exists for huge-memory machines. Each level’s tables are 4 KB (512 × 8 B entries).

4. ASCII Diagram

```
VA (48-bit): | 9b PGD | 9b PUD | 9b PMD | 9b PTE | 12b offset |
CR3 -> [PGD] --i1--> [PUD] --i2--> [PMD] --i3--> [PTE] --i4--> frame
           512e           512e           512e           512e
Sparse win: unmapped region => NULL at top level, no lower tables exist.
Cost of a TLB miss: up to 4 dependent memory reads (~100+ ns).
PTE flags: [P|RW|US|A|D|NX| PFN.....]
```

5. Real Production Example

Every Linux/Windows process. Fork copies page-table structure (not data pages — COW); `fork()` of a huge-RSS Redis is dominated by copying these tables (a 60 GB RSS ≈ ~120 MB of page tables to copy → tens of ms pause).

6. Advantages

Memory ∝ *mapped* space, not address-space size; hierarchical permissions; hardware-walkable; natural huge-page support.

7. Trade-offs

Walk latency multiplies at each level (mitigated by TLB + partial-walk caches); table updates need TLB shutdowns; deep trees complicate the kernel’s mm code.

8. Common Mistakes

- Can't do the 512 GB flat-table math (memorize: 48-bit, 4 KB, 8 B PTE $\Rightarrow 2^{36} \times 8 = 512$ GB).
- Saying page tables live in the process's virtual memory (they're kernel-managed physical structures; the process never sees them).
- Forgetting the *dirty/accessed* bits — they drive page replacement (Module 6).

9. Performance Implications

TLB miss cost = up to 4 (or 5) dependent loads; huge pages remove one level *and* multiply reach; fork cost scales with mapped memory (why big processes prefer `posix_spawn`/`vfork` semantics).

10. Common Interview Questions

- “Why multi-level page tables? Do the math.” “Walk me through translating one address on x86-64.”

11. Follow-up Questions

- “What are inverted page tables?” (global frame $\rightarrow$ (pid,vpn) hash — memory $\propto$ RAM not VA; used by PowerPC historically)
- “How does the kernel find which processes map a physical page?” (reverse mapping — `rmap`; needed for eviction)

12. Coding/Debugging Scenarios

- Redis latency spike every BGSAVE $\rightarrow$ fork page-table copy; mitigations: huge pages reduce PTE count (careful: THP+fork COW amplification!), or diskless replication.

13. Best Practices

Know your per-level math cold; mention accessed/dirty bits and shutdowns to show depth.

14. Practice Questions

1. For VA 0x00007F1234ABC987, compute the four 9-bit indices (hex ok).
2. A process maps 1 TB contiguously with 4 KB pages: how much page-table memory? (~2 GB PTEs + ~4 MB PMDs + ...) With 2 MB pages? (~4 MB)

5.7 Demand Paging

1. Why Interviewers Ask This

Explains program startup, `mmap`, lazy allocation, and the minor/major fault distinction that shows up in every perf investigation.

2. Core Concept

Pages are loaded/allocated **only on first access**, not upfront. `exec` maps the binary without reading it; `malloc` returns address space without RAM; touching a page triggers a fault that materializes it.

3. Internal Working

1. Access to non-present page $\rightarrow$ CPU raises page fault $\rightarrow$ kernel handler.
2. Kernel checks the VMA: valid mapping? - **Minor fault**: page already in RAM (page cache, zero page, COW) — just map it (~ μ s). - **Major fault**: read from disk (binary, swap, mapped file) — schedule I/O, block the task (μ s–ms). - No valid VMA / bad perms $\rightarrow$ SIGSEGV.
3. Update PTE, resume the instruction transparently. Optimizations: readahead (fault-around), zero-page sharing for untouched anonymous memory.

4. ASCII Diagram

```
touch page --> fault --> VMA valid?
                        |-- no --> SIGSEGV (bug)
                        |-- yes, in page cache --> minor fault: map (fast)
                        |-- yes, on disk/swap --> major fault: I/O, block (slow)
Startup of a 2GB binary: maps in ms; pages fault in as code paths execute.
```

5. Real Production Example

- Fast container/process cold starts (only touched pages load).
- `ps -o min_flt,maj_flt; sar -B` majflt/s as a thrashing signal.
- ML serving: first inference is slow because weights fault in from disk — fixed by pre-touch/`mlock`/warmup requests.

6. Advantages

Fast startup, memory efficiency (unused features never load), enables overcommit and huge sparse mappings.

7. Trade-offs

First-touch latency cliffs; unpredictable performance until warm; page-fault storms under memory pressure (thrashing).

8. Common Mistakes

- Not distinguishing minor vs major faults (interviewers probe this exactly).
- Treating page faults as errors — most are the design working as intended.

9. Performance Implications

Minor ~1–5 μ s; major ~100 μ s (NVMe) to ~10 ms (HDD). p99-sensitive services `mlock` hot data or pre-touch after deploy; majflt/s > ~0 sustained on a latency-critical box is an incident.

10–11. Interview & Follow-ups

- “What happens on `malloc(1GB)` — when is RAM actually used?” “Minor vs major fault?” “Why was the first request after deploy slow?”

12. Coding/Debugging Scenario

Post-deploy p99 spike for 2 minutes: `sar -B` shows major faults → warm-up: sequentially touch the model/index files (`vmtouch`, `madvise(WILLNEED)`), or `mlock`.

13. Best Practices

Warm critical paths before taking traffic; `mlockall` for hard-latency processes (with care); watch majflt/s per service.

14. Practice Questions

1. `memset(malloc(1<<30), 0, 1<<30)`: describe every fault and when RSS grows.
2. Why can reading a fresh `mmap`ed file be slower than `read()` for a single sequential pass? (per-page faults vs batched copies; readahead interplay)

5.8 Copy-on-Write (COW)

1. Why Interviewers Ask This

“How does `fork()` copy 60 GB instantly?” is a top interview question, and COW connects `fork`, Redis, and memory spikes into one story.

2. Core Concept

Instead of copying memory at `fork()`, parent and child **share all pages read-only**; the first write by either side faults, and the kernel copies **only that page** (then makes both writable). Copy cost is deferred and proportional to *pages actually written*.

3. Internal Working

1. `fork()`: copy page *tables*, mark both sides' PTEs read-only, bump page refcounts.
2. Write → protection fault → kernel sees COW mapping → allocate frame, `memcpy` 4 KB, remap writer's PTE writable, decrement refcount (last owner just gets it back writable — no copy).
3. TLB shutdown for the changed PTE. Also used for: `mmap(MAP_PRIVATE)` file mappings, KSM (same-content page merging), zygote-style app forking (Android), and the zero page.

4. ASCII Diagram

```
fork():
Parent PTEs -----> [frame X (R0)] <----- Child PTEs   refcount=2
Child writes page:
  fault -> copy X -> X' ; child PTE -> X' (RW); parent PTE -> X (RW when last)
Memory cost after fork = pages *written*, not total size.
Redis 60GB fork: instant; but heavy writes during BGSAVE => up to 2x RAM!
```

5. Real Production Example

- **Redis BGSAVE/AOF-rewrite**: fork child snapshots memory; COW keeps it cheap — but a write-heavy period during the save can double resident memory (classic prod incident + interview scenario).
- Android Zygote: preloaded framework pages shared COW across all app processes.
- `fork+exec` for shells/servers: near-zero copy since `exec` replaces the space anyway.

6. Advantages

O(page-tables) fork; memory dedup; snapshot semantics for free; enables fork-based concurrency patterns.

7. Trade-offs

Write-fault latency blips; memory usage becomes workload-dependent and *spiky* (COW break storms); refcounting/rmap complexity; THP amplifies COW cost (2 MB copies).

8. Common Mistakes

- “Fork copies all memory” (pre-1980s answer).
- Forgetting the worst case: fork + write-everything = 2× memory → OOM (must size Redis boxes for it).
- Missing that only the *writer's* side copies; reader keeps the original.

9. Performance Implications

COW break = fault + 4 KB copy + shutdown (~μs). A snapshotting child pins old pages: long saves + hot writes = memory balloon; monitor `fork` duration and RSS during BGSAVE.

10–11. Interview & Follow-ups

- “Explain how Redis snapshots 60 GB without doubling memory — and when it *does* double.”
- “What exactly is copied at fork time?” (page tables + task metadata) “How does COW interact with huge pages?”

12. Coding/Debugging Scenario

Redis OOM-killed during BGSAVE on a 64 GB box with 40 GB dataset → COW inflation under write load; fixes: replica-based persistence, more headroom, `vm.overcommit_memory=1` (Redis's own recommendation), schedule saves off-peak.

13. Best Practices

Budget worst-case COW memory for fork-snapshot systems; keep snapshot windows short; prefer `posix_spawn` for spawn-only use.

14. Practice Questions

1. Parent has 10 GB RSS, forks, child reads everything, parent rewrites 1 GB: final memory footprint?
2. Why is `vfork/posix_spawn` still faster than COW fork? (skips even page-table copy)

5.9 Memory Allocation (malloc & friends)

1. Why Interviewers Ask This

Bridges OS and everyday engineering: “what happens on malloc?” tests the user-allocator vs kernel boundary; fragmentation questions test long-running-service judgment.

2. Core Concept

Two layers: the **kernel** hands out address space in pages (`brk/sbrk` for the heap end, `mmap` for large blocks); the **user-space allocator** (glibc `ptmalloc`, `jemalloc`, `tcmalloc`, `mimalloc`) slices pages into objects with free lists/size classes. `malloc` is a library call, *not* a syscall (usually).

3. Internal Working

- Small allocs: size-class bins + per-thread caches (tcache/arenas) → no lock on the fast path, ~10–20 ns.
- Large allocs (glibc ≥ 128 KB default): direct `mmap`, returned to OS on free.
- `free` returns to bins; heap top can shrink via `brk` only if the top is free → **freed ≠ returned to OS** (RSS stays high — the classic “leak that isn’t”).
- Fragmentation: **external** (free space split into unusable gaps — allocator-level now, since paging fixed it at the physical level) and **internal** (size-class rounding waste).
- Kernel side on first touch: page fault → zeroed frame (demand paging as above).

4. ASCII Diagram

```
malloc(64B):
  thread cache bin[64] pop --> pointer (ns, no syscall)
  bin empty -> refill from central arena -> maybe mmap/brk new pages
malloc(10MB):
  mmap anonymous pages --> returned directly; freed via munmap

Heap fragmentation:
|used|free 24B|used|free 40B|used|   malloc(48B) FAILS to fit gaps
-> allocator grows heap despite 64B "free" (external fragmentation)
```

5. Real Production Example

- Long-running services switching glibc → jemalloc/tcmalloc for fragmentation and multi-thread scaling (Redis ships jemalloc; Meta/Google run their allocators fleet-wide).
- “RSS climbs forever but heap profiler shows constant live set” → fragmentation + free-but-not-returned pages; jemalloc `background_thread/muzzy_decay` tuning.

6. Advantages

User-space fast paths avoid syscalls; size classes bound external fragmentation; per-thread caches scale.

7. Trade-offs

Size classes waste memory (internal frag up to ~25%); per-thread caches hold memory hostage; allocator behavior varies wildly across implementations (portability of performance).

8. Common Mistakes

- “malloc is a system call.”
- Interpreting stable-RSS-after-free as a leak; or actual leaks as “fragmentation”.
- Ignoring allocation in hot loops (allocation is cheap; *cache misses and contention it causes* are not).

9. Performance Implications

Fast path ~ns; arena contention with many threads (glibc arenas vs tcmalloc per-CPU caches); huge-alloc mmap/munmap churn → page faults + TLB shootdowns (pool them).

10–11. Interview & Follow-ups

- “Walk through `malloc(100)` end-to-end, including when the kernel gets involved.”
- “Why doesn’t RSS drop after free?” “brk vs mmap allocations?” “Why jemalloc?”

12. Coding/Debugging Scenario

RSS grows 1%/day, heap profiler flat → fragmentation: check allocator stats (`malloc_stats`, `je_malloc_stats_print`); fix with jemalloc, arena caps, or object pooling for the offending size mix.

13. Best Practices

Pool/reuse hot-path objects; pick a modern allocator for multithreaded services; expose allocator stats in metrics; avoid mixed-lifetime allocations sharing regions (fragmentation fuel).

14. Practice Questions

1. Explain each transition: `malloc(100)` → tcache → arena → mmap/brk → page fault → zeroed frame.
2. Design an object pool for 4 KB request buffers — what fragmentation/waste do you eliminate?

5.10 Heap vs Stack

1. Why Interviewers Ask This

Deceptively basic; seniors are graded on the *why*: allocation cost mechanics, lifetime rules, overflow behavior, thread interactions.

2. Core Concept

- **Stack**: per-thread LIFO region for frames (locals, args, return addresses). Allocation = bump SP (one instruction); free = automatic at return. Fixed max size (default 8 MB Linux threads).
- **Heap**: process-wide region for dynamic, arbitrary-lifetime data via allocator. Slower, must be managed (free/GC), but sized by RAM not by a per-thread cap.

3. Internal Working

- Stack grows downward; guard page below → overflow = SIGSEGV on guard touch (usually). Frames: saved return address, saved regs, locals — why stack smashing is a security class (canaries, NX).
- Heap: allocator on top of brk/mmap (5.9). Escape analysis (Java/Go) moves would-be heap objects to the stack; Go stacks are growable (start ~2–8 KB, copied on growth) — contrast with fixed pthread stacks.

4. ASCII Diagram

```

High addr +-----+
          | stack T1 (8MB) | grows v [guard page]
          | stack T2 (8MB) | grows v [guard page]
          | ...mmap...    |
          | heap          | grows ^ via allocator
          | bss / data    |
Low addr  | code (R0,X)   |
          +-----+
Stack alloc: sub rsp, N (1 insn)  Heap alloc: allocator logic (ns-µs)

```

5. Real Production Example

- Stack overflow from deep recursion (JSON parsers on hostile input!) → crash; iterative rewrite or bigger stack.
- 10k-thread server: 10k × 8 MB virtual stacks (fine) but touched pages accumulate → tune stack size down.
- GC languages: allocation-rate-driven GC pauses → reduce heap churn (object reuse), rely on escape analysis.

6. Advantages

Stack: fastest possible alloc/free, cache-hot, no fragmentation, automatic lifetime. Heap: flexible size/lifetime, shareable across threads/functions.

7. Trade-offs

Stack: fixed limit, frame lifetime only (dangling pointers if escaped!), per-thread cost. Heap: allocator overhead, fragmentation, leaks/GC pressure, synchronization in allocator.

8. Common Mistakes

- Returning a pointer to a stack local (C/C++ classic).
- “Stack is in the CPU / heap is in RAM” — both are ordinary RAM regions.
- Not knowing default stack sizes or that thread stacks are just mmaps.

9. Performance Implications

Stack alloc ~0 cost and prefetch-friendly; heap alloc ~ns plus later cache misses; allocation-heavy hot paths often gain 2–10× from stack/arena conversion; GC cost ∝ allocation rate more than heap size (generational).

10–11. Interview & Follow-ups

- “Where do locals/globals/new objects/string literals live?” “What happens physically on stack overflow?” “How do goroutine stacks differ from pthread stacks?” (growable, movable → why C interop needs care)

12. Coding/Debugging Scenario

SIGSEGV with fault address just below the stack VMA → stack overflow, not a wild pointer; confirm via core dump RSP vs stack bounds; fix recursion or `ulimit -s/pthread_attr_setstacksize`.

13. Best Practices

Prefer stack/value types on hot paths; bound recursion on untrusted input; size thread stacks intentionally at high thread counts; profile allocation rate, not just heap size.

14. Practice Questions

1. For a C program: classify 8 variables (global, static local, local, malloc'd, literal, argv) into segments.
2. Why is `alloca`/VLA dangerous in servers? (unbounded stack use, no failure signal until SIGSEGV)

Module 5 Cheat Sheet (one page)

Numbers to know: page 4 KB (2^{12}); x86-64 4-level walk (9|9|9|9|12); flat table would be 512 GB; TLB ~1.5K entries → ~6 MB reach (4 KB) / ~3 GB (2 MB pages); minor fault μ s, major fault 100 μ s–10 ms; default pthread stack 8 MB.

Concept	One-liner	Interview hook
Virtual memory	Private per-process space, MMU-translated	RSS vs VSZ; overcommit → OOM killer
Paging	Fixed 4 KB pages ↔ frames	kills external frag, keeps internal
Segmentation	Variable semantic regions	external frag; why “segfault” is named so
TLB	Translation cache	reach math; shootdowns; PCID
Multi-level PT	Radix tree, pay for what you map	do the 512 GB math
Demand paging	Materialize on first touch	minor vs major faults; warmup
COW	Share RO at fork, copy on write	Redis BGSAVE 2× memory risk
malloc	Library size-classes over brk/mmap	freed ≠ returned; fragmentation vs leak
Stack vs heap	Bump-SP vs allocator	overflow mechanics; escape analysis

Fault decision tree: no VMA → SIGSEGV · in page cache/zero/COW → minor · on disk/swap → major.

Top Interview Questions

1. What happens when you dereference a pointer — full path including TLB, walk, faults.
2. Why multi-level page tables? (do the math) Walk one translation.
3. How does fork copy 60 GB instantly, and when does Redis double its memory?
4. malloc end-to-end; why doesn't RSS drop after free?
5. Minor vs major page fault; why was the first request slow?
6. Huge pages: why do they help (TLB reach) and when do they hurt (THP stalls, COW amplification)?
7. Stack vs heap: allocation mechanics, overflow, lifetimes.

Common Mistakes (module-wide)

- VSZ as usage; “virtual memory = swap”; “malloc is a syscall”.
- Explaining translation without the TLB; missing dirty/accessed bits.
- Fragmentation vs leak confusion; forgetting COW's 2× worst case.
- No minor/major fault distinction; returning stack pointers.

Mock Interview (self-test, ~25 min)

1. (Depth) From `mov rax, [rbx]` to data: enumerate every hardware/OS step for TLB hit, TLB miss, minor fault, major fault, and segfault paths.
2. (Math) 48-bit VA, 4 KB pages, 8 B PTEs: flat table size; then page-table memory for a process mapping 8 GB densely; then with 2 MB pages.
3. (Prod) Redis 45 GB on a 64 GB host is OOM-killed at 03:00 daily during BGSAVE. Explain mechanism and give three fixes with trade-offs.
4. (Prod) A service's RSS grows forever; heap profiler says live set flat. Give your differential diagnosis tree (fragmentation, freed-not-returned, native/mmap leaks, page cache accounting) and the measurement for each.
5. (Trap) “We disabled swap, so page faults can't happen.” Correct them precisely (file-backed pages, demand paging, page-cache eviction still fault).

MODULE 6 — Memory (Page) Replacement

Senior SWE Interview Track — Operating Systems

6.0 Why Replacement Exists (framing)

When RAM is full and a page must come in, the kernel evicts a victim. The same math governs **every cache you will ever build** — CPU caches, page cache, Redis **maxmemory-policy**, CDN eviction — which is why interviewers love this module: it's OS theory you use weekly. Goal: minimize fault/miss rate; the unbeatable oracle is **OPT/Belady's algorithm** (evict the page used farthest in the future) — unimplementable, but the benchmark. Clean pages are cheap to evict (drop); dirty pages must be written back first (the dirty bit from Module 5 pays off here).

6.1 FIFO

1. Why Interviewers Ask This

Baseline + the setup for **Belady's anomaly**. Also a hand-simulation favorite.

2. Core Concept

Evict the page that has been *resident longest*, regardless of use. Queue: evict head, insert at tail.

3. Internal Working

Ring buffer/queue of frames; $O(1)$ eviction; no per-access bookkeeping (nothing updates on a hit — that's both its cheapness and its flaw).

4. ASCII Diagram

```
Ref string: 1 2 3 4 1 2 5 1 2 3 4 5      3 frames
[1][ ][ ] F  [1][2][ ][ ] F  [1][2][3] F  [4][2][3] F  (evict 1)
[4][1][3] F (evict 2) ... total = 9 faults
Flaw: evicts "oldest" even if it's the hottest page.
```

5. Real Production Example

Simple firmware/embedded caches; message-buffer recycling; anywhere metadata budget is ~zero. Rarely acceptable for real page caches.

6. Advantages

Trivial, $O(1)$, no hit-path overhead, no scan cost.

7. Trade-offs

Ignores recency/frequency entirely → evicts hot pages; suffers **Belady's anomaly** (more frames can mean *more* faults — 6.5).

8. Common Mistakes

- Updating the queue on hits (that turns it into LRU — in FIFO, hits do nothing).
- Sloppy hand simulation: mark hit/fault per step, track queue order explicitly.

9. Performance Implications

Typically 20–50% worse fault rate than LRU on workloads with locality; fine when accesses are uniform/scan-like (no locality to exploit).

10–11. Interview & Follow-ups

- “Simulate this reference string with 3 frames.” “What’s FIFO’s core flaw?” “Which algorithms are anomaly-free?” (stack algorithms: LRU, OPT)

12. Coding/Debugging Scenario

Implement FIFO cache in 10 lines (deque + set); use as the control in an eviction A/B benchmark.

13. Best Practices

Use FIFO only when hit-path cost must be zero or access patterns lack locality; otherwise start at LRU/Clock.

14. Practice Questions

1. Count faults: `7 0 1 2 0 3 0 4 2 3 0 3 2` with 3 frames (answer: 9... verify by hand).
2. Construct a workload where FIFO beats LRU. (looping scan slightly larger than cache — LRU thrashes at 0% hit, FIFO too; sequential one-shot scans — tie; adversarial LRU patterns)

6.2 LRU (Least Recently Used)

1. Why Interviewers Ask This

Double-duty: OS theory **and** the #1 data-structure design question (“Design an LRU cache” — LeetCode 146, asked constantly at Meta/Amazon). Also probes why true LRU is infeasible in hardware.

2. Core Concept

Evict the page unused for the longest time — exploits **temporal locality** (recent past predicts near future). It’s OPT’s mirror: OPT looks forward, LRU looks backward.

3. Internal Working

- **Software (the interview classic)**: hash map (key → node) + doubly-linked list in recency order. Hit: move node to front, $O(1)$. Evict: remove tail, $O(1)$.
- **Why the OS can’t do true LRU**: it would need to timestamp/reorder on *every memory access* — hardware only gives a per-page **accessed bit**. So kernels approximate (Clock, 6.4; Linux uses active/inactive lists, and MGLRU in newer kernels).
- LRU is a **stack algorithm** (contents with k frames $\subseteq$ contents with $k+1$) → immune to Belady’s anomaly.

4. ASCII Diagram

```
HashMap: k -> node          Doubly-linked list (MRU ... LRU)
get(k):  lookup node, unlink, insert at head      O(1)
put(k,v): if full: evict tail (also remove from map); insert head

head [k4] <-> [k9] <-> [k2] <-> [k7] tail  <- evict k7
Same ref string as FIFO example: LRU = 10 faults there; usually LRU wins
on real (locality-heavy) traces, not always on adversarial ones.
```

5. Real Production Example

- Redis `maxmemory-policy allkeys-lru` (actually *approximate*: samples 5 random keys, evicts oldest — near-LRU at a fraction of the cost; a great “theory vs practice” talking point).
- MySQL InnoDB buffer pool: LRU with a **midpoint insertion** (new pages enter 3/8 from the tail) so table scans can’t flush the hot set — “scan resistance”.
- CDN/memcached eviction; CPU caches use pseudo-LRU bits.

6. Advantages

Best simple predictor under temporal locality; $O(1)$ software implementation; anomaly-free; intuitive to reason about.

7. Trade-offs

Per-access bookkeeping (list ops + locking → contention in concurrent caches); zero **frequency** awareness (one touch of a cold page makes it “hot”); catastrophically bad on sequential scans slightly larger than the cache (loop of $N+1$ pages over N frames → 100% miss).

8. Common Mistakes

- LRU-cache coding slips: forgetting to remove evicted key from the map; not moving to front on **put** of existing key; $O(n)$ list search instead of storing node refs in the map.
- Claiming OSes implement true LRU.
- Not knowing the scan-pollution weakness (leads into LFU/2Q/midpoint discussions).

9. Performance Implications

Software hit path $\sim O(1)$ but a shared lock on the list serializes; sharded/segmented LRU or per-CPU structures fix it. Fault-rate: excellent with locality; degenerate on loops/scans (mention MRU as the fix for pure loops).

10. Common Interview Questions

- “Design LRU cache with $O(1)$ get/put.” (write it cold)
- “Why can’t the kernel use true LRU?” “How does Redis approximate LRU?”

11. Follow-up Questions

- “Make your LRU thread-safe — now make it scale.” (lock striping, per-shard LRU)
- “How does InnoDB stop a full-table scan from evicting the working set?” (midpoint LRU)
- “LRU-K / 2Q / ARC — what do they add?” (frequency/scan resistance)

12. Coding/Debugging Scenarios

- Cache hit rate collapses every night at 02:00 → batch job scans a huge table through the same cache → pollution; fix: scan-resistant policy, separate cache, or bypass hints (**MADV_DONTNEED**-style).

13. Best Practices

Segment or midpoint-insert for scan resistance; sample-based approximation at large scale; measure hit ratio *per class of traffic*.

14. Practice Questions

1. Implement LRU cache (map + DLL) in your language, then add TTL.
2. Simulate loop 1..4 repeatedly with 3 frames under LRU vs MRU — fault rates?
3. Sketch sharded LRU for 10M entries, 32 threads.

6.3 LFU (Least Frequently Used)

1. Why Interviewers Ask This

The frequency-vs-recency trade-off, plus a harder design variant (“Design LFU cache”, LeetCode 460) that filters senior candidates.

2. Core Concept

Evict the page with the *fewest* accesses. Captures long-term popularity that LRU misses (a page touched 1000 times shouldn’t die because of a 10-second quiet spell).

3. Internal Working

- Naive: counter per page; evict min — $O(n)$ scan or heap $O(\log n)$.

- O(1) design: hash key → node; nodes grouped in **frequency buckets** (DLL per frequency, frequencies in a list); on access, move node to freq+1 bucket; evict from the lowest non-empty bucket (LRU within it for ties).
- **Aging is mandatory**: raw counters never forget — yesterday's viral object blocks today's. Fixes: periodic halving of counters, or windowed/decayed counts. Redis LFU: 8-bit *probabilistic* counter (increments with probability decreasing as count grows) + configurable decay time (**lfu-decay-time**).
- Modern practice: **TinyLFU/W-TinyLFU** (Caffeine's policy) — a tiny count-min sketch as an *admission filter* in front of a windowed LRU: only admit a new key if it's likely more popular than the eviction victim.

4. ASCII Diagram

```
freq buckets: [1: c->a] [2: d] [5: b]
access(a): move a from bucket1 to bucket2
evict: head of lowest bucket -> c
Without decay: old celebrity item at freq=50000 is immortal
Decay: every T, counter /= 2 -> popularity has a half-life
```

5. Real Production Example

- Redis **allkeys-lfu** (recommended over LRU for skewed key popularity).
- Caffeine (W-TinyLFU) — default high-performance Java cache, near-optimal hit rates in benchmarks; used across ad-tech/serving layers.
- CDNs: admission-controlled frequency caches so one-hit-wonders (~60–70% of CDN objects!) never displace hot content.

6. Advantages

Superior for skewed, stable popularity (Zipf traffic — i.e., most internet workloads); scan/one-hit-wonder resistant (new items start at freq 1).

7. Trade-offs

Cache pollution by *stale* frequency without decay; new hot items ramp slowly (cold-start penalty); heavier bookkeeping; the pure-LFU O(1) structure is fiddly.

8. Common Mistakes

- Proposing LFU without decay/aging — interviewers pounce on the “old viral video” problem.
- Ignoring tie-breaking (use LRU within a frequency bucket).
- Not knowing the sketch-based modern form (TinyLFU) — the senior-level upgrade.

9. Performance Implications

On Zipfian traces LFU/TinyLFU beat LRU hit rates by several points (huge at CDN scale: +1% hit rate = big origin-traffic cut); on shifting working sets, undecayed LFU is the *worst* of the classic policies.

10–11. Interview & Follow-ups

- “LRU vs LFU — when does each win?” “Design an O(1) LFU.” “How does Redis implement LFU in 8 bits?” “What is TinyLFU admission?”

12. Coding/Debugging Scenario

CDN node keeps evicting current-hour hot objects while month-old assets sit cached → frequency staleness; add decay or switch to W-TinyLFU-style admission + windowed LRU.

13. Best Practices

Always pair frequency with decay; use admission filters for one-hit-wonder-heavy traffic; prefer proven libraries (Caffeine) over hand-rolled LFU.

14. Practice Questions

1. Implement O(1) LFU (get/put) with frequency buckets.

- Given Zipf(1.0) traffic and a cache 1% of the corpus, argue which policy wins and why.
- Design decay: half-life 1 hour, 8-bit counters — sketch the update rule.

6.4 Clock Algorithm (Second Chance)

1. Why Interviewers Ask This

“How does the OS *actually* approximate LRU?” — Clock is the canonical answer, and it tests whether you know what the hardware accessed-bit gives you.

2. Core Concept

Frames arranged in a circle with a hand. Each page has a hardware-set **reference (accessed) bit**. To evict: advance the hand; if ref bit = 1, clear it and move on (**second chance**); if 0, evict. Recently used pages survive one sweep; untouched pages don't.

3. Internal Working

- Hardware sets the accessed bit on any read/write to the page (in the PTE); the kernel clears it — no per-access software cost (the whole point).
- Enhanced (NRU-style) variant uses (ref, dirty) pairs: prefer evicting (0,0) > (0,1) > (1,0) > (1,1) — clean-and-cold first, since dirty pages need writeback.
- Real kernels elaborate: Linux classic active/inactive lists = a two-tier clock (accessed pages promote; eviction from inactive tail; refault tracking to size the tiers); MGLRU (recent kernels) generalizes to multiple generations. FreeBSD/Mach used explicit two-handed clocks (front hand clears, back hand evicts; hand spread tunes the “window”).

4. ASCII Diagram

```

    [P3 r=1]
[P2 r=0]   [P4 r=0]
  HAND--> ^
[P1 r=1]   [P5 r=1]
    [P0 r=0]
Evict pass: P4 r=0 -> EVICT. If it were r=1: clear, advance.
Worst case: all bits 1 -> full lap (clears all) -> degrades to FIFO.

```

5. Real Production Example

Linux page-cache/anon reclaim (kswapd walks LRU lists driven by accessed bits — clock in spirit); PostgreSQL's buffer pool uses **clock-sweep** with a usage counter (0–5) — a multi-bit clock; hypervisors sample accessed bits for working-set estimation.

6. Advantages

Near-LRU hit rates at ~zero hit-path cost; O(1) amortized eviction; hardware does the tracking; no list reordering (no lock contention on access).

7. Trade-offs

Coarse (1 bit ≈ “touched since last sweep?”); sweep cost under pressure; degenerates to FIFO when everything's referenced; eviction quality depends on sweep frequency vs access rate.

8. Common Mistakes

- Not knowing the accessed bit is set by *hardware* (candidates invent per-access software hooks — that's exactly what Clock avoids).
- Forgetting the dirty-bit interaction (evicting dirty pages costs a writeback → prefer clean victims, hence enhanced clock / background flusher).

9. Performance Implications

Approximates LRU within a few percent on typical workloads at ~0 overhead — the textbook engineering trade. Under extreme memory pressure the hand spins fast (visible as kswapd CPU + `pgscan/pgsteal` rates) — a real triage signal.

10–11. Interview & Follow-ups

- “Explain second chance.” “What does Linux actually do?” (active/inactive or MGLRU — say “clock-family approximation of LRU with accessed bits”)
- “What happens when every page has ref=1?” “How does the dirty bit change victim choice?”

12. Coding/Debugging Scenario

Host under memory pressure: `sar -B` shows huge `pgscan/s` vs low `pgsteal` → reclaim scanning hard for few evictable pages (all hot/dirty) → thrashing verge; add RAM, cut working set, or tune dirty writeback.

13. Best Practices

Cite Clock when asked to build a cheap LRU-ish cache with no per-hit locking (a ring + reference flags is a legit design); watch `pgscan/pgsteal` ratios in prod.

14. Practice Questions

1. Simulate clock on ref string `1 2 3 4 1 5` with 4 frames, bits shown per step.
2. Design a lock-free-ish cache eviction using per-entry “touched” flags + a sweeper thread — what did you just reinvent?

6.5 Belady’s Anomaly

1. Why Interviewers Ask This

A counterintuitive gem: **more memory can cause more page faults**. Tests rigor (can you demonstrate it?) and theory depth (why LRU is immune).

2. Core Concept

Under FIFO, increasing frame count can *increase* faults for some reference strings. Canonical string: `1 2 3 4 1 2 5 1 2 3 4 5` → **9 faults with 3 frames, 10 with 4**.

3. Internal Working

Why: FIFO’s eviction depends on *arrival order*, and the set of resident pages with k frames is **not necessarily a subset** of the set with $k+1$ frames — the caches “diverge”. **Stack algorithms** (LRU, OPT, LFU-by-true-frequency) maintain the inclusion property: $\text{resident}(k) \subseteq \text{resident}(k+1)$ at every step → more frames can never hurt → anomaly-impossible.

4. ASCII Diagram

```
Ref: 1 2 3 4 1 2 5 1 2 3 4 5
3 frames (FIFO): F F F F F F H H F F H = 9 faults
4 frames (FIFO): F F F F H H F F F F F = 10 faults (!)
Inclusion property (LRU): 3-frame content is always a subset of
4-frame content -> extra frame can only convert misses to hits.
```

5. Real Production Example

The general lesson bites for real: any FIFO-flavored cache (some hardware TLBs/queues, naive eviction in app caches) can *regress after a capacity upgrade*. “We doubled the cache and hit rate dropped” is a real incident class — always benchmark policy × size, don’t assume monotonicity.

6–7. Advantages / Trade-offs

(A property, not a tool.) Its value: proves eviction-policy choice matters *structurally*, not just by constants; motivates stack algorithms.

8. Common Mistakes

- Claiming the anomaly applies to LRU (it cannot — inclusion property; be ready to state why).
- Being unable to reproduce the canonical example under time pressure (practice the 12-step table until mechanical).
- Overstating frequency: it's rare and workload-specific — a curiosity with a deep lesson, not a daily hazard.

9. Performance Implications

Direct: capacity planning for FIFO-like caches must be empirical. Indirect: prefer policies with the inclusion property when you want “more RAM never hurts” guarantees (miss-ratio curves monotone → simple autoscaling).

10–11. Interview & Follow-ups

- “What is Belady’s anomaly? Demonstrate it.” “Why is LRU immune?” (stack/inclusion property) “What’s Belady’s *algorithm*?” (OPT — different thing, same Belady; evict farthest-future-use; the optimality benchmark)

12. Coding/Debugging Scenario

Write a 30-line simulator (policy × frames × ref string); verify 9 → 10 on the canonical string; use the same harness to pick eviction policies with production traces — the actually-useful takeaway.

13. Best Practices

Validate cache sizing with trace-driven simulation (miss-ratio curves); don’t extrapolate “bigger is better” for non-stack policies.

14. Practice Questions

1. Hand-run the canonical string at 3 and 4 frames; produce both fault counts.
2. Prove (informally) that LRU satisfies the inclusion property.
3. Distinguish: Belady’s anomaly vs Belady’s optimal algorithm.

6.6 Thrashing (bonus — the production failure mode)

The reason this module matters operationally: when the **working set** exceeds RAM, every fault evicts a page that’s needed again soon → the system spends its time paging, not computing. Signature: CPU% low, load high, disk 100%, majflt/s high, everything crawls. Fixes: add RAM, shrink working sets (better locality, smaller heaps), cap concurrency (fewer simultaneous working sets — admission control), kill the offender (OOM killer as last resort), never “fix” by adding more swap (extends the agony). Interview line: “*Thrashing is when the aggregate working set exceeds physical memory; the cure is reducing demand or adding supply — never more swap.*”

Module 6 Cheat Sheet (one page)

Policy	Evicts	Cost	Strength	Fatal flaw
OPT (Belady)	Farthest future use	Needs oracle	Optimal benchmark	Unimplementable
FIFO	Oldest resident	$O(1)$, no hit cost	Simplicity	Ignores use; Belady’s anomaly
LRU	Longest unused	$O(1)$ w/ map+DLL	Temporal locality	Scan pollution; infeasible in HW
LFU	Fewest accesses	Buckets $O(1)$	Skewed popularity	Stale frequency (needs decay)
Clock	ref-bit=0 on sweep	~0 hit cost	Practical LRU-approx	1-bit coarse → FIFO when all hot

Kernel reality: hardware accessed+dirty bits → clock-family reclaim (Linux active/inactive or MGLRU; Postgres clock-sweep; Redis sampled LRU / 8-bit decayed LFU; Caffeine W-TinyLFU). **Stack algorithms** (LRU, OPT): $\text{resident}(k) \subseteq \text{resident}(k+1)$ → no Belady's anomaly. FIFO: anomaly possible (canonical string: 9 faults @3 frames, 10 @4). **Dirty pages** cost a writeback on eviction → prefer clean victims; background flushers exist for this. **Thrashing:** working set > RAM; fix demand or supply, not swap.

Top Interview Questions

1. Design an LRU cache, $O(1)$ get/put (write it cold). Then thread-safe. Then sharded.
2. Simulate FIFO/LRU/OPT on a reference string, count faults.
3. Why can't the OS do true LRU? What does it do instead? (accessed bit → Clock family)
4. LRU vs LFU — workload-based decision + decay requirement.
5. Belady's anomaly: demonstrate; why is LRU immune?
6. How do Redis / InnoDB / Postgres actually evict? (sampling, midpoint LRU, clock-sweep)
7. What is thrashing and how do you detect/fix it?

Common Mistakes (module-wide)

- LRU-code bugs (map/list desync, no move-to-front on update).
- LFU without decay; FIFO simulation updating on hits.
- "Kernel keeps an exact LRU list per access."
- Claiming Belady's anomaly for LRU; conflating Belady's anomaly with Belady's algorithm.
- Ignoring dirty-page writeback cost and scan pollution.

Mock Interview (self-test, ~20 min)

1. (Code) LRU cache $O(1)$; interviewer adds: "now TTL per key, and eviction must not block gets" — evolve the design.
2. (Simulate) 2 3 2 1 5 2 4 5 3 2 5 2 with 3 frames: fault counts for FIFO, LRU, OPT.
3. (Design) Pick and justify the eviction policy for: (a) CDN edge with 65% one-hit-wonders, (b) DB buffer pool subject to nightly full scans, (c) session cache with strong recency.
4. (Prod) Host: CPU 15%, load 30, disk util 100%, majflt/s 8000. Diagnose and give the fix ladder.
5. (Trap) "We upgraded the cache from 8 GB to 16 GB and the hit rate fell." Give two mechanisms that make this possible (FIFO-family anomaly; sharding/hash changes) and how to verify.

MODULE 7 — File Systems

Senior SWE Interview Track — Operating Systems

7.1 File System Basics

1. Why Interviewers Ask This

Backend engineers live on top of file systems (DB storage engines, logs, object stores). Interviewers test whether you know what actually happens on `open/read/write` and where durability really comes from.

2. Core Concept

A file system maps names → files → disk blocks, layered as: **VFS** (uniform API over ext4/XFS/NFS/tmpfs) → concrete FS (allocation, layout, journaling) → block layer (I/O scheduling) → device. Core objects: **superblock** (FS metadata), **inode** (per-file metadata), **dentry** (name → inode), **data blocks**.

3. Internal Working

`open("/var/log/app.log")`: VFS walks the path (dentry cache first), permission-checks each component, loads the inode, creates a `struct file` (offset + flags), installs it in the process FD table, returns the index (the fd). `read()` goes fd → file → inode → block mapping (extents) → **page cache** (hit: memcpy; miss: block I/O then memcpy). `write()` typically dirties page-cache pages; actual disk writes happen later (writeback) unless you `fsync` — the root of all durability questions.

4. ASCII Diagram

```

open/read/write/fsync (syscalls)
  v
  [ VFS ] -- dentry cache / inode cache
    v
  [ext4] [xfs] [nfs] [tmpfs] [procfs]
    v
  [ page cache ] <- most reads/writes stop here (RAM!)
    v (writeback / readahead)
  [ block layer: scheduler, merging ]
    v
  [ NVMe/SSD/HDD, on-device write cache ]

```

5. Real Production Example

Kafka's speed secret: sequential appends + page cache + `sendfile` zero-copy — “disk” throughput at RAM speed. Postgres/MySQL fight the opposite battle: forcing durability (`fsync/O_DIRECT`) despite the cache.

6. Advantages

Uniform API over wildly different backends; page cache makes hot I/O RAM-fast; names/permissions/hierarchy for free.

7. Trade-offs

The cache lies about durability (`write()` ≠ on disk); many layers to reason about during incidents; POSIX semantics (atomicity of rename, not of write) are subtle.

8. Common Mistakes

- Believing a returned `write()` is durable (it's in RAM; power loss eats it).
- Not knowing `rename()` is atomic (the foundation of safe config/file swaps) while multi-block `write()` is not.
- Forgetting directory `fsync`: creating a file durably requires `fsync` of the *directory* too.

9. Performance Implications

Page-cache hit $\approx$ memcpy (GB/s); miss $\approx$ device latency (NVMe $\sim 100\ \mu\text{s}$, HDD $\sim 10\ \text{ms}$). Free RAM is not wasted — it's cache (`free -h` “available” vs “free” — classic misreading).

10–11. Interview & Follow-ups

- “What happens on `open()` end-to-end?” “When is my write actually on disk?” “Why does Kafka read at memory speed?”
- Follow-ups: “What does the dentry cache buy?” “Atomic file replace pattern?” (write temp $\rightarrow$ fsync $\rightarrow$ rename $\rightarrow$ fsync dir)

12. Coding/Debugging Scenario

Config corruption after power loss $\rightarrow$ app wrote in place without fsync/rename; fix with the atomic-replace pattern.

13. Best Practices

Durability = fsync (file **and** directory); atomic updates via rename; treat page cache as your friend for throughput and your enemy for durability reasoning.

14. Practice Questions

1. Write pseudocode for crash-safe “replace config file”.
2. Trace `cat /etc/hosts` through every layer named above.

7.2 Inode

1. Why Interviewers Ask This

“What’s an inode?” is a Linux screening staple; the deeper hooks — hard links, deleted-but-open files, inode exhaustion — are real incident material.

2. Core Concept

The inode is a file’s identity and metadata record: type, permissions, owner, size, timestamps, **link count**, and pointers to data blocks. It contains **no filename** — names live in directory entries (name $\rightarrow$ inode number). Multiple names can reference one inode = **hard links**.

3. Internal Working

- Classic layout: 12 direct block pointers + single/double/triple indirect (ext2/3); ext4 uses **extents** instead (start + length runs).
- Directory = a file whose contents are (name, inode#) pairs.
- Deletion: `unlink()` decrements link count; data blocks are freed only when link count = 0 **and** no process holds it open — hence “deleted” log files still consuming disk (visible via `ls -l +L`; `du` vs `df` disagree).
- Inodes are a fixed pool created at mkfs (ext4) — you can run out of inodes with disk space free (millions of tiny files).

4. ASCII Diagram

```

dir entry: ("app.log", #5231)      dir entry: ("app_hardlink", #5231)
      \                             /
       v                             v
      inode #5231 [mode,uid,size,mtime,links=2]
        | extents / block ptrs
        v
      [data blocks...]
unlink both + still open by PID 42 -> blocks freed only on close
Symlink: its OWN inode whose data = "/path/to/target" (can dangle)

```

5. Real Production Example

- “Disk full but `du` shows 40% used” → deleted-but-open file held by a running process (restart/`truncate` via `/proc/<pid>/fd/N`).
- “No space left on device” with free GBs → inode exhaustion (`df -i`) from millions of small files (session files, mail queues).
- Log rotation (`logrotate`) exists precisely because of unlink-vs-open semantics (`copytruncate` vs `move+signal`).

6. Advantages

Name/identity separation enables hard links, atomic rename, open-file stability across renames; fixed-size record enables fast stat.

7. Trade-offs

Fixed inode pool (ext4); hard links complicate reasoning (“which name is real?” — none is); metadata-heavy workloads (small files) bottleneck on inode/dentry ops.

8. Common Mistakes

- “The inode stores the filename.”
- Hard link vs symlink confusion: hard link = same inode, same FS only, survives target deletion; symlink = separate inode holding a path, can cross FS and dangle.
- Not knowing `df` vs `du` divergence causes.

9. Performance Implications

`stat`-heavy workloads (web servers checking mtimes, build tools) live on the inode/dentry caches; a `find /` can evict them. Directories with millions of entries degrade lookup (ext4 htree helps; still, shard directories).

10–11. Interview & Follow-ups

- “Hard vs soft link — table.” “What happens on `rm` of an open file?” “Why can a disk be ‘full’ with space free?”

12. Coding/Debugging Scenario

Disk filling with nothing visible: `ls -l | sort -k7 -n` → 80 GB deleted log held by java → rotate properly (reopen signal) or truncate through `/proc/<pid>/fd`.

13. Best Practices

Rotate logs with reopen (SIGHUP/copytruncate); shard huge directories; monitor `df -i`; use `ls -li/stat` fluently.

14. Practice Questions

1. Draw the state after: `touch a; ln a b; ln -s a c; rm a` — what does each of b, c resolve to?
2. mkfs choices for a filesystem storing 500M 2 KB files — what do you tune? (inode ratio, or use a different design entirely)

7.3 Journaling

1. Why Interviewers Ask This

Crash consistency is a systems-design fundamental — the same write-ahead-log idea powers databases, and interviewers love the “what happens on power loss mid-write?” scenario.

2. Core Concept

A multi-block FS update (e.g., append: data block + inode size + bitmap) is not atomic — a crash mid-way corrupts.

Journaling = write-ahead logging: record the intent in a sequential journal, commit it, *then* apply in place. Recovery replays committed transactions and discards partial ones — mount-time recovery in seconds vs hours of full-disk `fsck`.

3. Internal Working

1. Write transaction (affected metadata ± data) to the journal (sequential).
2. Write **commit record** — the transaction is now atomic-durable.
3. **Checkpoint**: apply blocks to home locations.
4. Reclaim journal space. Crash before commit → transaction ignored; after commit → replayed idempotently. Modes (ext4): **journal** (data+metadata journaled — safest, double-write cost), **ordered** (default: only metadata journaled, but data written *before* the metadata commits — no garbage-in-file), **writeback** (metadata only, data anytime — fastest, may expose stale data post-crash).

4. ASCII Diagram

```
append to file = 3 in-place writes: [data][inode][bitmap] (not atomic!)
Journaling:
  JOURNAL (sequential): | T42: data,inode,bitmap | COMMIT T42 |
  crash here? --^ no commit -> ignore (file unchanged, consistent)
  after commit -> replay T42 -> in-place writes -> checkpoint done
Same idea as: DB WAL / redo log / Raft log.
```

5. Real Production Example

ext4/XFS journals (XFS: metadata-only by design); databases replicate the pattern (Postgres WAL, InnoDB redo log) — and *stack* on it: DB WAL on a journaled FS (why DBs often use O_DIRECT to avoid double-buffering, and fdatasync to avoid double journaling costs). Copy-on-write filesystems (ZFS, Btrfs) achieve crash consistency differently — never overwrite in place.

6. Advantages

Seconds-fast recovery; metadata always consistent; sequential journal writes are cheap; atomicity building block.

7. Trade-offs

Write amplification (up to 2× in full-journal mode); journal is a serialization point; ordered mode still doesn't make *your* data transactional — only FS structures (your app still needs fsync + its own WAL for app-level atomicity).

8. Common Mistakes

- Believing journaling makes application writes atomic/durable (it protects FS *metadata* consistency; your data needs fsync and app-level protocols).
- Not knowing the three ext4 modes and the default (ordered).
- Confusing journaling (redo intent log) with CoW snapshots (ZFS/Btrfs).

9. Performance Implications

fsync latency is governed by journal commit (flush + FUA/barrier to the device); many small fsyncs serialize on it — group commit (batching) is the fix, in ext4 and in every database.

10–11. Interview & Follow-ups

- “Power fails mid-append: what states are possible with/without journaling, per ext4 mode?”
- “Why do databases use O_DIRECT + their own WAL instead of trusting the FS journal?”

12. Coding/Debugging Scenario

fsync-heavy service slow on ext4: check `data=journal` accidentally enabled (double writes), device write-cache/FUA behavior, and batch commits (group multiple records per fsync).

13. Best Practices

App-level durability = your own WAL/fsync discipline; keep FS defaults (ordered) unless measured; put DB WAL on separate low-latency device if fsync-bound.

14. Practice Questions

1. Enumerate post-crash states of “append 4 KB” under: no journal, writeback, ordered, full journal.
2. Design group commit for a WAL that must sustain 50k inserts/s with fsync durability.

7.4 ext4

1. Why Interviewers Ask This

The default Linux FS — asked as “what do you actually run and why”, plus a vehicle for extents/delayed allocation concepts.

2. Core Concept

ext4 = journaling FS with **extents** (contiguous run descriptors instead of per-block pointers), **delayed allocation** (buffer writes, allocate blocks at flush → better contiguity), multiblock allocator, htree directory indexing, and jbd2 journal. Solid general-purpose default; XFS often preferred for huge files/parallelism.

3. Internal Working

- Layout: block groups (superblock/GDT copies, block+inode bitmaps, inode table, data). Flex groups aggregate metadata.
- Extent tree in inode: up to 4 extents inline; deeper trees for fragmented files. An extent maps up to 128 MB contiguous.
- Delayed allocation: dirty pages accumulate; allocation at writeback picks contiguous ranges (this also creates the famous “zero-length file after crash” pattern for apps that skip fsync — mitigated by heuristics for rename patterns).
- Limits: 1 EB volume / 16 TB file (4 KB blocks); timestamps ns-precision to 2446.

4. ASCII Diagram

```
ext2/3 block ptrs: [b1][b2]...[b12] -> [indirect] -> [double] -> [triple]
                  (1000 blocks = 1000+ pointers)
ext4 extents:      [start=8000, len=1000]    (one record!)
Delayed alloc: write() -> dirty pages ... writeback -> allocate 1 big extent
Result: less fragmentation, less metadata, faster fsck.
```

5. Real Production Example

Default on most distro servers and Android (userdata); typical choices: ext4 for general workloads/boot volumes, XFS for large-file/parallel-write servers (RHEL default), ZFS/Btrfs when snapshots/checksums are required.

6. Advantages

Mature/stable, fast recovery (jbd2), extents+delalloc = good contiguity, online resize, wide tooling (e2fsck, tune2fs).

7. Trade-offs

No data checksums (metadata checksums only) — silent bit rot passes; no native snapshots/compression (vs ZFS/Btrfs); fixed inode count at mkfs; delayed allocation vs crash expectations of naive apps.

8. Common Mistakes

- Selling ext4 features it lacks (checksums, snapshots, dedup).
- Not knowing extents vs indirect blocks is *the* ext3 → ext4 headline.
- Ignoring reserved blocks (default 5% for root — “disk full” at 95% for non-root).

9. Performance Implications

Extents shrink metadata I/O for big files dramatically; delalloc turns many small writes into large sequential allocations; jbd2 commit (default 5 s interval or fsync-driven) is the durability heartbeat.

10–11. Interview & Follow-ups

- “ext3 vs ext4?” (extents, delalloc, faster fsck, larger limits) “When XFS/ZFS instead?” “What’s in a block group?”

12. Coding/Debugging Scenario

“Disk full” at 95%: `tune2fs -m` reserved blocks; or inode exhaustion `df -i`; or deleted-open files — the ext4 triage trio.

13. Best Practices

Defaults are good; `noatime/relatime` for read-heavy; align mkfs to workload (inode ratio for small-file storms); LVM under ext4 if you need snapshots.

14. Practice Questions

1. Explain why delayed allocation reduces fragmentation but complicates crash semantics for careless apps.
2. Compare ext4/XFS/ZFS across: checksums, snapshots, max scale, typical use.

7.5 NTFS (High Level)

1. Why Interviewers Ask This

Cross-platform breadth check (Microsoft interviews especially); the interesting content is what design ideas differ from Unix FSes.

2. Core Concept

Windows’ native FS. Central idea: **everything is a file record in the MFT** (Master File Table), and *everything is an attribute* — filename, security descriptor, even data are attributes of an MFT record. Journaling (\$LogFile, metadata-only), B-tree directories, ACL-based security richer than POSIX bits, alternate data streams, compression/encryption built in.

3. Internal Working

- MFT: 1 KB records per file; **small files live entirely inside the record** (“resident data” — very fast small-file access, like inline files).
- Larger data → non-resident attribute with extent-like “data runs”.
- Directories: B-trees of filename attributes (vs classic Unix linear/htree).
- USN change journal (change tracking for indexers/backup), Volume Shadow Copy (snapshot service above FS), hard links + reparse points (symlink/mount equivalents).

4. ASCII Diagram

```
MFT: | rec0 $MFT | rec1 $LogFile | ... | rec N: file.txt |
rec N attributes:
  [$STANDARD_INFO][$FILE_NAME][$SECURITY][$DATA: "hello" (resident)]
big file: [$DATA -> runs: (LCN 8000,len 500),(LCN 12000,len 300)]
Analogy: MFT record ~ inode+dirent+small-data fused; runs ~ extents.
```

5. Real Production Example

Windows servers/desktops, SQL Server hosts; interop via exFAT (removable) since NTFS write support elsewhere is limited; SMB shares expose NTFS ACL semantics that Linux services must map (Samba).

6. Advantages

Rich ACLs, resident small files, change journal, mature online defrag/self-healing, compression/encryption/quotas native.

7. Trade-offs

Proprietary/Windows-centric; MFT fragmentation/zone management; case-insensitive (by default) name semantics — a real bug source when code assumes Unix behavior.

8. Common Mistakes

- Only knowing “it’s the Windows one” — name MFT + attributes + journaling to show breadth.
- Assuming POSIX semantics on NTFS (case-insensitivity, file-locking differences — files open elsewhere can’t be deleted, unlike Unix unlink).

9. Performance Implications

Resident small files beat separate-inode+block designs; ACL checks are richer/heavier; the “can’t delete an open file” model changes deploy patterns (why Windows updates need reboots vs Unix replace-while-running).

10–11. Interview & Follow-ups

- “NTFS vs ext4 — three design differences.” “Why can Linux replace a running binary but Windows can’t?” (unlink semantics vs mandatory sharing locks)

12–14. Scenario / Practices / Questions

- Scenario: a cross-platform build breaks only on Windows: two files differing by case. Fix naming policy.
- Practice: map inode/dentry/journal to their NTFS counterparts (MFT record / \$FILE_NAME index / \$LogFile).

7.6 File Descriptors

1. Why Interviewers Ask This

FDs unify files/sockets/pipes/epoll — “everything is a file descriptor” is the mental model behind all Unix I/O, and FD leaks (“too many open files”) are a top-3 production incident.

2. Core Concept

An FD is a small per-process integer indexing the **process FD table**, whose entries point to entries in the system-wide **open file table** (offset, flags, mode), which point to inodes (or sockets/pipes/etc.). 0/1/2 = stdin/stdout/stderr.

3. Internal Working

- Three levels: FD table (per-process) → open-file description (offset lives HERE) → inode/socket.
- `dup()/dup2()` and `fork()` share one open-file description → **shared offset** (two FDs, one cursor — interview trap!). Two separate `open()`s of the same file → independent offsets.
- `O_CLOEXEC` closes on exec (prevents FD leaks into children).
- Limits: `ulimit -n` per-process (soft/hard), system-wide `fs.file-max`. Sockets, pipes, epoll instances, eventfds, timerfds all consume FDs.

4. ASCII Diagram

```

Process A FD table      Open file table (system)      inodes
fd0 stdin -----+
fd3 -----> | 0FD#7: offset=4096, 0_RDWR |----> inode app.log
fd4 (dup of 3) ---> | (SAME offset!) |
Process B (forked):
fd3 -----> same 0FD#7 -> parent/child share the cursor
Fresh open() by B: 0FD#9: offset=0 -----> same inode

```

5. Real Production Example

“Too many open files” (EMFILE) taking down proxies/DB pools: leaked sockets from missing close on error paths; default 1024 soft limit vs 10k connections. Nginx/envoy deployments always raise `nofile`. `lsof -p / ls /proc/<pid>/fd | wc -l` are the triage tools.

6. Advantages

One API (read/write/close/poll) across files, sockets, pipes, devices, timers, events — composability that epoll exploits.

7. Trade-offs

Small-integer namespace leaks easily; shared-offset semantics surprise; limits are per-process and commonly too low by default.

8. Common Mistakes

- Not knowing offset lives in the open-file description (→ can’t explain dup/fork sharing).
- Forgetting sockets/pipes/epoll count against `nofile`.
- Leaking FDs on early-return error paths (use RAII/defer/try-with-resources).

9. Performance Implications

FD table lookups are $O(1)$; the real costs are leak-driven exhaustion and, historically, `select()`’s 1024-FD bitmap ceiling (Module 8). Millions of FDs are fine for epoll-based servers with raised limits.

10–11. Interview & Follow-ups

- “Two processes read the same file — do they share the offset? Depends on *how* they got the FDs — explain.”
- “Walk through fixing ‘too many open files’ in production.” “What is `O_CLOEXEC` for?”

12. Coding/Debugging Scenario

Sockets in `CLOSE_WAIT` piling up + FD count climbing → app not closing after peer disconnect; fix close handling; interim: raise limit + alert on `fd_used/fd_limit`.

13. Best Practices

RAII everywhere; set `nofile` deliberately per service; monitor FD usage; `O_CLOEXEC` by default (or use `SOCK_CLOEXEC`).

14. Practice Questions

1. Parent opens file, forks; parent reads 100 bytes; where does the child’s next read start? Now both processes `open()` independently — same question.
2. Compute a sane `nofile` for a proxy: 50k conns in, 50k out, plus logs/epoll — show the math and headroom.

7.7 mmap()

1. Why Interviewers Ask This

`mmap` connects Modules 5 and 7 (page cache + demand paging as a file API) and powers real systems (Kafka, RocksDB/LMDB, LLM weight loading) — a perfect senior probe.

2. Core Concept

`mmap(fd)` maps a file’s pages directly into your address space: bytes are accessed with loads/stores instead of read/write syscalls. Pages fault in on demand from the **page cache** — file data with zero copies into user buffers. Modes: `MAP_SHARED` (writes visible to others / write back to file) vs `MAP_PRIVATE` (COW, your own copy); also `MAP_ANONYMOUS` (no file — how malloc gets big blocks).

3. Internal Working

- `mmap` creates a VMA; no I/O yet. First touch → page fault → page cache lookup → (maybe disk read) → PTE maps the *page-cache page itself* into your process. Multiple processes mapping one file share the same physical pages.
- Writes to `MAP_SHARED` dirty page-cache pages; kernel writeback (or `msync`) persists them; the dirty bit tracks it.
- vs `read()`: read copies page cache → your buffer (1 copy + syscall per call); `mmap` = 0 copies, no per-access syscalls, but a page fault per new page and TLB costs.
- `madvise` tunes it: `SEQUENTIAL/WILLNEED` (readahead), `DONTNEED` (drop), `RANDOM`.

4. ASCII Diagram

```
read(): disk -> [page cache] --copy--> user buffer (syscall each time)
mmap(): disk -> [page cache] <--- PTEs map SAME pages into process
        ^ shared by all mappers; writes dirty the page
file bytes 0..N <-> vaddr M..M+N (load/store = file I/O)
Persistence: writeback or msync; MAP_PRIVATE: first write -> COW copy.
```

5. Real Production Example

- **LMDB / SQLite (optional) / RocksDB (optional) / early MongoDB**: B-trees on `mmap` — the OS is the buffer pool.
- **Kafka**: `mmap` for index files.
- LLM inference (llama.cpp): `mmap` weights → instant “load”, pages fault in as used, shared across processes.
- Every process’s code/libraries are `mmaps` of binaries (demand-paged executables).

6. Advantages

Zero-copy reads; page-cache sharing across processes; random access without seek/read choreography; lazy loading; file-backed data structures for free.

7. Trade-offs

Page-fault latency is *invisible* in code (a memory access can block on disk — no place to handle EIO nicely; `SIGBUS` on truncated files!); writeback timing is fuzzy without `msync`; TLB shutdown cost on `munmap`; not great for streaming-once workloads (read+readahead wins); hard-to-account memory (RSS shows shared pages).

8. Common Mistakes

- “`mmap` is always faster than `read`” — per-page faults + TLB churn can lose to batched reads for sequential one-pass I/O.
- Forgetting `SIGBUS` (file truncated by another process while mapped) — must handle for robustness.
- Assuming `MAP_SHARED` writes are durable without `msync`/`fsync`.

9. Performance Implications

Wins: hot random reads on cacheable working sets, many-process sharing, huge sparse files. Loses: cold sequential streaming, tiny files (setup overhead), write-heavy with strict durability (`msync` granularity). DB folklore: “`mmap` considered harmful for DBs” (Andy Pavlo’s paper) — error handling, eviction control, and TLB costs — know both sides.

10–11. Interview & Follow-ups

- “`mmap` vs `read/write` — mechanics and when each wins.” “What happens on the first access to a mapped page?” “`MAP_SHARED` vs `MAP_PRIVATE`?”
- Follow-up: “Why did MongoDB abandon its `mmap` storage engine?” (control over eviction/writeback/durability → `WiredTiger`)

12. Coding/Debugging Scenario

Service crashes with SIGBUS reading a mapped file → another process truncated it; guard with file locks or fstat+bounds, handle SIGBUS, or switch to pread.

13. Best Practices

mmap for shared read-mostly data and file-backed structures; explicit read/write + O_DIRECT for DB-grade control; always plan durability (msync) and SIGBUS handling.

14. Practice Questions

1. Implement a persistent append-only index with mmap: growth (ftruncate+mremap), durability, torn-write handling.
2. Two processes map the same file MAP_SHARED; P1 stores to a page. When does P2 see it? When does the disk? (immediately — same physical page; disk at writeback/msync)

7.8 Buffered vs Direct I/O

1. Why Interviewers Ask This

The capstone of FS interviews: “why do databases use O_DIRECT?” tests page cache, double buffering, durability, and design judgment in one question.

2. Core Concept

- **Buffered I/O** (default): read/write go through the **page cache**; writes return after dirtying RAM; kernel does readahead + writeback batching.
- **Direct I/O** (**O_DIRECT**): DMA straight between your user buffer and the device, bypassing the page cache. Requirements: buffer/offset/length aligned (typically 512 B/4 KB). You own caching, scheduling, and readahead now.
- Orthogonal: **O_SYNC** (buffered but wait for durability per write), **fsync/fdatasync** (flush on demand). O_DIRECT ≠ durable by itself (device cache! still may need flush).

3. Internal Working

Buffered write: copy user → page cache, mark dirty, return; writeback threads flush by age/pressure (**dirty_ratio**, **dirty_expire_centisecs**). Buffered read: page cache hit or miss+readahead. Direct write: pin user pages, DMA to device, return on device ack (not necessarily on media — flush/FUA for that). No copy, no cache pollution, no readahead.

4. ASCII Diagram

```
Buffered:  user buf --copy--> [page cache] --later--> disk
           read: [page cache] --copy--> user buf (+readahead)
Direct:    user buf <===== DMA =====> disk    (no cache, aligned only)
DB double-buffering problem (buffered):
  [InnoDB buffer pool 64GB] + [page cache caching the SAME pages] = waste
=> O_DIRECT for data files; keep WAL buffered-or-O_DSYNC per engine.
```

5. Real Production Example

- MySQL InnoDB: **innodb_flush_method=O_DIRECT** standard practice (avoid double buffering with the buffer pool).
- Postgres: deliberately relies on the page cache (small shared_buffers + OS cache) — the counter-example; know both philosophies.
- Kafka: buffered + sendfile (page cache *is* the design). Video streaming/backup tools: O_DIRECT to avoid trashing the cache for one-pass data.

6. Advantages

Buffered: free caching/readahead/batching, forgiving API. Direct: no double buffering/copy, predictable latency (no writeback storms), cache preserved for others, app-controlled eviction.

7. Trade-offs

Buffered: double-caching with app caches, writeback latency spikes (dirty flush storms), durability requires fsync discipline. Direct: alignment pain, you must rebuild caching/readahead (or your throughput craters), sync writes expose full device latency.

8. Common Mistakes

- “O_DIRECT is faster” unqualified — for most apps buffered wins (readahead + write batching); O_DIRECT wins when *you* cache better than the kernel.
- Believing O_DIRECT implies durability (device write cache — still need flush/FUA or O_DSYNC).
- Forgetting alignment requirements → EINVAL surprises.

9. Performance Implications

Buffered burst writes absorb at RAM speed then stall at `dirty_ratio` (foreground throttling — the mysterious “writes suddenly slow”); O_DIRECT gives flat device-speed latency. Cache-pollution math: one 500 GB backup pass can evict every hot page on the box (`posix_fadvise(DONTNEED)` / O_DIRECT for the backup is the fix).

10–11. Interview & Follow-ups

- “Why does InnoDB use O_DIRECT but Postgres doesn’t?” “What exactly does fsync guarantee vs write?” “How would you stop a backup job from destroying cache hit rates?”

12. Coding/Debugging Scenario

Nightly backup tanks DB p99: page-cache eviction by the backup’s buffered reads → O_DIRECT/fadvise the backup, ionice it, and cap dirty bytes.

13. Best Practices

Default buffered + fsync discipline; O_DIRECT only with your own cache layer; tune `vm.dirty_*` for write-heavy boxes; `fdatasync` over `fsync` when metadata timestamps don’t matter.

14. Practice Questions

1. Design the I/O path for a new LSM storage engine: which files direct, which buffered, where fsync — justify.
2. Explain what each guarantees: `write()` return, `write()+fsync`, O_SYNC write, O_DIRECT write, O_DIRECT+O_DSYNC.

Module 7 Cheat Sheet (one page)

Concept	One-liner	Interview hook
VFS stack	syscalls → VFS → FS → page cache → block → device	where is my write? (RAM until writeback)
Inode	metadata + block map; no name	hard links; deleted-but-open; <code>df -i</code>
Journaling	WAL for FS metadata	ext4 modes: journal/ordered/writeback; ≠ app durability
ext4	extents + delayed alloc + jbd2	vs ext3; no checksums/snapshots
NTFS	MFT records; everything is an attribute	resident small files; can’t delete open files
FDs	<code>fd</code> → open-file-desc(offset!) → inode	dup/fork share offset; EMFILE incidents
mmap	map page cache into address space	zero-copy; SIGBUS; MAP_SHARED vs PRIVATE
Buffered vs Direct	page cache vs DMA bypass	DB double-buffering; fsync semantics

Durability ladder: `write()` = RAM → `+fsync` = on device (+ `dir fsync` for new files) → `O_DIRECT+flush` = bypass cache. **Atomicity:** rename yes, write no. **Triage classics:** `du≠df` → deleted-open files (`ls -lsof +L1`); “No space” with space → `df -i`; EMFILE → `ls /proc/pid/fd | wc -l`.

Top Interview Questions

1. What happens on `open()/read()/write()` end-to-end? When is data durable?
2. Inode contents; hard vs symlink; rm of an open file.
3. Power-loss mid-write: outcomes with/without journaling; ext4 modes.
4. Do two processes share a file offset? (`dup/fork` vs independent opens)
5. `mmap` vs `read` — mechanics, when each wins, SIGBUS.
6. Why `O_DIRECT` for InnoDB but page cache for Postgres/Kafka?
7. Debug: “too many open files” / disk-full paradoxes.

Common Mistakes (module-wide)

- `write()` treated as durable; journaling treated as app-level atomicity.
- “Inode stores the filename”; symlink/hardlink confusion.
- Missing shared-offset semantics; forgetting sockets count as FDs.
- “`mmap/O_DIRECT` always faster”; ignoring alignment and SIGBUS.
- Misreading `free -h` (cache ≠ used-up RAM).

Mock Interview (self-test, ~20 min)

1. (Design) Crash-safe key-value store on a raw directory: file layout, WAL, `fsync` points, atomic compaction via `rename`. Defend each choice.
2. (Depth) `echo hi >> file` — enumerate every structure touched (dentry, inode, page cache, journal, bitmaps) and every crash window.
3. (Prod) `df` says 100%, `du` says 60%, and deletes don't help. Full triage script, in order.
4. (Trade-off) Your cache service memory-maps a 200 GB read-mostly dataset. Argue for and against replacing it with `pread` + an app cache.
5. (Trap) “We set `O_DIRECT` so our writes are safe from power loss.” Correct every wrong assumption in that sentence.

MODULE 8 — I/O

Senior SWE Interview Track — Operating Systems

8.1 Blocking vs Non-blocking I/O

1. Why Interviewers Ask This

The vocabulary of every server-architecture discussion. Interviewers check you can define it precisely (it's about the *call*, not the program) before letting you say “epoll”.

2. Core Concept

- **Blocking**: the syscall doesn't return until it can do work — `read()` on an empty socket sleeps the thread.
- **Non-blocking** (`O_NONBLOCK`): the syscall returns *immediately*: with data if available, else `EAGAIN/EWOULDBLOCK`. The thread never sleeps in the call — but now *you* must discover readiness (polling or readiness APIs). Blocking/non-blocking describes the syscall behavior; sync/async (8.2) describes who completes the operation.

3. Internal Working

Blocking read on empty socket: kernel puts the task on the socket's wait queue (state S), context-switches away; packet arrival (softirq) fills the receive buffer and wakes the queue. Non-blocking read: kernel checks the buffer, copies what's there or returns `EAGAIN` — never enqueues the task. Regular *files* are a trap: `O_NONBLOCK` effectively doesn't apply — file reads always “block” for the disk (why event loops use thread pools or `io_uring` for file I/O).

4. ASCII Diagram

BLOCKING T: read(fd) -----+ [thread SLEEPS] pkt data ready -----+ T: <- returns data 1 thread per waiting fd	NON-BLOCKING T: read(fd) -> EAGAIN (instantly) T: ...do other work... T: read(fd) -> EAGAIN pkt arrives T: read(fd) -> data 1 thread, many fds (needs readiness API)
--	---

5. Real Production Example

Thread-per-connection blocking servers (classic Tomcat/Apache prefork) vs non-blocking event loops (Nginx, Node.js, Netty, Redis). The blocking model dies at ~10k connections from thread memory + context switches — the C10K problem that birthed the non-blocking ecosystem.

6. Advantages

Blocking: simple linear code, errors in-line, great with few connections or virtual threads. Non-blocking: thousands–millions of connections per thread; no per-connection stack.

7. Trade-offs

Blocking: thread cost per idle connection (~MBs stack + switch overhead). Non-blocking: inversion of control (state machines/callbacks), `EAGAIN` handling everywhere, *must not run blocking calls on the loop*.

8. Common Mistakes

- Conflating non-blocking with asynchronous (non-blocking read still copies data synchronously when ready).
- Forgetting partial reads/writes: non-blocking `write()` may accept 10 KB of your 1 MB — you must track progress (top interview coding trap).
- Blocking file I/O or DNS lookups inside an event loop (freezes every connection).

9. Performance Implications

Idle blocked thread costs memory, not CPU; 100k idle threads is memory-infeasible → readiness-based single-thread loops handle the same with ~MBs. Busy-polling non-blocking fds without a readiness API burns 100% CPU (hence 8.6–8.9).

10–11. Interview & Follow-ups

- “What happens inside the kernel when `read()` blocks?” “What does `read()` return on a non-blocking empty socket? On a closed one?” (EAGAIN vs 0=EOF — know the difference)
- “Why do event loops still use thread pools for files?”

12. Coding/Debugging Scenario

Node.js service freezes 2 s at a time: someone added synchronous file reads (`readFileSync`) on the request path → move to `async/worker` threads; watchdog the loop lag.

13. Best Practices

Never block the loop; handle EAGAIN + partial I/O rigorously; use blocking code only with cheap threads (virtual threads/goroutines).

14. Practice Questions

1. Write a correct non-blocking “write all N bytes” loop (EAGAIN + partial writes + EPOLLOUT re-arm).
2. Explain each return of non-blocking `read()`: `n>0`, `0`, `-1/EAGAIN`, `-1/ECONNRESET`.

8.2 Synchronous vs Asynchronous I/O

1. Why Interviewers Ask This

Everyone says “async”; few define it. Precise taxonomy (sync-blocking / sync-non-blocking / async) is a fast senior signal, and `io_uring` makes it current.

2. Core Concept

- **Synchronous:** *your thread* performs the I/O when it happens (even if it waited via `epoll` first — `epoll+read` is synchronous non-blocking).
- **Asynchronous:** you *submit* an operation; the kernel completes it in the background and notifies you (completion event); your thread never executes the copy/wait. Readiness model (`epoll`: “you may now read without blocking”) vs completion model (`io_uring`/IOCP: “your read finished, buffer is full”).

3. Internal Working

- POSIX AIO: mostly a library thread pool — historically weak.
- **Linux `io_uring`:** two shared-memory ring buffers (submission queue SQ, completion queue CQ). App writes SQEs, kernel consumes, executes (inline, async worker, or polled), posts CQEs. Zero syscalls possible in steady state (SQPOLL); batches naturally; covers files *and* sockets (fixing `epoll`’s file blind spot).
- Windows IOCP: completion ports — the original mainstream completion API (.NET, SQL Server).

4. ASCII Diagram

```

SYNC + epoll (readiness):
wait: epoll_wait -> "fd 7 readable"
you:  read(fd7, buf) <- your CPU
      does the copy

ASYNC io_uring (completion):
submit: SQE{read fd7 into buf}
kernel: does the read itself
you:   reap CQE{res=4096} -> buf full

App --SQ ring--> Kernel --CQ ring--> App    (shared memory, few syscalls)

```

5. Real Production Example

io_uring adoption: high-performance proxies/storage engines (e.g., TigerBeetle, some RocksDB/ScyllaDB-style engines use polling designs; ScyllaDB via Seastar uses its own async stack + AIO/io_uring). IOCP powers Windows server stacks. Netty/libuv are readiness-based with worker pools for files — know which is which.

6. Advantages

Async/completion: batching (many ops per syscall), file I/O without pools, fewer wakeups, kernel-side polling options; the copy overlaps your compute.

7. Trade-offs

Buffer lifetime management (kernel owns your buffer until completion — bugs waiting), harder cancellation, newer/less portable (io_uring: Linux 5.x+, security scrutiny caused some hosts to restrict it), more complex mental model.

8. Common Mistakes

- Calling epoll “asynchronous I/O” (it’s synchronous non-blocking with readiness notification) — precision here scores points.
- Ignoring buffer ownership rules in completion APIs.
- “async/await = OS async” (language async is usually cooperative scheduling atop readiness APIs).

9. Performance Implications

Syscall-bound workloads (many tiny ops): io_uring batching cuts syscalls per op toward 0 → big wins post-Spectre (syscalls got pricier). For large sequential I/O, models converge — the device is the limit.

10–11. Interview & Follow-ups

- “Four-quadrant table: blocking×sync/async with an example each.”
- “Readiness vs completion models?” “What problem does io_uring solve that epoll can’t?” (regular-file async, syscall batching)

12. Coding/Debugging Scenario

Migrating a log-ingestion service from read()-per-line to io_uring with registered buffers: measure syscalls/op drop (strace -c) and CPU reclaim.

13. Best Practices

Default to your platform’s mature readiness stack (epoll/kqueue via libuv/Netty/tokio); reach for io_uring when profiling shows syscall overhead or file-I/O-in-loop pain.

14. Practice Questions

1. Classify: blocking read; O_NONBLOCK read; epoll_wait+read; aio_read; io_uring read; sendfile.
2. Design the I/O model for a video-upload gateway (large streaming writes + many idle keepalives).

8.3 Polling

1. Why Interviewers Ask This

Polling vs interrupts is a fundamental cost trade-off that reappears everywhere (device drivers, spinlocks, message queues, Kubernetes controllers) — interviewers use it to test latency/throughput reasoning.

2. Core Concept

Polling = repeatedly checking state (“is data ready?”) instead of being notified. Busy-wait polling burns CPU for minimal latency; periodic polling saves CPU but adds up to one period of latency.

3. Internal Working

- CPU polling a device: read status register in a loop — sub- μ s reaction, one core consumed.
- App-level: loop over non-blocking reads (bad), or sleep-then-check (latency).
- Hybrid reality: **NAPI** in the Linux network stack — interrupt for the first packet, then *switch to polling* while traffic is high (amortizes interrupt cost), back to interrupts when idle. DPDK/SPDK: pure userspace polling, kernel bypass, 100% core burn for μ s-scale networking/storage. io_uring SQPOLL/IOPOLL: kernel-thread polling rings.

4. ASCII Diagram

```

Busy poll:  while(!ready){}           latency ~ns-us, CPU 100%
Periodic:   while(1){check; sleep(T)} latency ~T/2 avg, CPU ~0
Interrupt:  sleep until IRQ           latency ~us (handler+wakeup), CPU 0
NAPI hybrid: IRQ once -> poll while busy -> IRQ when idle
Rule: poll when events are FREQUENT (arrival interval < handling cost),
      interrupt when RARE.

```

5. Real Production Example

HFT/trading NICs and DPDK apps pin cores to poll (latency is money); Redis/Nginx use epoll (event-driven) not polling; Kafka consumers `poll()` in a loop (long-poll with timeout — polling shape, blocking wait inside); Kubernetes controllers reconcile on watch events + periodic resync (hybrid again).

6. Advantages

Lowest possible latency; no interrupt/context-switch overhead per event; predictable (no IRQ jitter); simple.

7. Trade-offs

Burns cores while idle; doesn't scale to many idle sources; energy cost; periodic polling trades latency for CPU linearly.

8. Common Mistakes

- Painting polling as always-bad — at high event rates polling is *cheaper* than interrupts (interrupt storms!).
- Writing accidental busy-loops (checking a queue with no backoff/blocking) — a real CPU-100% incident class.

9. Performance Implications

Break-even: if events arrive every $E \mu$ s and interrupt overhead is $I \mu$ s, polling wins when $E \leq I \times k$. 10 GbE at line rate $\approx$ 14.8 Mpps — interrupts can't keep up (livelock) → NAPI/DPDK exist.

10–11. Interview & Follow-ups

- “When is polling better than interrupts?” “What is NAPI and why hybrid?” “Why does DPDK bypass the kernel?”

12. Coding/Debugging Scenario

A worker checking an internal queue in a `while(true){ if(q.empty()) continue; }` loop pegs a core at 100% with zero traffic → replace with condition-variable wait or blocking dequeue.

13. Best Practices

Block or subscribe when idle; poll when saturated; hybrid adaptively (the NAPI pattern is the template); always bound busy-wait with backoff.

14. Practice Questions

1. Compute average latency and CPU cost of 1 ms periodic polling vs interrupts for 10 events/sec vs 1M events/sec.
2. Design the consumption model for an internal job queue at 5 jobs/min vs 50k jobs/sec.

8.4 Interrupts

1. Why Interviewers Ask This

Interrupts are how the OS learns anything happened; top/bottom-half design is a classic depth probe, and softirq/CPU0 saturation is a real prod incident.

2. Core Concept

An interrupt is a hardware signal that preempts the CPU and vectors it to a kernel handler. Types: hardware IRQs (NIC, disk, timer), exceptions/faults (page fault, divide-by-zero — synchronous), software interrupts (syscall entry historically). The timer interrupt is what makes preemptive scheduling possible at all.

3. Internal Working

1. Device raises IRQ (modern: MSI-X message) → interrupt controller (APIC) routes to a CPU.
2. CPU finishes current instruction, saves minimal state, jumps via the interrupt descriptor table to the handler.
3. **Top half**: minimal, interrupts-disabled work — ack device, grab data pointer, schedule deferred work.
4. **Bottom half** (softirq/tasklet/workqueue/NAPI poll): the heavy lifting (e.g., TCP processing) runs later with interrupts enabled.
5. Return restores the preempted context (possibly triggering a reschedule). Affinity: each IRQ can be steered to CPUs (`/proc/irq/*/smp_affinity`); RSS/RPS/RFS spread network processing across cores.

4. ASCII Diagram

```
NIC gets packet -> MSI-X IRQ -> APIC -> CPU3
CPU3: [save ctx] -> top half: ack, queue skb, schedule NAPI (us, IRQs off)
      [restore ctx]
later: softirq NET_RX on CPU3: TCP/IP processing -> socket buffer
      -> wake blocked reader (Module 8.1 wakeup!)
Watch: %irq/%soft in mpstat; single-queue NIC = one hot CPU.
```

5. Real Production Example

- “CPU0 at 100% softirq, others idle” — all NIC queues/IRQs pinned to one core; fix with multi-queue NIC + irqbalance/affinity + RPS. Common on packet-heavy proxies/LBs.
- Interrupt storms from a faulty device freezing a box; interrupt coalescing settings (ethtool -C) trading latency for CPU.

6. Advantages

Zero CPU cost while idle; μ s-scale reaction; the foundation of preemption, timers, and I/O completion.

7. Trade-offs

Per-event overhead (context save, cache pollution); storms under load (→ NAPI); handler constraints (no sleeping in top half); jitter for latency-critical threads (hence IRQ isolation on trading/RT boxes).

8. Common Mistakes

- No top/bottom-half story (“the handler processes the packet” — no; it defers).
- Not knowing page faults are (synchronous) interrupts too — same vector machinery.
- Missing softirq as a triage category (`%soft` in mpstat, `/proc/softirqs`).

9. Performance Implications

Interrupt $\approx \mu$ s each including cache damage; at 100k+ events/s per core, coalescing/NAPI mandatory. ksoftirqd running hot = deferred work exceeding budget = the box is network-saturated even if “CPU looks free” elsewhere.

10–11. Interview & Follow-ups

- “Packet arrives → my `read()` returns: full path.” (IRQ → NAPI/softirq → socket buffer → wait-queue wakeup → scheduler → read copies)
- “Why split top/bottom halves?” “What is interrupt coalescing and its trade-off?”

12. Coding/Debugging Scenario

Load balancer drops packets at 40% total CPU: `mpstat -P ALL` shows CPU0 100% %soft → distribute NIC queue IRQs across cores, enable RPS, retest.

13. Best Practices

Balance IRQs on multi-queue NICs; isolate latency-critical cores from IRQs; watch `%irq/%soft` and `/proc/softirqs` deltas in triage.

14. Practice Questions

1. Trace a keystroke and a 10 GbE packet through interrupt handling — what differs?
2. When would you *increase* interrupt coalescing, and what latency cost do you accept?

8.5 DMA (Direct Memory Access)

1. Why Interviewers Ask This

Completes the I/O picture: how bytes actually move without burning CPU, and the substrate of “zero-copy” — a favorite senior topic via `sendfile`/Kafka.

2. Core Concept

DMA lets devices read/write RAM directly, without the CPU copying byte-by-byte (PIO). CPU’s role shrinks to: set up the transfer (addresses, lengths), get an interrupt on completion. Modern NICs/NVMe are DMA engines with rings of descriptors.

3. Internal Working

- Driver builds descriptor rings (scatter-gather lists of physical addresses); device DMAs payloads into/out of those buffers autonomously; completion raises an IRQ (coalesced).
- **IOMMU** translates/limits device addresses (protection against rogue DMA; enables VM device passthrough).
- **Zero-copy** path (`sendfile`, Kafka): disk →DMA→ page cache →(device DMAs straight from page cache)→ NIC. CPU copies: **zero**. Classic read+write path: 2 DMA + 2 CPU copies + 4 context switches; `sendfile`: 2 DMA + ~0 CPU copies.

4. ASCII Diagram

Classic file->socket:	<code>sendfile (zero-copy):</code>
disk -DMA-> pagecache	disk -DMA-> pagecache
pagecache -CPU copy-> user buf	pagecache --(NIC DMAs directly,
user buf -CPU copy-> socket buf	scatter-gather + headers)--> wire
socket buf -DMA-> NIC	
2 DMA + 2 CPU copies	2 DMA + 0 CPU copies

5. Real Production Example

Kafka’s `sendfile` is the canonical story (serving MBs/s per core of consumer traffic with almost no CPU); Nginx `sendfile on`; for static content; NVMe drives doing millions of IOPS are only possible because DMA + deep queues remove the CPU from the data path; RDMA extends the idea across the network (remote DMA — HPC/storage fabrics).

6. Advantages

CPU freed for compute; enables line-rate I/O; scatter-gather removes buffer-assembly copies; foundation of every zero-copy API.

7. Trade-offs

Setup overhead (descriptors, mappings) — not worth it for tiny transfers; cache-coherence management (device wrote RAM behind the CPU's back); security (needs IOMMU); physical-memory pinning for buffers.

8. Common Mistakes

- “Zero-copy means no copies at all” — DMA transfers remain; *CPU* copies are eliminated.
- Not knowing `sendfile` can't apply when data must be transformed (TLS used to break it; kernel TLS/kTLS restores it).
- Forgetting DMA needs pinned/physically-addressable memory (interaction with paging).

9. Performance Implications

Copy cost ~ per-GB CPU time (`memcpy` at ~10 GB/s/core): pushing 40 Gb/s through 2 extra copies eats cores; zero-copy reclaims them. Small messages: `syscall/setup` dominates, zero-copy irrelevant — batch instead.

10–11. Interview & Follow-ups

- “Explain how Kafka serves consumers with near-zero CPU.” “Count the copies in read+write vs `sendfile`.” “What's an IOMMU for?” “When does zero-copy *not* help?” (tiny payloads, transformations, TLS w/o kTLS)

12. Coding/Debugging Scenario

Static-file CDN node CPU-bound at high throughput: perf shows `copy_user` dominant → enable `sendfile/splice` path (+kTLS if TLS), CPU drops multi- $\times$.

13. Best Practices

Use `sendfile/splice/kTLS` for bulk unmodified data; batch small I/O; keep hot buffers aligned/pinned where APIs require.

14. Practice Questions

1. Diagram all copies/DMA for: (a) `read()+write()` proxying, (b) `sendfile`, (c) `splice` between two sockets, (d) `io_uring` with registered buffers.
2. Your service TLS-encrypts everything — does `sendfile` help? What changes with kTLS?

8.6 `select()`

1. Why Interviewers Ask This

The historical baseline of I/O multiplexing; its limitations are the exam — they motivate `poll` and `epoll`. “`select` → `poll` → `epoll` evolution” is a canonical interview arc.

2. Core Concept

`select(nfds, readfds, writefds, exceptfds, timeout)`: pass bitmaps of FDs; kernel blocks until any is ready; returns *modified bitmaps* you must scan. One thread watches many FDs → the first escape from thread-per-connection.

3. Internal Working

Per call: copy three bitmaps in, kernel iterates **every FD 0..nfds-1**, attaches to each file's poll wait queue, sleeps; on any event, wakes, re-checks all, copies bitmaps out. You then loop over all FDs testing `FD_ISSET`. Bitmaps are destroyed each call → re-build every iteration.

4. ASCII Diagram

```
fd_set bits: [0..1023] <- FD_SETSIZE hard ceiling!
loop:
  FD_ZERO/FD_SET(all fds)          0(n) userspace
  select(...)                      0(n) kernel scan + copy in/out
  for fd in all: if FD_ISSET ...    0(n) userspace scan
Cost per event ~ 0(n_total_fds) -> collapses at high n
```

5. Real Production Example

Legacy portable daemons and simple tools; still the only truly universal API (every OS). Any modern server on select at scale is a bug — the 1024 limit has caused real outages when connection counts crept up (silent corruption if `fd ≥ FD_SETSIZE` is `FD_SET!`).

6. Advantages

Ubiquitous/portable; fine for a handful of FDs; microsecond timeout precision (timeval).

7. Trade-offs

`FD_SETSIZE=1024` hard cap; $O(n)$ per call in kernel *and* user space; bitmap rebuild each call; three copies per call; no extra per-FD data.

8. Common Mistakes

- Not stating the 1024 limit and $O(n)$ scans (the two headline flaws).
- Forgetting the sets are input-output (must rebuild).
- Believing “exceptfds = errors” (it’s OOB data mostly; errors show on read/write).

9. Performance Implications

10k FDs, 100 ready: select touches all 10k three-plus times per loop → CPU melts at ~thousands of FDs; epoll touches only the 100 ready. This asymptotic gap *is* the interview point.

10–11. Interview & Follow-ups

- “Limitations of select?” (rehearse: cap, $O(n)$, rebuild, copies) “Why does the kernel have to scan everything?” (stateless API — kernel keeps no interest set between calls)

12. Coding/Debugging Scenario

Legacy gateway breaks mysteriously above ~1000 connections: fd numbers exceed `FD_SETSIZE` → memory corruption via `FD_SET`; migrate to poll/epoll immediately.

13. Best Practices

New code: never select. Reading legacy: know its shape. Cross-platform: use libevent/libuv abstractions.

14. Practice Questions

1. Write the canonical select echo-server loop; annotate every $O(n)$ step.
2. Why can’t select’s API be fixed without becoming... poll/epoll? (stateless bitmaps are the API)

8.7 poll()

1. Why Interviewers Ask This

The intermediate step: fixes select’s cap, keeps its asymptotics — tests whether you understand *which* problem each API solved.

2. Core Concept

`poll(fds[], nfds, timeout)`: array of `pollfd {fd, events, revents}` instead of bitmaps. No `FD_SETSIZE` limit; input (events) and output (revents) separated → no rebuild; richer event set (`POLLIN/OUT/ERR/HUP/RDHUP`).

3. Internal Working

Same kernel engine as `select`: copy the whole array in, iterate all entries, attach to wait queues, sleep, wake, re-scan, copy out. Still **$O(n)$ per call**, still stateless between calls — the kernel forgets your interest set every time.

4. ASCII Diagram

```
struct pollfd fds[] = {{fd:5, events:POLLIN}, {fd:9, events:POLLIN|POLLOUT}}
poll(fds, 2, timeout)
-> kernel scans all n entries ( $O(n)$ ), fills revents
-> you scan all n entries for revents != 0 ( $O(n)$ )
Fixed vs select: no 1024 cap, no rebuild. Same:  $O(n)$ , full copy per call.
```

5. Real Production Example

Portable middleware and moderate-connection tools; `ppoll` for signal-safe waiting. Historically Apache pre-event MPMs. Anything beyond ~1k active FDs moved to `epoll/kqueue`.

6. Advantages

Unlimited FDs; cleaner API; portable (POSIX everywhere); per-FD event richness.

7. Trade-offs

$O(n)$ scan + $O(n)$ copy per call unchanged → same collapse at scale; array of 12-byte structs copied every call (10k FDs = 120 KB per syscall!).

8. Common Mistakes

- “poll fixed select’s performance” — it fixed the *limit and API*, not the asymptotics.
- Not resetting/ignoring revents properly; missing `POLLHUP` handling (peer close).

9. Performance Implications

Identical $O(n)$ core: fine to ~1k FDs, painful at 10k, impossible at 100k. The measured `select` → `poll` → `epoll` benchmark curve (flat vs flat vs constant-per-event) is worth citing.

10–11. Interview & Follow-ups

- “select vs poll — exactly what changed?” “Why is poll still $O(n)$ and what API change removes that?” (stateful registration → `epoll`)

12. Coding/Debugging Scenario

Profiling a 5k-connection poll server: 60% CPU inside poll+scan loops with 1% connections active → `epoll` migration cuts CPU to noise.

13. Best Practices

poll for small-scale portable code; `epoll/kqueue` beyond ~1k FDs; `libuv/libevent` to abstract.

14. Practice Questions

1. Compute bytes copied per second by poll with 20k FDs at 1k wakeups/s.
2. Convert a select loop to poll — list every semantic difference you must handle.

8.8 epoll()

1. Why Interviewers Ask This

The engine of Nginx/Redis/Netty/Node and the star of “how do you handle 1M connections?” — the most bankable I/O answer in backend interviews.

2. Core Concept

epoll makes the kernel *remember your interest set*: register FDs once (`epoll_ctl`), then `epoll_wait` returns **only ready FDs**. Cost per wait $\propto$ number of *ready* events, not registered FDs — $O(1)$ -ish per event vs poll's $O(n)$.

3. Internal Working

- `epoll_create1` → kernel object: red-black tree (registered FDs) + **ready list**.
- `epoll_ctl(ADD)` → insert into tree, hook a callback into the file's wait queue.
- Event happens (packet arrives) → the file's wakeup callback appends it to the ready list (event-driven — no scanning!).
- `epoll_wait` → pop the ready list, copy those events out; sleep only if empty.
- **LT (level-triggered, default)**: fd reported *while* readable — forgiving; may re-report.
- **ET (edge-triggered)**: reported only on *transitions* (empty → data). You must read until EAGAIN or you lose wakeups forever — pairs with `O_NONBLOCK` mandatorily; fewer wakeups, sharper hazard.
- `EPOLLONESHOT` for multi-threaded dispatch; epoll fd is itself pollable (nestable).

4. ASCII Diagram

```
epoll instance:
[RB-tree: all 1,000,000 registered fds]
[ready list: {fd42, fd977}] <- appended by wakeup callbacks
epoll_wait -> returns 2 events (not 1M scans!)

ET pitfall: 2 packets arrive, you read 1 -> buffer still has data,
            NO new edge -> you sleep forever. ET rule: drain to EAGAIN.
```

5. Real Production Example

Nginx (worker per core, epoll ET), Redis (single-threaded epoll LT — event loop *is* the server), Netty/Java NIO, libuv/Node, HAProxy, Envoy. WhatsApp-scale “millions of connections per box” stories are epoll/kqueue (+tuning) stories.

6. Advantages

$O(\text{ready})$ not $O(\text{registered})$; kernel-resident interest set (no per-call copies); scales to millions of idle connections; ET minimizes wakeups; thread-safe ctl.

7. Trade-offs

Linux-only; ET correctness burden (drain loops, starvation of other fds if one fd streams forever — need fairness budgets); thundering herd when multiple threads wait on one instance (`EPOLLEXCLUSIVE` helps); regular files always report ready (useless for disk I/O — `io_uring`'s job).

8. Common Mistakes

- Explaining epoll as “faster poll” without the *stateful registration + ready list* mechanism.
- ET without draining to EAGAIN (the classic stuck-connection bug); ET with blocking fds.
- Assuming epoll helps file reads.

9. Performance Implications

1M idle conns, 1k active: poll $\approx$ scans 1M entries per wakeup; epoll returns 1k events — orders of magnitude. Syscall count becomes the next bottleneck (→ batching, `io_uring`). C10K solved; C10M = epoll + affinity + `SO_REUSEPORT` + NIC tuning.

10–11. Interview & Follow-ups

- “How does epoll achieve O(1) per event? What data structures?” “LT vs ET — semantics, bugs, when each?” “How does Nginx use epoll across workers?” (SO_REUSEPORT / EPOLLEXCLUSIVE vs accept mutex)

12. Coding/Debugging Scenario

An ET-based server occasionally freezes single connections under burst → partial drain bug; add read-until-EAGAIN loop + per-fd fairness cap; connections recover.

13. Best Practices

LT unless you need ET’s wakeup economy; always O_NONBLOCK; drain loops with byte budgets; one epoll per worker thread (shared-nothing) beats shared instances.

14. Practice Questions

1. Build the event-loop skeleton: `epoll_create/ctl/wait` + `accept` + `echo`, LT then ET — list every code change ET forces.
2. Design “1M websockets on one box”: epoll strategy, memory budget per conn, accept distribution, backpressure.

8.9 kqueue()

1. Why Interviewers Ask This

Breadth check (BSD/macOS) and a design-comparison probe: `kqueue` is widely considered the *cleaner* design — one API for many event types.

2. Core Concept

BSD/macOS’s scalable event API: a **kernel event queue** plus one syscall, `kevent()`, that both registers changes and reaps events. Filters generalize beyond FDs: `EVFILT_READ/WRITE`, `EVFILT_TIMER` (timers), `EVFILT_PROC` (process exit/fork), `EVFILT_SIGNAL`, `EVFILT_VNODE` (file changes — powers fsevents-style watching).

3. Internal Working

`kq = kqueue()`; `kevent(kq, changelist, nchanges, eventlist, nevents, timeout)` — submit registrations *and* collect completions in one call (batching built-in; epoll needs one `epoll_ctl` syscall per registration). Each `kevent` = (ident, filter, flags, udata...): per-event user data pointer, `EV_CLEAR` for edge-triggered per event, `EV_ONESHOT`. Same O(ready) ready-queue architecture as epoll.

4. ASCII Diagram

```
one call does both:
kevent(kq, [ADD fd7 READ, ADD timer 100ms], 2, <- register (batched!)
          events_out, 64, timeout)           <- reap
Filters: READ | WRITE | TIMER | SIGNAL | PROC | VNODE | USER
epoll equivalents need: epoll_ctl xN + timerfd + signalfd + pidfd + inotify
```

5. Real Production Example

Nginx/HAProxy/Netty/libuv on FreeBSD/macOS use `kqueue` automatically; Netflix’s FreeBSD-based Open Connect CDN appliances (famous 100s-of-Gb/s per box) run `kqueue`+`sendfile` stacks; macOS dev tooling (file watchers) rides `EVFILT_VNODE`.

6. Advantages

Unified event model (fds, timers, signals, processes, file changes — one queue, one wait); syscall batching by design; per-event udata (no separate lookup map); clean ET semantics per-event.

7. Trade-offs

Not on Linux (portability splits: epoll vs kqueue vs IOCP → why libuv/libevent exist); fewer tuning knobs in some areas; same regular-file limitation for disk reads.

8. Common Mistakes

- Only knowing “kqueue = BSD epoll” — mention filters/batching/udata to show real breadth.
- Assuming code ports directly (semantics differ: registration model, triggering flags).

9. Performance Implications

Same asymptotics as epoll ($O(\text{ready})$); batching registration can beat epoll at high churn (many short-lived connections = many `ctl` calls). In practice both saturate NICs before the API is the limit.

10–11. Interview & Follow-ups

- “Compare epoll and kqueue designs — which is cleaner and why?” “How do you watch a timer + socket + child process in one loop on each platform?” (kqueue natively; Linux: `timerfd`/`signalfd`/`pidfd` all as `fds` into epoll — note how Linux converges by *making everything an fd*)

12. Coding/Debugging Scenario

Porting a Linux epoll server to macOS for dev laptops: introduce libuv (or a small event-loop shim) instead of hand-porting; verify ET assumptions against `EV_CLEAR`.

13. Best Practices

Write against an abstraction (libuv/libevent/tokio/Netty) unless you’re building the abstraction; test event-loop code on both platforms in CI.

14. Practice Questions

1. Sketch a kqueue loop handling: 10k sockets, a 1 s heartbeat timer, `SIGTERM`, and a child-process exit — no extra `fds` needed.
2. Table: `select` / `poll` / `epoll` / `kqueue` / `IOCP` / `io_uring` — model (readiness/completion), asymptotics, portability, killer feature.

Module 8 Cheat Sheet (one page)

Taxonomy: blocking vs non-blocking = does the *call* sleep; sync vs async = who performs the I/O; readiness (epoll/kqueue: “you may read”) vs completion (io_uring/IOCP: “read done”).

API	Model	Cost/call	Limit	Killer fact
<code>select</code>	readiness	$O(n)$ scan $\times 3$	1024 <code>fds</code>	bitmaps rebuilt every call
<code>poll</code>	readiness	$O(n)$ scan+copy	none	fixed cap, not asymptotics
<code>epoll</code>	readiness	$O(\text{ready})$	Linux only	RB-tree + ready list; LT vs ET (drain to EAGAIN!)
<code>kqueue</code>	readiness	$O(\text{ready})$	BSD/macOS	filters: timers/signals/procs too; batched kevent
<code>io_uring</code>	completion	~ 0 syscalls (rings)	Linux 5.x+	async <i>file</i> I/O; SQ/CQ shared rings

Data path: IRQ (top half: `ack+defer`) → `softirq`/NAPI (protocol work) → socket buffer → wait-queue wakeup → epoll ready list → your `read()`. **Polling vs interrupts:** poll when events frequent (NAPI switches adaptively; DPDK always); interrupt when rare. **DMA/zero-copy:** `sendfile` = disk → pagecache → NIC, 0 CPU copies (Kafka/Nginx); zero-copy ≠ zero transfers; TLS needs kTLS. **Traps:** `O_NONBLOCK` on files ≈ no-op; ET without drain = frozen connection; blocking call in event loop = frozen server; partial writes must be tracked.

Top Interview Questions

1. Blocking/non-blocking × sync/async — 4-quadrant table with examples.
2. select vs poll vs epoll — evolve the design, state each fix.
3. epoll internals: RB-tree, ready list, callbacks; LT vs ET with the classic ET bug.
4. Packet arrival → read() returns: the full kernel path.
5. How does Kafka/Nginx serve at near-zero CPU? (sendfile/DMA copy math)
6. When is polling better than interrupts? (NAPI/DPDK)
7. What does io_uring add over epoll?
8. Design: 1M concurrent connections on one box.

Common Mistakes (module-wide)

- “epoll is async I/O”; “non-blocking = async”.
- ET without drain-to-EAGAIN; blocking calls on the loop; unhandled partial I/O.
- “poll fixed select’s performance”; forgetting FD_SETSIZE corruption.
- “zero-copy = no copies”; expecting O_NONBLOCK/epoll to help disk files.

Mock Interview (self-test, ~25 min)

1. (Design) Chat backend: 2M idle websockets, 20k msgs/s, 16 cores. Full I/O architecture: API choice, threading, accept strategy, backpressure, file I/O plan.
2. (Depth) Interviewer: “epoll_wait just returned fd 42 readable.” Walk backwards to the electrons: every kernel structure and event that made that true.
3. (Code) Implement ET-mode echo handling: the exact drain loop, EAGAIN, partial write buffering, EPOLLOUT re-arm.
4. (Prod) Proxy at 40% CPU drops packets; mpstat: CPU0 100% %soft. Diagnose and fix (IRQ affinity/RSS/NAPI story).
5. (Trap) “We made all our file reads non-blocking with O_NONBLOCK so the loop never stalls.” What actually happened, and name two real fixes (thread pool, io_uring).

MODULE 9 — Linux Concepts

Senior SWE Interview Track — Operating Systems

9.1 fork()

1. Why Interviewers Ask This

“What does fork return?” and “how many processes does this loop create?” are evergreen screens; the senior layer is COW mechanics and fork’s hazards in threaded/big processes.

2. Core Concept

`fork()` clones the calling process: child gets a copy of the address space (COW — Module 5.8), FD table (sharing open-file descriptions/offsets!), signal handlers, cwd, environment. Returns **twice**: child’s PID in the parent, **0 in the child**, -1 on failure. Only the calling thread exists in the child.

3. Internal Working

`fork` → `clone()` with child-copy flags → duplicate `task_struct`, copy page tables with all writable pages marked read-only (COW), bump file refcounts, copy signal table; child enters the run queue. Cost ∝ mapped memory (page-table copy), not RSS content.

4. ASCII Diagram

```
pid = fork();
    parent          child
pid = 1234 (child's pid)  pid = 0
same code, same line, COW-shared memory, SHARED file offsets

for (i=0;i<3;i++) fork();  -> 2^3 = 8 processes total (draw the tree!)
```

5. Real Production Example

Shells (fork+exec per command), Nginx/Postgres worker spawning, Redis BGSAVE (fork for COW snapshot), Unicorn/uWSGI prefork models, Android Zygote.

6. Advantages

Simple process creation with full context inheritance; COW makes it cheap; natural prefork server pattern; snapshot semantics.

7. Trade-offs

Threads don’t survive (locks held by dead threads → child deadlocks in malloc — the classic!); page-table copy cost for huge processes; COW spike risk; fork in a 60 GB process can fail on overcommit settings.

8. Common Mistakes

- Return-value confusion (0 = you’re the child; not “child gets nothing”).
- The fork-loop count: `n` forks in a loop → 2^n processes.
- Forgetting FD/offset sharing (parent and child interleave writes to the same log offset — actually fine for `O_APPEND`, corrupting otherwise).
- Calling non-async-signal-safe functions between fork and exec in threaded programs.

9. Performance Implications

fork of a small process ~100 μs; of a 60 GB-mapped process ~tens of ms (page tables) + COW faults after.

`posix_spawn/vfork` avoid the copy when you’ll exec immediately.

10–11. Interview & Follow-ups

- “Output of this fork snippet?” (trace both branches; note buffered stdout duplication — printf without newline before fork prints twice!)
- “Why is fork dangerous in multithreaded programs?” “fork vs vfork vs posix_spawn vs clone?”

12. Coding/Debugging Scenario

Python service using multiprocessing (fork) hangs sporadically: a thread held a logging lock at fork time → child deadlocks; fix with spawn start-method or fork-server, or `pthread_atfork` handlers.

13. Best Practices

fork+exec immediately in threaded programs (or posix_spawn); flush stdio before fork; prefer spawn-type APIs in high-level languages.

14. Practice Questions

1. How many processes and what's printed: `fork() && fork() || fork();` — work it through.
2. Why does Redis recommend `vm.overcommit_memory=1`? (fork of huge process would fail conservative overcommit check)

9.2 exec()

1. Why Interviewers Ask This

fork/exec separation is *the* Unix design idea — interviewers test that you know exec **replaces** the image and why the split is powerful.

2. Core Concept

`exec*()` (execve and its wrappers execl/execv/execvp...) replaces the current process image with a new program: new code/data/heap/stack — **same PID**, same open FDs (minus `O_CLOEXEC` ones), same cwd/credentials (unless setuid binary). On success it **never returns**; code after exec runs only on failure.

3. Internal Working

`execve(path, argv, envp)`: kernel opens the binary, parses ELF (or shebang), tears down old VMAs, maps new segments (demand-paged), maps the dynamic linker if needed, resets signal handlers to default (ignored dispositions persist), preserves FD table, jumps to entry point. The fork/exec split is why redirection is trivial: *between* fork and exec, the child rearranges its own FDs (`dup2(logfd, 1)`) before exec — no API needed on exec itself.

4. ASCII Diagram

```
shell: ls > out.txt
fork() ----- child:
                  open("out.txt") -> fd 5
                  dup2(5, 1)        // stdout now the file
                  close(5)
                  execve("/bin/ls", ...) // image replaced, fd table kept
parent: wait(...)
Same PID before/after exec; new program inherits fds 0,1,2 as arranged.
```

5. Real Production Example

Every shell pipeline and container entrypoint; supervisors (systemd, runit) fork+exec services; CGI/subprocess APIs (`subprocess.run`, `ProcessBuilder`) are fork/(vfork/clone)+exec underneath.

6. Advantages

Orthogonality: process creation (fork) and program loading (exec) compose — redirection, pipes, environment setup all become child-side code between the two calls.

7. Trade-offs

Two syscalls where Windows has one (CreateProcess); fork's COW cost paid even when exec discards it (mitigations: vfork/posix_spawn); FD leaks into new programs without O_CLOEXEC (security issue).

8. Common Mistakes

- “exec creates a new process” — no; same process, new program.
- Writing code after exec expecting it to run (it runs only on failure — always perror+exit there).
- Forgetting FD inheritance → leaked sockets/db handles in child programs (set CLOEXEC).

9. Performance Implications

exec cost = ELF mapping + dynamic linking (demand paging defers most); large static binaries exec faster to first instruction than heavy dynamic linking; frequent short-lived fork+exec (shell-out per request!) is a real performance antipattern.

10–11. Interview & Follow-ups

- “How does the shell implement `a | b > f`?” (fork ×2, pipe, dup2 choreography, exec)
- “What survives exec?” (PID, FDs sans CLOEXEC, cwd, umask, ignored signals) “What doesn't?” (memory, handlers, threads)

12. Coding/Debugging Scenario

A spawned child inherits the parent's listening socket → parent can't rebind after restart (“address in use”, ghost listener) → set SOCK_CLOEXEC.

13. Best Practices

O_CLOEXEC everywhere by default; check exec failure explicitly; use posix_spawn/subprocess APIs rather than raw fork+exec in threaded services.

14. Practice Questions

1. Implement `ls | grep foo` in C: pipe(), two forks, dup2s, execs, waits — write it.
2. Why do setuid binaries + exec need extra care with inherited FDs and environment? (privilege escalation surface)

9.3 wait() / waitpid()

1. Why Interviewers Ask This

wait is the reaping half of the process lifecycle — it's how you *prevent zombies*, and interviewers chain it directly into the zombie/orphan questions.

2. Core Concept

`wait(&status)` blocks until a child changes state (usually exits), returns its PID and status (exit code via `WEXITSTATUS`, killing signal via `WTERMSIG`). `waitpid(pid, &status, options)` targets a specific child; `WNOHANG` makes it non-blocking (poll for finished children). Reaping frees the child's PCB — until then it's a zombie.

3. Internal Working

Exited child → state Z, holds only PCB (exit status, rusage); kernel sends `SIGCHLD` to the parent. `wait*` finds a zombie child, copies status out, frees the task. If the parent ignores `SIGCHLD` explicitly (`SA_NOCLDWAIT`), children are auto-reaped.

4. ASCII Diagram

```
parent: fork -> child ... child exits -> [ZOMBIE: pcb only]
parent: wait(&status) -----> reaps: pcb freed
patterns:
  blocking:  pid = wait(&st);
  targeted:  waitpid(cpid, &st, 0);
  poll-all:  while ((p=waitpid(-1,&st,WNOHANG))>0) reap(p);  // in SIGCHLD handler
```

5. Real Production Example

Shells report exit codes via wait; systemd/supervisord reap and restart services; container init (tini/dumb-init) exists to reap orphaned zombies inside PID namespaces; CI runners harvest job exit statuses.

6. Advantages

Reliable exit-status delivery; resource cleanup; parent-driven lifecycle (restart policies, exit-code semantics).

7. Trade-offs

Blocking wait stalls the parent (use SIGCHLD+WNOHANG loop or a reaper thread); signal-based reaping has classic races (multiple children, coalesced SIGCHLD → must loop WNOHANG).

8. Common Mistakes

- Reaping one child per SIGCHLD (signals coalesce! loop until WNOHANG returns 0).
- Ignoring status macros (raw status int is encoded — use WIFEXITED/WEXITSTATUS).
- Believing kill -9 removes zombies (they're already dead; only reaping or parent death helps).

9. Performance Implications

Unreaped zombies leak PIDs → `fork: retry: Resource temporarily unavailable` at `pid_max/threads-max`; each zombie is tiny (KBs) but PID exhaustion halts the box.

10–11. Interview & Follow-ups

- “How do you run 100 children and collect all exit codes without blocking sequentially?”
- “What exactly does WNOHANG change?” “What happens if the parent never waits?” (zombies → next topic)

12. Coding/Debugging Scenario

Job runner accumulating thousands of Z processes: SIGCHLD handler reaps once per signal → convert to `while(waitpid(-1,&st,WNOHANG)>0)` loop; zombies drain.

13. Best Practices

Always reap; WNOHANG loops in handlers; in containers, run a real init (tini) as PID 1; propagate child exit codes meaningfully.

14. Practice Questions

1. Write the canonical SIGCHLD reaper (async-signal-safe: only waitpid + write).
2. Parent spawns children A(exit 0) and B(killed by SIGSEGV): show status decoding for both.

9.4 Zombie Process

1. Why Interviewers Ask This

Top-5 Linux interview question and a real incident type (“10,000 defunct processes”). Tests the exit-status handshake and containers’ PID-1 problem.

2. Core Concept

A zombie (Z, `<defunct>`) is a process that has **exited** but whose parent hasn't `wait()`ed yet. Only the PCB survives (PID, exit status, rusage) — no memory, no FDs, no CPU. It exists *by design*: the kernel must keep the exit status until the parent collects it.

3. Internal Working

exit → free memory/FDs → state Z → SIGCHLD to parent → parent `wait()` → PCB freed. If the parent exits first, the zombie is **reparented** (to `init/subreaper`) which reaps it. Zombies are un-killable (already dead; signals are meaningless). Fixing a live incident = make the *parent* reap: fix its code, or kill the parent so `init` inherits and reaps.

4. ASCII Diagram

```
child exits          parent wait()s
[running] --exit--> [ZOMBIE: pid+status only] --reap--> gone
                    ^ kill -9: NO EFFECT (already dead)

parent dies first:
[ZOMBIE] --reparent--> init/subreaper --auto-reap--> gone
Danger: buggy parent + spawn loop -> Z count grows -> PID exhaustion
```

5. Real Production Example

- Docker container whose entrypoint is your app as PID 1: your app never reaps grandchildren → zombies accumulate → solved by `--init`/tini (why that flag exists — great interview detail).
- CI agents/process-pool managers with broken SIGCHLD handling filling `ps` with `<defunct>` until fork fails fleet-wide.

6. Advantages

(Of the mechanism) Guarantees no exit status is ever lost; enables reliable supervision trees.

7. Trade-offs

Requires parental diligence; PID-space leak when neglected; confuses monitoring (process count alarms).

8. Common Mistakes

- “Zombies waste memory/CPU” — effectively neither; they waste **PIDs**.
- Trying `kill -9` on zombies; not knowing the kill-the-parent remediation.
- Confusing zombie (dead, unreaped) with orphan (alive, parentless) — next topic.

9. Performance Implications

Each zombie ≈ one `task_struct` (~KBs) + a PID; `kernel.pid_max/threads-max` exhaustion stops all process creation on the host — outage via bookkeeping.

10–11. Interview & Follow-ups

- “What is a defunct process and how do you get rid of it?” “Why can't you kill it?” “Why does `docker run --init` exist?”

12. Coding/Debugging Scenario

`ps aux | grep defunct | wc -l` = 30k, forks failing: identify the common PPID (`ps -eo ppid,stat | awk '$2~/Z/'` → count by ppid), fix or bounce that parent, add tini for containers.

13. Best Practices

Reap religiously (9.3 patterns); real `init` as PID 1 in containers; alert on Z-state counts and PID utilization.

14. Practice Questions

1. Write a 5-line C program that intentionally creates a zombie for 60 s; then modify to prevent it two different ways (`wait`; ignore SIGCHLD with `SA_NOCLDWAIT`).

2. Explain the full remediation ladder for 30k zombies with parent PID 812.

9.5 Orphan Process

1. Why Interviewers Ask This

The paired concept with zombies; also the mechanism behind daemonization and a real container gotcha (PID 1 semantics).

2. Core Concept

An orphan is a *live* process whose parent exited. The kernel **reparents** it to **init** (PID 1) — or the nearest **subreaper** (`prctl(PR_SET_CHILD_SUBREAPER)`, used by systemd/tmux/container inits). Init reaps it on exit, so orphans don't become permanent zombies. Orphans are normal and harmless per se — daemonization *deliberately* orphans (fork, parent exits, child continues under init).

3. Internal Working

Parent exit → kernel walks its children → reparent to subreaper-or-init → SIGCHLD flows there henceforth. Related: **orphaned process groups** and terminal control — background jobs may receive SIGHUP when the session leader dies (why `nohup`/`setsid`/`disown` exist). In containers, *your app* is PID 1: it inherits every orphan and must reap them (zombie link), and it doesn't get default signal handling (SIGTERM ignored unless you install a handler — the classic “container won't stop, Docker waits 10 s then SIGKILLs” bug).

4. ASCII Diagram

```

init(1)                                init(1)
  \                                     | \
  parent (exits!) ==>                 |  child (ORPHAN, alive, adopted)
    \                                 |
    child (running)                 SIGCHLD now goes to init; reaped on exit

daemonize: fork -> parent exits -> setsid() -> fork again -> daemon under init

```

5. Real Production Example

- SSH session dies → your long-running script gets SIGHUP and dies with it (unless `nohup`/tmux) — orphan/session semantics in action.
- Kubernetes pod: app as PID 1 ignoring SIGTERM → every deploy takes `terminationGracePeriod` and ends in SIGKILL → dropped in-flight requests. Fixed by proper signal handling or tini.

6. Advantages

Automatic adoption keeps the process tree consistent and guarantees eventual reaping; enables daemons and supervisors.

7. Trade-offs

Orphans escape their original supervision (restart logic loses them); accidental orphaning leaks “runaway” workers that keep consuming resources after their manager died.

8. Common Mistakes

- Zombie/orphan confusion (dead-unreaped vs alive-adopted) — have the one-line contrast ready.
- Not knowing subreapers exist (modern systemd/user session behavior).
- Assuming PID 1 in a container behaves like a normal process (signals! reaping!).

9. Performance Implications

Minimal by itself; the cost is operational — orphaned workers double-processing jobs, or holding locks/files their dead parent was supposed to manage.

10–11. Interview & Follow-ups

- “Parent dies before child — what happens? Who reaps?” “How does daemonization use orphaning?” “Why do containers need tini / signal handlers?”

12. Coding/Debugging Scenario

Deploy kills a job-manager but its 40 workers keep running (now under init), double-consuming queues → use process groups + kill(-pgid), cgroup-based teardown (systemd/K8s does this), or supervisor-managed PIDs.

13. Best Practices

Kill by process group/cgroup, not single PID; handle SIGTERM in anything that may run as PID 1; use subreapers for supervisors that spawn deep trees.

14. Practice Questions

1. Contrast zombie vs orphan across: alive?, PPID after event, who reaps, is it a problem?
2. Write the classic double-fork daemonization and explain each step's purpose.

9.6 Signals

1. Why Interviewers Ask This

Signals are the Unix control plane: kill, Ctrl-C, graceful shutdown, timeouts. Interviews probe SIGKILL vs SIGTERM, handler safety, and the kill-9-mythology.

2. Core Concept

Signals = asynchronous per-process (or per-thread-directed) notifications. Dispositions: default (terminate/core/ignore/stop), ignored, or **handled** (user function). Key set: SIGTERM (polite terminate, *catchable*), **SIGKILL (9)** and **SIGSTOP** (uncatchable, unblockable), SIGINT (Ctrl-C), SIGSEGV (fault), SIGCHLD (child state), SIGHUP (terminal death; repurposed as “reload config”), SIGPIPE (write to closed pipe — the silent killer of naive network code), SIGUSR1/2 (app-defined).

3. Internal Working

- Sender (**kill**, kernel fault, terminal driver) sets a bit in the target's pending mask; delivery happens at kernel → user transition (syscall return / interrupt return): kernel rigs the user stack to run the handler, then **sigreturn** restores context.
- Standard signals **don't queue** — 5 SIGCHLDs may deliver once (why reap loops!); real-time signals (SIGRTMIN+) queue with payloads.
- Per-thread masks (**pthread_sigmask**); process-wide handlers. Blocking syscalls may return **EINTR** when a signal arrives (handle-or-SA_RESTART — a classic subtle bug).
- Handler code must be **async-signal-safe** (no malloc/printf/locks!) — the professional pattern is the **self-pipe trick / signalfd / eventfd**: handler just writes one byte; the event loop does the real work.

4. ASCII Diagram

```
kill -TERM 1234 --> pending mask of pid 1234: {SIGTERM}
1234 returns from a syscall -> kernel: pending & ~blocked ?
-> push signal frame on user stack -> run handler() -> sigreturn -> resume
SIGKILL path: no handler possible -> kernel terminates directly
Graceful shutdown: SIGTERM -> handler sets flag/wakes loop ->
drain requests -> exit(0); K8s: SIGTERM ... 30s ... SIGKILL
```

5. Real Production Example

Kubernetes/systemd stop sequence (SIGTERM → grace → SIGKILL) — every service you ship must drain on SIGTERM; Nginx `SIGHUP` reload / `SIGUSR1` log reopen; Go runtime turns SIGSEGV into panics and uses signals for preemption; JVM `kill -3` thread dumps.

6. Advantages

Universal, zero-dependency control channel; works on any process; kernel-enforced (SIGKILL always works — almost).

7. Trade-offs

Tiny bandwidth (a number, mostly); non-queuing loses events; handler safety is a minefield; races (signal between check and sleep) require `signalfd/pselect` patterns; not portable to Windows semantics.

8. Common Mistakes

- “kill = kill -9”: default `kill` sends SIGTERM (catchable, graceful); `-9` is the last resort that skips *all* cleanup (temp files, locks, in-flight writes).
- Believing SIGKILL kills anything: **D-state** processes and zombies don’t die (Module 1.6).
- `printf/malloc` in handlers; ignoring EINTR; forgetting SIGPIPE on socket writes (send with `MSG_NOSIGNAL` or ignore SIGPIPE).

9. Performance Implications

Signal delivery ≈ a forced user-context detour (~μs) — irrelevant at low rates; high-rate signal use (old-style timers) is an antipattern → `timerfd/eventfd`. EINTR storms under signal-heavy load surface as mysterious short reads/EINTR errors.

10–11. Interview & Follow-ups

- “SIGTERM vs SIGKILL — and design a graceful shutdown for your service.”
- “What can’t SIGKILL kill?” “What is async-signal-safety and how do real servers handle signals?” (self-pipe/`signalfd` into the event loop)

12. Coding/Debugging Scenario

Service drops requests on every deploy: no SIGTERM handler (or PID-1 default-ignore) → add handler: stop accepting, drain with deadline, exit; verify with `kill -TERM` locally.

13. Best Practices

Handle SIGTERM everywhere; handlers set flags/write `eventfd` only; `SA_RESTART` or EINTR loops around blocking calls; document a signal interface (reload, reopen, dump) for your daemons.

14. Practice Questions

1. Implement graceful shutdown for an `epoll` server via `signalfd` — sketch the loop changes.
2. Why may a process survive `kill -9`? Give both cases and the operator’s next steps.

9.7 Pipes

1. Why Interviewers Ask This

Pipes are IPC 101 plus a shell-mechanics probe (“how does `a | b` work?”) and the gateway to backpressure — a concept interviewers love to surface.

2. Core Concept

A pipe is a unidirectional in-kernel byte stream: `pipe(fds)` → `fds[0]` read end, `fds[1]` write end. Works between related processes (inherit FDs via fork). **Named pipes (FIFOs)** (`mkfifo`) get a filesystem name so unrelated processes can connect. Capacity: 64 KB default buffer.

3. Internal Working

Ring buffer in kernel memory. Semantics that matter: - Write to full pipe → **blocks** (backpressure! this is why `cmd1 | head` naturally throttles `cmd1`). - Read from empty pipe → blocks (or EAGAIN). - All write ends closed → read returns 0 (EOF) — how pipelines terminate. - All read ends closed → write gets **SIGPIPE**/EPIPE — how `head` kills upstream. - Writes ≤ **PIPE_BUF** (4 KB on Linux) are **atomic** (no interleaving between concurrent writers — key for shared log pipes). The shell builds `a | b`: `pipe()` → fork a (`dup2` write → stdout) → fork b (`dup2` read → stdin) → close unused ends everywhere (forgotten closes = the classic hang) → exec both.

4. ASCII Diagram

```
a's stdout -> [fd1 ===64KB kernel ring buffer=== fd0] -> b's stdin
full buffer -> a's write blocks (backpressure)
empty buffer -> b's read blocks
a exits (write ends closed) -> b reads EOF -> pipeline done
b exits first -> a gets SIGPIPE (default: dies) <- why `yes | head` stops
```

5. Real Production Example

Every shell pipeline and CI script; `docker logs`-style stdio plumbing; FIFOs for simple daemons' command channels; the pattern generalizes: bounded queues between microservices = pipes with the same backpressure math.

6. Advantages

Dead simple; automatic flow control (bounded buffer = built-in backpressure); composability (the Unix philosophy is literally this API); atomic small writes.

7. Trade-offs

Unidirectional (two pipes for duplex); byte stream (no message boundaries — you frame); related-process constraint (or FIFO); 64 KB buffer = deadlock fuel in naive parent-child stdout+stderr handling.

8. Common Mistakes

- The **subprocess deadlock**: parent reads child's stdout to EOF while child blocks writing a full stderr pipe (or vice versa) — both stuck. Fix: read both concurrently (threads/poll) or merge streams. Interviewers who've written CI systems *will* ask.
- Forgetting to close unused pipe ends after fork → readers never see EOF (pipeline hangs).
- Ignoring SIGPIPE in long-lived writers.

9. Performance Implications

Pipe throughput = memcpy speed (GB/s) with syscall-per-chunk overhead — size your reads/writes (64 KB chunks); `splice()` can move pipe ↔ socket/file data zero-copy. Latency ~μs (two syscalls + wakeup).

10–11. Interview & Follow-ups

- “Implement `ls | wc -l` in C.” “Why does `head` terminate the whole pipeline?” “What does PIPE_BUF atomicity give you?”
- “How is a pipe different from a socketpair / message queue?”

12. Coding/Debugging Scenario

CI step hangs forever spawning a chatty subprocess: stderr pipe full while parent reads only stdout → switch to `communicate()`-style concurrent drain; add output limits.

13. Best Practices

Close what you don't use immediately after fork; drain stdout+stderr concurrently; handle SIGPIPE/EPIPE; frame your messages or use `PIPE_BUF` records.

14. Practice Questions

1. Write two-process pipe code with all fd-closing done correctly; explain what breaks with each omitted close.
2. Design the backpressure story for a log shipper: what replaces “write blocks” when the pipe becomes a network queue?

9.8 Sockets

1. Why Interviewers Ask This

Sockets are the substrate of every backend system; interviews target the API lifecycle, TCP state subtleties (TIME_WAIT, backlog), and Unix domain sockets as the local IPC workhorse.

2. Core Concept

A socket is an FD representing a communication endpoint. Families: **AF_INET/6** (TCP/UDP over network) and **AF_UNIX** (same-host IPC through the kernel, addressed by filesystem path). Types: SOCK_STREAM (reliable byte stream), SOCK_DGRAM (messages). Server lifecycle: `socket` → `bind` → `listen` → `accept`; client: `socket` → `connect`; then `read/write`, `close`.

3. Internal Working

- `listen(fd, backlog)` creates two queues: SYN (half-open) and **accept queue** (completed handshakes waiting for `accept()`); overflowing the accept queue drops/rejects connections — the invisible outage cause (`ss -lnt` Rcv-Q vs Send-Q; `somaxconn`).
- Each connection = send/rcv kernel buffers; TCP flow control (rwnd) + congestion control decide pacing; `write` success ≠ delivered (only “queued in kernel”).
- Close choreography → **TIME_WAIT** (~60 s) on the side closing first: protects late packets; 30k ephemeral-port churn problems for clients that open/close rapidly (fix: connection pooling/keep-alive, not `tw_reuse` cargo-culting).
- **Unix domain sockets**: no protocol stack — kernel memcpy between buffers; ~2× TCP-loopback throughput, lower latency, and **FD passing** (SCM_RIGHTS) + peer credentials — the superpower TCP lacks.

4. ASCII Diagram

```

Server: socket->bind:8080->listen(backlog)      Client: socket->connect
        |   SYN q -> [accept queue] <- 3-way handshake completes
accept() <---+ (queue full? drops!)
        -> new fd per connection -> epoll loop (Module 8)
write(fd) -> [send buf] --TCP--> [recv buf] -> read(fd)
close 1st -> FIN ... -> TIME_WAIT (60s, holds the 4-tuple)
AF_UNIX: client fd ==> kernel memcpy ==> server fd (no TCP/IP at all)

```

5. Real Production Example

Everything — plus the local flavor: nginx ↔ php-fpm/gunicorn via Unix sockets; Docker daemon (`/var/run/docker.sock`); Postgres local connections; sidecar proxies. TIME_WAIT port exhaustion is a rite-of-passage incident for high-QPS client services without pooling.

6. Advantages

Uniform FD semantics (epoll works — Module 8); location transparency (swap AF_UNIX ↔ AF_INET); rich control (TCP_NODELAY, SO_REUSEPORT, keepalive).

7. Trade-offs

Byte streams need framing (length-prefix/delimiters — “TCP is not a message protocol” trips juniors); lifecycle subtleties (half-close, RST vs FIN, CLOSE_WAIT leaks); buffers hide failure (write succeeded, peer dead).

8. Common Mistakes

- Assuming one `send` = one `recv` (fragmentation/coalescing — always frame).
- CLOSE_WAIT piles = *your* app didn’t close after peer’s FIN (FD leak signature, Module 7.6).
- Setting huge backlog but forgetting `net.core.somaxconn` caps it silently.
- Not knowing why Unix sockets beat localhost TCP (no stack, bigger effective copies, FD passing).

9. Performance Implications

Loopback TCP ~ GB/s with syscall overhead; AF_UNIX ~2× that; TCP_NODELAY vs Nagle for latency-sensitive small writes (40 ms delayed-ACK interactions — the classic p99 mystery); SO_REUSEPORT to spread accepts across workers.

10–11. Interview & Follow-ups

- “Walk through socket → accept → epoll for a web server.” “What is the backlog really?” “Why is TIME_WAIT there and when does it hurt?” “TCP vs Unix socket for two services on one host?”

12. Coding/Debugging Scenario

Load spikes cause silent connection failures though the app looks healthy: `ss -lnt` shows accept-queue overflow → raise backlog+somaxconn, accept faster (more workers), add SYN cookies check.

13. Best Practices

Frame messages explicitly; pool client connections; monitor TIME_WAIT/CLOSE_WAIT counts and accept-queue drops; prefer AF_UNIX for same-host RPC; tune NODELAY consciously.

14. Practice Questions

1. Implement a length-prefixed echo protocol over TCP — handle partial reads across frames.
2. A client fleet exhausts ephemeral ports at 50k conn/s churn — give three fixes ranked (pooling, keep-alive, more source IPs/tw tuning).
3. Demonstrate FD passing over a Unix socket — what does it enable architecturally? (zero-downtime listener handoff)

9.9 Shared Memory

1. Why Interviewers Ask This

The fastest IPC — and the one that reintroduces every synchronization problem from Module 3. Interviewers use it to test the “speed vs safety” IPC trade-off and modern usage (ring buffers, /dev/shm).

2. Core Concept

Two+ processes map the **same physical pages** into their address spaces: after setup, communication = memory access — zero syscalls, zero copies. APIs: POSIX `shm_open` + `mmap` (a file in tmpfs `/dev/shm`), `mmap(MAP_SHARED)` on a real file, System V `shmget/shmat` (legacy), `memfd_create` (anonymous, FD-passable).

3. Internal Working

`shm_open` creates a tmpfs object; `ftruncate` sizes it; each process `mmap`s it `MAP_SHARED` → page tables of both processes point at the same frames (Module 5 machinery). No kernel involvement per access — which means **no built-in synchronization**: you bring your own (process-shared mutexes `PTHREAD_PROCESS_SHARED`, unnamed semaphores in the segment, or lock-free rings on atomics). Robust mutexes (EOWNERDEAD) matter here: a crashed peer can die holding the lock.

4. ASCII Diagram

```

Process A page tables \          / Process B page tables
  vaddr 0x7f00... <----> [same physical frames] <----> vaddr 0x7f80...
                        /dev/shm/myseg (tmpfs)
Pattern: SPSC ring buffer in the segment:
[ head | tail | slots..... ] producer CAS/store tail, consumer head
Sync options: process-shared mutex+condvar | semaphores | lock-free atomics
Crash risk: peer dies holding lock -> robust mutex / sequence validation

```

5. Real Production Example

- Postgres shared buffers (all backends attach one segment); Chrome renderer ↔ GPU transport; aeron/LMAX-style shared-memory rings for trading (~100 ns latency); ML: model weights in /dev/shm shared across worker processes (PyTorch dataloaders); `/dev/shm` as scratch (it's RAM — counts against memory!).

6. Advantages

Fastest possible IPC (memory speed, no per-message syscalls/copies); natural for large objects (map once, share forever); works with lock-free structures for μs-to-ns messaging.

7. Trade-offs

You own all synchronization and memory layout (versioning! both sides must agree on struct layout/ABI); crash of one peer can leave corrupt state/held locks (need robust mutexes, sequence numbers, or restart-the-world); no message boundaries/flow control unless you build them; security = filesystem perms on the segment.

8. Common Mistakes

- Proposing shared memory without a synchronization story (instant Module-3 follow-up).
- Storing pointers in the segment (each process maps at different addresses — use offsets!).
- Forgetting /dev/shm consumes RAM and default-caps at 50% (tmpfs) — “disk full” on /dev/shm = memory issue.
- Ignoring cleanup: POSIX segments persist until `shm_unlink` even after all processes exit.

9. Performance Implications

Throughput = memory bandwidth; latency = cache-coherence (~100 ns cross-core) — 10–100× faster than sockets for large payloads. But false sharing/contention (Module 3.9) applies fully; NUMA placement of the segment matters on big boxes.

10–11. Interview & Follow-ups

- “Rank IPC mechanisms by speed and by safety; when is shared memory worth it?”
- “How do two processes synchronize access to a shared segment?” (process-shared mutex/robust; semaphores; lock-free) “What happens when one crashes mid-update?”

12. Coding/Debugging Scenario

Two services exchange 50 MB frames over localhost TCP at 30 fps → CPU burned on copies; move frames to a shared-memory ring + eventfd doorbell; CPU drops, latency 10×better. (The doorbell pattern — shm for data, fd for wakeup — is the pro answer.)

13. Best Practices

Use offsets not pointers; version your layout; pair shm (data plane) with a pipe/eventfd/socket (control plane + liveness); robust mutexes or lock-free with sequence checks; clean up segments on start (recover from crashes).

14. Practice Questions

1. Design a SPSC shared-memory ring buffer: layout, atomics/ordering, wrap handling, peer-crash recovery.
2. Compare passing a 1 GB dataset between processes via: pipe, Unix socket, TCP, shm, memfd+FD-passing — copies and syscalls for each.

Module 9 Cheat Sheet (one page)

Process lifecycle: fork (returns twice: child-pid/0; COW; FD-offset sharing) → exec (replace image, keep PID+FDs, never returns) → exit → **zombie** (dead, unreaped; PCB only; kill-proof) → wait (reap; WNOHANG loop for SIGCHLD) | parent dies → **orphan** (alive; adopted by init/subreaper). **Containers:** your app = PID 1 → must reap + handle SIGTERM (or use tini / `docker run --init`).

Signals: TERM catchable-graceful vs KILL/STOP uncatchable; D-state & zombies survive -9; handlers = async-signal-safe only (self-pipe/signalfd pattern); standard signals don't queue; EINTR; SIGPIPE on dead-peer writes.

IPC	Speed	Sync built-in	Boundaries	Killer feature / trap
Pipe/FIFO	GB/s, μ s	blocking = backpressure	byte stream	PIPE_BUF atomic $\leq 4K$; stdout/stderr drain deadlock
Unix socket	$\sim 2\times$ loopback TCP	flow control	stream or dgram	FD passing (SCM_RIGHTS), peer creds
TCP socket	network	flow+congestion ctl	byte stream (frame it!)	backlog/accept-queue; TIME_WAIT; CLOSE_WAIT leaks
Shared memory	memory speed	none — BYO	none	offsets not pointers; robust mutex; /dev/shm = RAM

Shell mechanics: `a | b > f` = pipe + fork $\times 2$ + dup2 + close-unused + exec; `head` kills upstream via SIGPIPE; EOF when all write ends close.

Top Interview Questions

1. fork return values; how many processes does this loop create; fork+threads hazard.
2. How does the shell implement pipelines and redirection? (write the C)
3. Zombie vs orphan — definitions, causes, fixes, container PID-1 angle.
4. SIGTERM vs SIGKILL; design graceful shutdown; what survives kill -9.
5. Why won't my container stop / why is it full of `<defunct>`? (tini)
6. TCP accept queue/backlog; TIME_WAIT vs CLOSE_WAIT diagnosis.
7. Rank IPC options for a given workload; shared memory + its sync story.

Common Mistakes (module-wide)

- Fork-count errors; expecting code after exec to run; unflushed stdio double-printing.
- One-reap-per-SIGCHLD; kill -9 as zombie cure; printf in signal handlers.
- Unclosed pipe ends (no EOF); single-stream subprocess draining (deadlock).
- Treating TCP as message-oriented; ignoring EINTR/SIGPIPE.
- Shared memory with raw pointers or no crash-recovery plan.

Mock Interview (self-test, ~25 min)

1. (Code) Implement `ps aux | grep java | wc -l` in C — full fd choreography, waits, and error paths.
2. (Prod) Host has 28k `<defunct>` processes, forks failing. Diagnose (find common PPID), remediate now, prevent forever (reaper patterns, tini).
3. (Design) Two same-host services must exchange 100k msgs/s of 200 B and occasionally 500 MB blobs. Pick IPC per path (socket control plane + shm data plane) and defend the sync/crash story.

4. (Depth) `kill -TERM <pid>` — trace everything from your shell to the target's handler running, including masks, pending sets, and delivery points.
5. (Trap) “Our app is PID 1 in the container; SIGTERM works in dev (not containerized) but the pod always takes 30 s to die.” Explain precisely and fix two ways.

MODULE 10 — Production Interview Scenarios

Senior SWE Interview Track — Operating Systems

These are the “you’re on call, the pager fires” questions that senior interviews at Google/Meta/Amazon/Uber/Netflix use to separate practitioners from textbook readers. Each scenario follows the same skeleton: **triage commands** → **hypothesis tree** → **root causes** → **fixes** → **prevention**. Interview scoring rewards *ordered, evidence-driven* investigation — always say what you’d measure before what you’d change.

The universal first minute (USE-style sweep)

```
uptime; dmesg -T | tail          # load, recent kernel events (OOM! I/O errors!)
top / htop                      # who eats CPU/mem; load vs cores; %us/%sy/%wa
vmstat 1                        # r (runnable), b (blocked), cs, si/so (SWAP!)
iostat -x 1 / sar -B            # disk util/latency, page faults
ss -s; ls /proc/<pid>/fd | wc -l  # sockets, fds
```

10.1 High CPU Usage

Why Interviewers Ask This

The most common incident; they watch whether you split user vs system vs steal time before touching code.

Triage Path

1. **top**: which process; then **top -H -p <pid>**: which **threads**.
2. Split the CPU: **%us** (your code) / **%sy** (kernel — syscalls, spinning in futex?) / **%wa** (not CPU! I/O wait) / **%si** (softirq — network) / **%st** (steal — noisy VM neighbor or cgroup throttle).
3. Profile: **perf top** / **perf record -g** (C/C++/Go), async-profiler/JFR (JVM), py-spy (Python). Flame graph the hot stacks.
4. Container check: **throttled_time** in **cpu.stat** — CPU *limit throttling* masquerades as slowness without high host CPU.

Root-Cause Menu (map to modules)

- Busy loop / accidental polling (M8): thread at 100% with trivial stack.
- Lock contention burning **%sy** in futex or spinning (M3).
- GC storms (allocation rate — M5); regex/serialization hot spots.
- Too many runnable threads → context-switch overhead (**vmstat cs**, **pidstat -w**) (M1/M2).
- Interrupt/softirq saturation on CPU0 (M8.4).
- **Livelock**: 100% CPU, zero goodput, retry counters spiking (M4.7).

ASCII Decision Sketch

```
CPU high ->
%st high -> neighbor/quota (move host; fix limits)
%si high -> IRQ/softirq (affinity, RSS, coalescing)
%sy high -> syscalls/locks (strace -c, perf; futex? spinlock?)
%us high -> perf/flame graph -> hot function -> fix code
none high but app slow in container -> cgroup throttling (cpu.stat)
```

Interview Traps

- Load average ≠ CPU usage (includes D-state — M1.6).
- 100% of one core on an 8-core box = 12.5% total — single-thread bottleneck, different fix (parallelize) than global saturation (scale/optimize).
- CPU high *and* healthy could be fine — tie to SLO impact before “fixing”.

Practice

“API p99 doubled; host CPU 85% (was 40%) after a deploy. Walk me through to the exact function.” (diff deploys, flame graph before/after, find the new hot frame, e.g., accidental $O(n^2)$ or debug logging in a loop)

10.2 High Memory Usage

Why Interviewers Ask This

Tests RSS/VSZ/cache literacy (M5) — most candidates misread `free` and “fix” non-problems.

Triage Path

1. `free -h`: is it *really* used, or page cache? (**available** is the number that matters; cache is reclaimable — M7.1.)
2. `top` sorted by %MEM / `ps aux --sort=-rss`; `smem`/PSS if shared memory muddies it (M5.1).
3. Per-process anatomy: `/proc/<pid>/status` (VmRSS breakdown), `pmap -x` — heap? mmapped? thread stacks? / dev/shm (M9.9)?
4. Container: cgroup `memory.current` vs limit; page cache counts *inside* the cgroup → “OOM” with cache-heavy workloads.
5. Trend: is it a step (new workload) or a slope (leak → 10.3)?

Root-Cause Menu

- Page cache (not a problem — don’t `drop_caches` as a “fix” in interviews!).
- Legit working-set growth (traffic, bigger heap) → capacity.
- Fragmentation / freed-not-returned allocator memory (M5.9).
- tmpfs/dev/shm data counting as RAM (M9.9); huge thread counts × stacks (M1.3).
- Leaks (next scenario). Swap thrash: `vmstat` si/so > 0 sustained = the real emergency (M6.6).

Interview Traps

- “Linux ate my RAM” (cache) — say it before they bait you.
- OOM killer forensics: `dmesg` shows the score/victim; killed process ≠ guilty process (biggest ≠ leaker).
- Sum of per-process RSS > physical RAM is normal (shared pages).

Practice

“Host shows 95% memory used; nothing looks wrong. Is there a problem?” (walk free/available, cache, pressure via `sar -B/PSI some avg10` in `/proc/pressure/memory` — modern answer)

10.3 Memory Leaks

Why Interviewers Ask This

Multi-layer debugging skill: language runtime → allocator → OS, with different tools per layer.

Triage Path

1. Confirm: RSS slope over days (metrics), not a one-step jump. Restarts “fix” it → leak-shaped.
2. Identify the layer: - **Managed heap** (JVM/Go/Python): heap profiler/dump — growing object graph? (jmap+MAT, pprof heap, tracemalloc). Watch for *logical* leaks: unbounded caches, listener registries, ThreadLocals in pools. - **Native/allocator**: heap flat, RSS grows → native leak (JNI, cgo, C libs) or fragmentation (M5.9): jemalloc profiling / valgrind massif / heaptrack in staging. - **Not memory at all**: FDs/mmapped/threads also inflate — check `/proc/<pid>/fd`, `pmap` diff, thread count.
3. Kernel-side: slab leaks (`slabtop`) if system memory vanishes with no process owning it.

ASCII Decision Sketch

```
RSS climbs forever:
  managed-heap climbs too -> object leak -> heap dump diff (2 snapshots!)
  managed-heap flat       -> native: pmap diff / jemalloc prof / valgrind
  heap tools clean        -> fragmentation (allocator stats) or
                           mmap/fd/thread growth
Golden technique: TWO snapshots, diff, biggest delta wins.
```

Interview Traps

- GC exists ≠ no leaks (reachable-but-useless is the definition of a managed leak).
- Fragmentation vs leak (M5.9) — allocator stats decide.
- The “leak” that’s a cache with no eviction (M6!) — bounded caches are the fix.

Practice

“Java service OOMs every ~4 days. Design the investigation end-to-end and name the tool at each step.” (GC logs → heap dump on OOM flag → MAT dominator tree → e.g., a static Map of sessions never evicted)

10.4 Thread Starvation

Why Interviewers Ask This

The classic “service is slow but CPU is idle” paradox — tests thread-pool math (M1/M2) and dump-reading (M1.7).

Triage Path

1. Symptom check: latency up, CPU low, queue depths up.
2. Thread dumps ×3, 10 s apart (jstack / Go SIGQUIT / py-spy dump): histogram the states.
3. Patterns: - All workers **WAITING on the same external call** (DB pool empty, slow downstream) → pool exhaustion cascade. - Workers **BLOCKED** on one lock → contention (10.6) not starvation. - Pool threads waiting on results of tasks queued in *the same pool* → **thread-pool deadlock/starvation** (M4.1 practice) — the single most-loved interview variant (async code doing blocking `.get()` on the common pool). - Low-priority/fairness starvation (M2.8, M4.8): one class of work never scheduled.

Fix Ladder

Separate pools per dependency (bulkheads); size pools = cores × (1 + wait/compute); timeouts + circuit breakers on all blocking calls; never block inside async executors; fairness modes for starved lock waiters (Go mutex starvation mode — M4.8).

Interview Traps

- “Add more threads” without the math — more threads with a saturated downstream just queues harm (and adds switch overhead — M1.5).
- Starvation vs deadlock vs slow dependency: dumps distinguish them; say how.

Practice

“Tomcat: 200/200 threads busy, CPU 10%, DB pool 20 connections at 100% utilization, DB p99 2 s. Explain the cascade and fix at every layer.” (Little’s law on the pool, timeouts, pool sizing, DB query fix)

10.5 Deadlocks (in production)

Why Interviewers Ask This

M4 theory turned operational: can you *find* one at 3 a.m.?

Triage Path

1. Smell: a subset of requests hang forever (no timeout errors — just stuck); thread count stable; specific endpoints frozen.
2. In-process: thread dump — JVM literally prints “Found one Java-level deadlock” with the cycle; Go: goroutine dump shows mutual channel/mutex waits; C++: gdb, inspect lock owners.
3. Database: `SHOW ENGINE INNODB STATUS` (latest deadlock), `pg_locks` joined with `pg_stat_activity`; error 1213/40P01 rates in app logs.
4. Distributed: no global detector — look for lock-timeout logs, stuck sagas, two services each holding a row the other wants.

Fix Ladder

Immediate: restart (in-process) / kill victim TX (DB). Real fix: lock ordering (M4.2), shrink transactions, single-lock-then-call-out design, jittered retry on DB deadlock errors (M4.6). Prevention: lock-order assertions in debug builds, CI deadlock tests with contention harnesses, alert on deadlock counters.

Interview Traps

- Distinguish “deadlocked” from “waiting on a dead downstream forever” (missing timeout) — dumps show *what* it waits on.
- DB deadlocks are *normal at low rate*; the bug is missing retries or hot-row design (say both).
- jstack shows the cycle but you still must map monitors → code sites → ordering fix.

Practice

“Every few days, 3 endpoints freeze until restart; dumps attached showing T1 holds A wants B, T2 holds B wants A across two @Service classes. Give the immediate action, the code fix, and the CI guard.”

10.6 Lock Contention

Why Interviewers Ask This

The subtler cousin of deadlock: everything *works*, just slowly — tests M3 cost models and scalability instincts.

Triage Path

1. Signature: throughput plateaus (or *drops*) as load/cores rise; CPU has headroom; p99 >> p50.
2. Measure, don't guess: JFR/async-profiler lock profiles (which monitor, how long), `perf lock record`, Go mutex profile (`runtime.SetMutexProfileFraction`), `%sy` + futex time in perf.
3. Identify the hot lock and its **hold time × acquisition rate** (utilization of the lock — M3.2's queueing math).

Fix Ladder (in order of preference)

1. **Shrink the critical section** (move I/O/alloc/logging out — M3.2).
2. **Shard/strip** the lock (per-bucket locks, `ConcurrentHashMap`-style; per-core counters for stats — M3.9).
3. Change structure: read-mostly → RWLock or better snapshot-swap/RCU (M3.6); queues → lock-free/MPSC (M3.10).
4. Reduce arrival rate: batch under one acquisition; thread-local buffers flushed periodically.

Interview Traps

- RW lock as a reflex “fix” can be slower (M3.6); contended atomic counters still serialize (M3.9 false sharing/ping-pong).
- Adding threads to a contended system *reduces* throughput (convoying, switch overhead) — say the negative-scaling curve out loud.
- Amdahl: 5% serialized caps you at 20× — quantify before micro-optimizing.

Practice

“A metrics library’s global histogram mutex is 40% of profile samples at peak. Design the fix (striped/per-CPU + periodic merge) and estimate the improvement.”

10.7 Too Many Open Files

Why Interviewers Ask This

Beloved because it chains config (ulimits), code (leaks), and protocol (CLOSE_WAIT) knowledge — M7.6 + M9.8 in one incident.

Triage Path

1. Error: **EMFILE** (per-process limit) vs **ENFILE** (system `fs.file-max`) — different knobs.
2. Count and classify: `ls /proc/<pid>/fd | wc -l`; `lsof -p <pid>` → sockets? files? pipes? anon inodes (epoll/eventfd)?
3. Sockets dominating: `ss -tan | awk '{print $1}' | sort | uniq -c`: - **CLOSE_WAIT** pile = *your code* never closes after peer FIN → FD leak, fix close paths. - **TIME_WAIT** pile = churning short connections (usually fine; ports, not FDs — but signals missing keep-alive/pooling).
4. Check the limit story: soft vs hard `ulimit -n`, systemd **LimitNOFILE**, container defaults — 1024 default vs 10k+ connection reality.

Fix Ladder

Immediate: raise soft limit (buy time), restart the leaker. Real: find the leak (error paths missing close — use RAII/defer/try-with-resources), add HTTP client pooling/keep-alive, close on all exception branches. Prevention: alert at 70% of `fd_used/fd_limit`; load tests that assert FD count returns to baseline.

Interview Traps

- Raising the limit is mitigation, not the fix — say it explicitly.
- Everything is an FD: epoll instances, timerfd, inotify watches count too (M7.6).
- CLOSE_WAIT vs TIME_WAIT direction (who owes the close) is a favorite rapid-fire check.

Practice

“Proxy dies nightly with EMFILE at ~1024 FDs; lsof shows 900 CLOSE_WAIT to one upstream. Reconstruct the bug (upstream closes idle conns; proxy ignores EOF) and fix at code + config + alerting levels.”

10.8 Process Crashes

Why Interviewers Ask This

Postmortem methodology under pressure: signals, cores, OOM forensics — M9.6 + M5 applied.

Triage Path

1. **How did it die?** Exit code decodes first: 128+N = killed by signal N (137 = SIGKILL → almost always **OOM killer** or **K8s limit**; 139 = SIGSEGV; 134 = SIGABRT/assert; 143 = SIGTERM — someone asked politely).
2. External evidence: `dmesg -T | grep -i oom` (killer logs victim + scores), `journalctl -u svc`, K8s **OOMKilled** status/events.
3. SIGSEGV path: enable core dumps (`ulimit -c`, `kernel.core_pattern`, `coredumpctl`) → `gdb bin core` → `bt` → faulting address analysis (NULL? stack guard = overflow (M5.10)? use-after-free — ASan in staging).
4. Crash *loop* vs one-off: supervisor backoff, crash on specific input (poison message replay!), or resource-based (OOM every N hours = leak → 10.3).

ASCII Decision Sketch

```
exit code:
137 (SIGKILL) -> dmesg OOM? cgroup limit? someone's kill -9?
139 (SIGSEGV) -> core dump -> gdb bt -> null/UAF/stack overflow
134 (SIGABRT) -> assert/fatal exception/allocator abort -> logs
143 (SIGTERM) -> orchestrator/deploy killed it (probe failures?)
0/1..n -> app-level exit -> logs
K8s: OOMKilled | CrashLoopBackOff (check probes!) | Evicted (node pressure)
```

Interview Traps

- OOM-killed ≠ has a leak (limit too low, cache growth, COW spike at fork — M5.8 Redis story).
- K8s liveness probe misconfig masquerades as “crashes” (SIGTERM from kubelet).
- No core dumps configured = flying blind — mention enabling them *before* the next crash.

Practice

“Container restarts every ~6 h, exit code 137, no app error logs. Full investigation, and three different root-cause families you must rule out (leak, limit-vs-working-set, COW/fork spike).”

10.9 Bonus rapid-fire scenarios (one-liners you should be able to expand)

- **Load average 60, CPU idle** → D-state pileup on NFS/disk (M1.6): `ps ... | awk '$8~/D/'`, `/proc/<pid>/stack`, fix storage.
- **Server slow after backup job** → page-cache eviction by sequential scan (M7.8): `fsync/O_DIRECT/ionice` the backup.
- **p99 spikes every 30 s, exactly** → dirty-page writeback flushes / journal commits (M7.3/7.8): tune `vm.dirty_*`, split WAL device.
- **First request after deploy slow** → demand paging cold start (M5.7): warmup, pre-touch, `mlock`.
- **Service can't be killed** → D-state (M1.6) or zombie confusion (M9.4): identify state before reaching for -9.
- **Sudden EADDRINUSE on restart** → no `SO_REUSEADDR` + `TIME_WAIT` (M9.8), or leaked child holding the listener (M9.2 CLOEXEC).
- **Box slows, si/so nonzero, disk busy** → swap thrash (M6.6): find the memory hog; do NOT add swap.
- **One CPU 100% softirq** → IRQ affinity/RSS (M8.4).

Module 10 Cheat Sheet (one page)

Exit codes: 137=SIGKILL(OOM?) · 139=SIGSEGV · 134=SIGABRT · 143=SIGTERM. **dmesg first** for OOM/hardware. **State reading:** R=CPU · D=disk/NFS (load lies!) · Z=unreaped · CLOSE_WAIT=you leak · TIME_WAIT=churn. **CPU split:** us=code · sy=kernel/locks · wa=disk(not CPU) · si=network · st=neighbor/quota. **Memory truths:** available>free matters · cache is reclaimable · RSS≠VSZ · PSI /proc/pressure/ is the modern signal. **Tool belt*:** `top/-H`, `vmstat 1`, `pidstat -w`, `iostat -x`, `sar -B`, `ss -s/-tan`, `lsof`, `/proc/pid/{fd,status,stack}`, `perf record -g/` flamegraphs, `strace -c`, `jstack/pprof/py-spy`, `dmesg`, `PSI`.

Symptom	First command	Likely module
Slow + CPU idle	thread dumps ×3	M1.7/M3 (waits/locks)
Slow + CPU busy	perf/flame graph	M2/M3 (code/locks)
Load high CPU idle	ps state D scan	M1.6 (I/O)
RSS slope	2-snapshot heap diff	M5 (leak/frag)
Throughput drops w/ more threads	lock profile	M3 (contention)
Frozen subset of endpoints	dump → cycle	M4 (deadlock)
EMFILE	/proc/fd count + ss states	M7.6/M9.8
Restart loop 137	dmesg + cgroup limits	M5/M10.8

Top Interview Questions

1. “Service is slow” — full structured triage (they grade the *order*).
2. Load 80, CPU 10% — explain and fix.
3. OOM-killed nightly — investigate without guessing.
4. All 200 worker threads busy, CPU idle — the pool-exhaustion cascade.
5. Throughput fell when we doubled threads — negative scaling story.
6. EMFILE at midnight — CLOSE_WAIT forensics.
7. Exit code 137 vs 139 vs 143 — what each tells you.

Common Mistakes (module-wide)

- Jumping to fixes before evidence; restarting away the diagnosis (say: capture dumps/cores *then* restart).
- Misreading load average, free memory, and %wa.
- “Add threads/RAM/swap” as reflexes (often each makes it worse).
- Not knowing the two-snapshot diff technique for leaks.
- Forgetting cgroup/container limits behave differently from host limits.

Mock Interview (self-test, ~30 min — the final boss)

1. Pager: “checkout p99 5 s (was 200 ms), started 14:02.” You have one SSH session. Narrate your first 10 commands and what each rules in/out.
2. `vmstat 1` shows: `r=1, b=14, si/so=0, wa=60%`. Three hypotheses, one discriminating test each.
3. A Go service: CPU 100%, goroutine dump shows thousands in `runtime.gopark` on one channel and one goroutine in a tight `select` retry loop. Deadlock, livelock, or starvation? Fix?
4. K8s pod: OOMKilled at 2 GB limit; jmap heap = 800 MB steady. Enumerate where the other 1.2 GB can live (metaspace, threads×stacks, direct buffers, mmap, glibc arenas, page cache in cgroup) and how to measure each.
5. Postmortem: write the 5-line timeline + root cause + 3 preventions for: FD leak (CLOSE_WAIT) → EMFILE → accept() fails → LB marks node down → cascade. What single alert would have caught it a week earlier?